Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

División de Ciencias de la Ingeniería

Ingeniería en Ciencias y Sistemas

Sistemas de Bases de Datos 1

Proyecto 2

Base de Datos Hospitalaria: Consultas Avanzadas, Lógica del Negocio y Seguridad

Estudiante	Juan Carlos Rodríguez López
Carné	202231970
Fecha	09/11/2026

Contenido

1. Descripción general **4**

Resumen del esquema final . . . 4

Decisiones principales . . . 4

2. Modelo de datos extendido (notación Barker) **5**

2.1 Estrategia de mapeo del supertipo y relación recursiva . . . 10

3. Diccionario de datos de las tablas nuevas y modificadas **11**

Tabla personal (nueva) . . . 11

Tabla auditoria_personal (nueva) . . . 12

Tabla clinica (nueva) . . . 13

Tabla orden_laboratorio (nueva) . . . 13

Tabla orden_examen (nueva) . . . 14

Tabla examen_laboratorio (nueva) . . . 14

Tabla forma_farmaceutica (nueva) . . . 15

Tabla funcion_instrumento (nueva) . . . 15

Tabla funcion_equipo (nueva) . . . 15

Tabla clasificacion_asa (nueva) . . . 16

Tabla tipo_medico (nueva) . . . 16

Tabla medio_asignacion (nueva) . . . 16

Tabla medico (modificada) . . . 17

Tabla enfermero (modificada) . . . 19

Tabla cita (modificada) . . . 20

Tabla medicamento (modificada) . . . 21

Tabla instrumento (modificada) . . . 21

Tabla equipo (modificada) . . . 22

Tabla chequeo_preanestesico (modificada) . . . 23

4. Objetivos **24**

Bloque A — Consultas de inteligencia de negocio . . . 24

Bloque B — Extensión del modelo . . . 43

Bloque C — Reportes sobre el personal médico . . . 44

Bloque D — Programa de fidelidad . . . 48

Bloque E — Revisión del reporte de cirugías . . . 50

Bloque F — Trazabilidad de cambios sensibles . . . 51

5. ORM: objetivos 8, 11 y 18 con JPA **52**

Igualdad de resultados . . . 52

Consultas JPQL (ConsultasOrm.java) . . . 52

Sentencia preparada enviada por Hibernate (objetivo 8) . . . 54

Anexo A. Diccionario de datos completo **56**

Anexo B. Sentencias SQL del Proyecto 2 **80**

1. Descripción general

El Proyecto 2 extiende la misma base de datos del Proyecto 1 (mismo esquema y datos, ampliados). Se resolvieron los 20 objetivos del enunciado organizados en seis bloques: consultas de inteligencia de negocio (A), extensión del modelo (B), reportes del personal médico (C), programa de fidelidad (D), revisión del reporte de cirugías (E) y trazabilidad de cambios sensibles (F).

Resumen del esquema final

Elemento	Cantidad
Tablas	64
Tablas nuevas del Proyecto 2	personal, auditoria_personal, clinica, orden_laboratorio, orden_examen, examen_laboratorio, forma_farmaceutica, funcion_instrumento, funcion_equipo, clasificacion_asa, tipo_medico, medio_asignacion
Tablas modificadas por el Proyecto 2	medico, enfermero, cita, medicamento, instrumento, equipo, chequeo_preanestesico
Llaves foráneas	109
Restricciones CHECK	95
Disparadores	2
Consultas	Una sentencia SELECT por objetivo en sql/consultas/objetivo_XX.sql

Decisiones principales

• El rol administrativo (**admin**) es dueño de las tablas desde el DCL del Proyecto 1, porque en PostgreSQL solo el dueño de una tabla puede alterarla; así la extensión del Bloque B y la auditoría del Bloque F las ejecuta admin, como pide el enunciado. Las consultas se ejecutan con el rol de solo lectura **visor**.

• Cada objetivo es una sentencia SELECT en su propio archivo (sql/consultas/objetivo_XX.sql), con el lapso de tiempo (enero a octubre de 2026) escrito en el WHERE; se ejecutan con el rol visor.

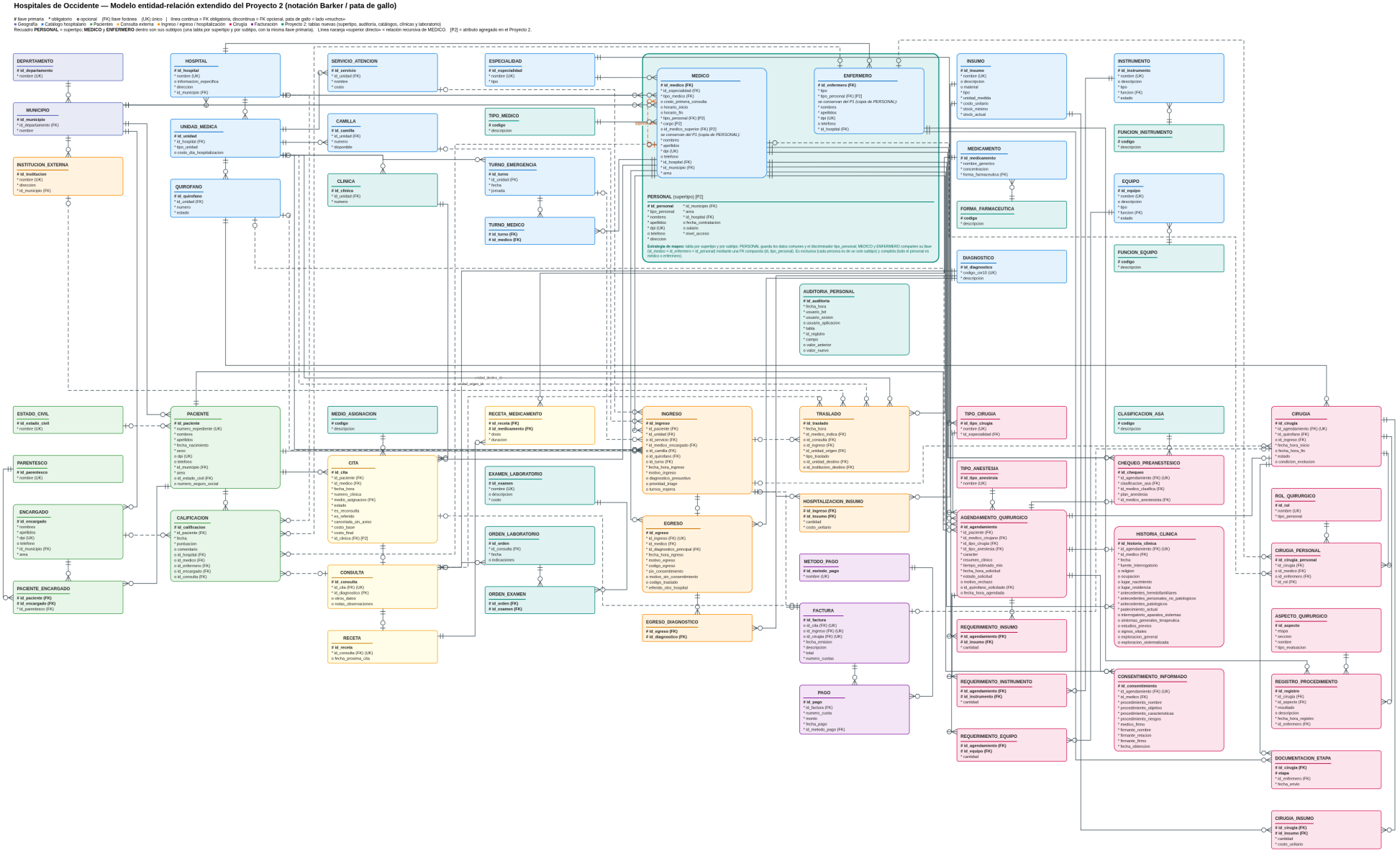
• La extensión es una migración no destructiva: no se borra ninguna columna ni tabla del Proyecto 1.

• Se agregaron dos hospitales y datos de enero a octubre de 2026 generados de forma reproducible (siempre producen los mismos valores), con 15 o más registros en cada tabla relevante.

• Para los objetivos con criterios ambiguos se fijó una métrica explícita, descrita junto a cada objetivo.

2. Modelo de datos extendido (notación Barker)

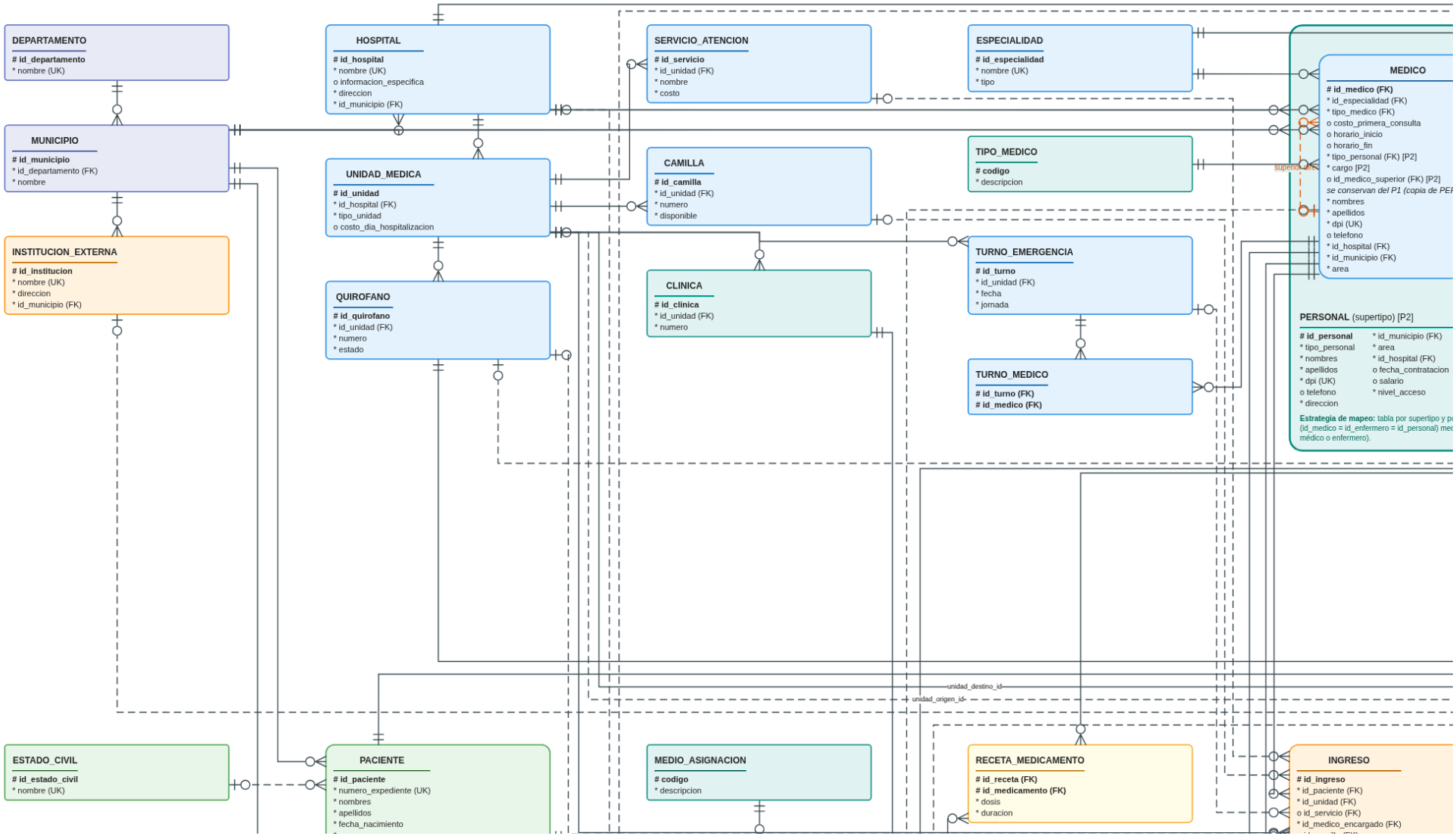
Archivo editable: modelo/modelo_barker_p2.drawio (draw.io). El recuadro **PERSONAL** es el supertipo y contiene a sus subtipos **MEDICO** y **ENFERMERO**; la línea naranja «superior directo» es la relación recursiva de MEDICO; [P2] marca los atributos agregados en este proyecto.



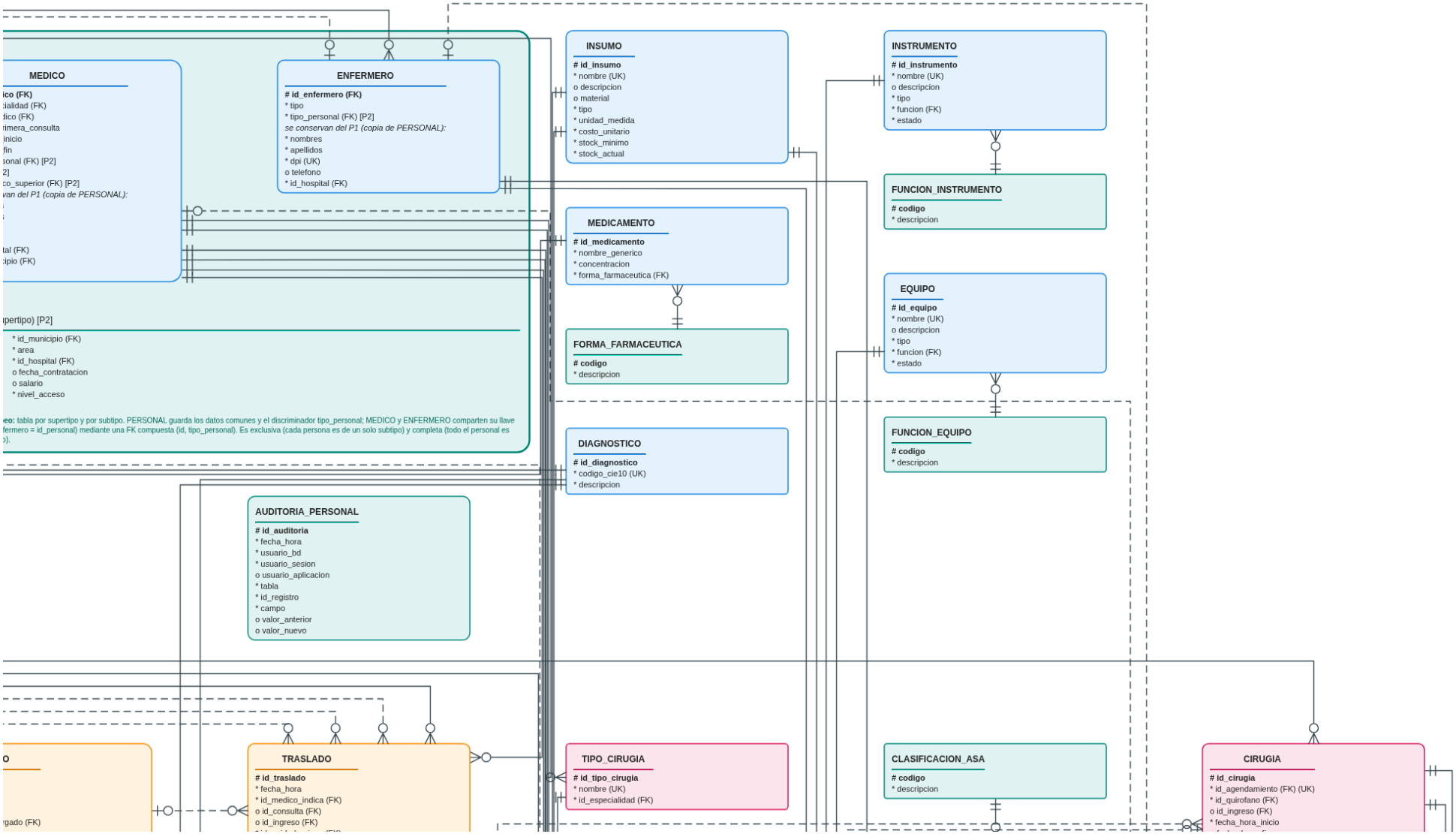
Modelo extendido — cuadrante superior izquierdo

Hospitales de Occidente — Modelo entidad-relación extendido del Proyecto 2 (notación Barker / pata de gallo)

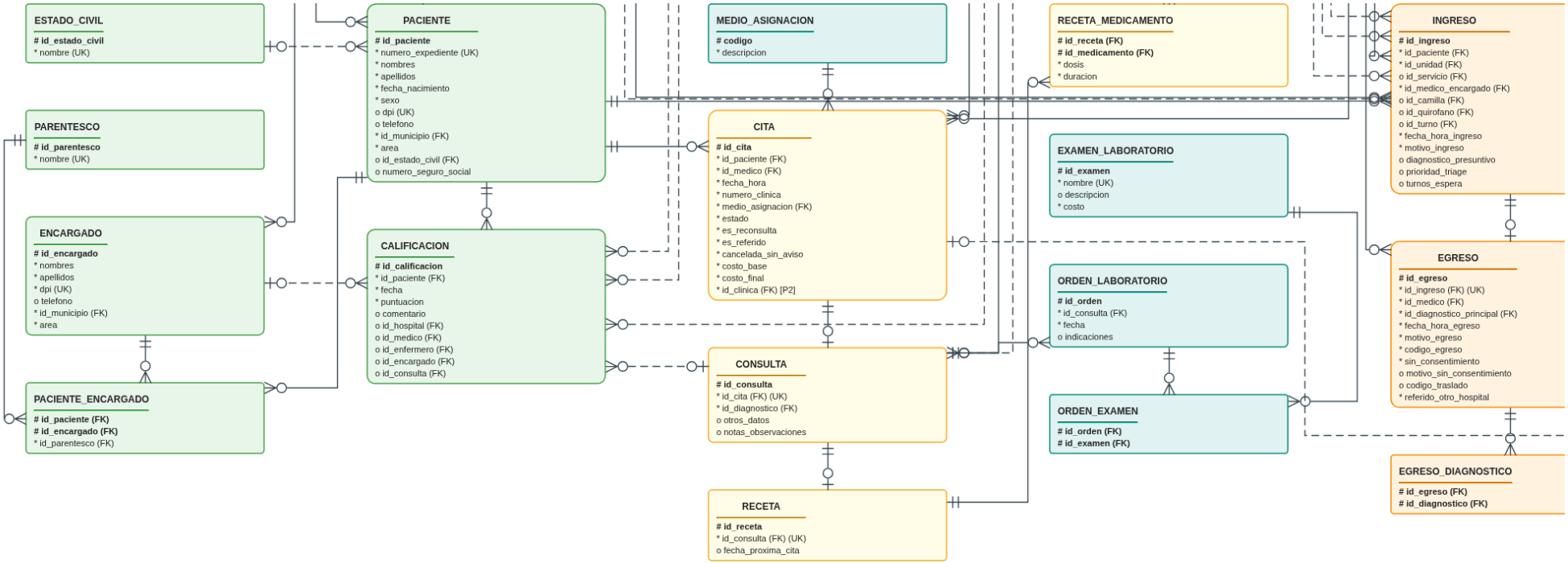
llave primaria * obligatorio o opcional (FK) llave foránea (UK) único | línea continua = FK obligatoria, discontinua = FK opcional, pata de gallo = lado «muchos»
■ Geografía ■ Catálogo hospitalario ■ Pacientes ■ Consulta externa ■ Ingreso / egreso / hospitalización ■ Cirugía ■ Facturación ■ Proyecto 2: tablas nuevas (supertipo, auditoría, catálogos, clínicas y laboratorio)
Recuadro **PERSONAL** = supertipo; **MEDICO** y **ENFERMERO** dentro son sus subtipos (una tabla por supertipo y por subtipo, con la misma llave primaria). Línea naranja «superior directo» = relación recursiva de MEDICO. [P2] = atributo agregado en el Proyecto 2.



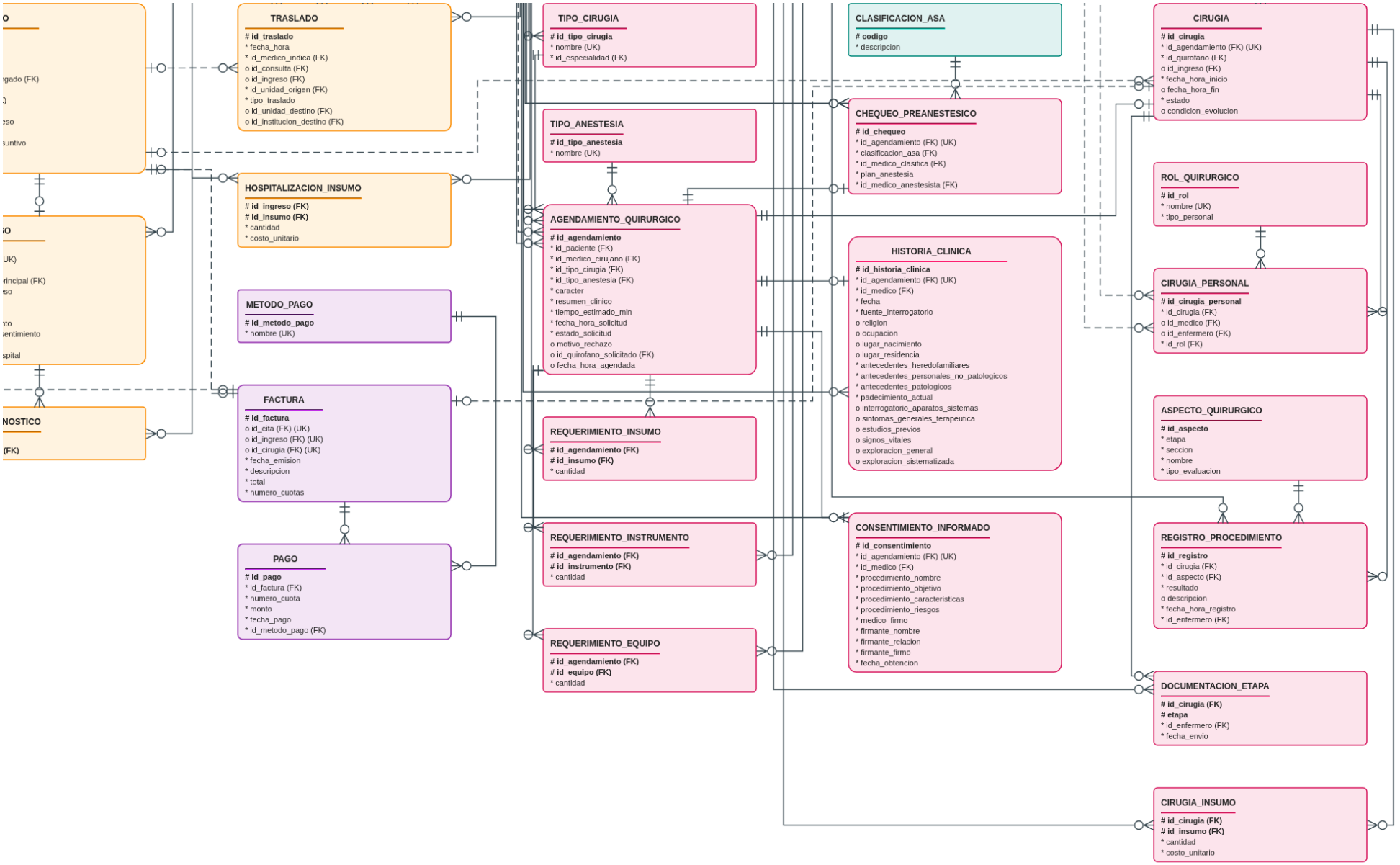
Modelo extendido — cuadrante superior derecho



Modelo extendido — cuadrante inferior izquierdo



Modelo extendido — cuadrante inferior derecho



2.1 Estrategia de mapeo del supertipo y relación recursiva

• **Tabla por supertipo y por subtipo (class table inheritance).** PERSONAL guarda los datos que comparte todo el personal (nombres, DPI, teléfono, dirección, municipio, área, hospital, fecha de contratación, salario y nivel de acceso) y el discriminador *tipo_personal*, restringido con CHECK a medico o enfermero.

• **Llave compartida.** id_medico e id_enfermero son el mismo valor que id_personal. Cada subtipo referencia a PERSONAL con una FK compuesta (id, tipo_personal) contra el UNIQUE (id_personal, tipo_personal): así un médico solo puede apuntar a personal de tipo médico. La jerarquía es **exclusiva** (una persona es de un solo subtipo) y **completa** (todo el personal es médico o enfermero).

• **Por qué esta estrategia.** Una sola tabla con todas las columnas dejaría muchas nulas (los enfermeros no tienen especialidad ni horario de consulta); una tabla por subtipo sin supertipo repetiría los datos comunes y no permitiría auditar o consultar al personal como un todo. Con supertipo y subtipos cada dato vive una vez y las FK del Proyecto 1 hacia medico y enfermero siguen funcionando sin cambios.

• **Migración no destructiva.** Como el enunciado no permite modificar de forma destructiva las tablas existentes, medico y enfermero conservan sus columnas del Proyecto 1. Al registrar personal nuevo, sus datos comunes se guardan en PERSONAL y también en su subtipo (el INSERT del subtipo los toma de PERSONAL).

• **Relación recursiva.** medico.id_medico_superior referencia a medico (ON DELETE SET NULL). El cargo (director_medico, jefe_servicio, medico) se valida con CHECK, igual que nadie puede ser su propio superior y que el director no tiene superior.

3. Diccionario de datos de las tablas nuevas y modificadas

Una tabla por entidad nueva o modificada en el Proyecto 2: el supertipo PERSONAL y sus subtipos (Bloque B), la bitácora (Bloque F), los catálogos que reemplazan a los CHECK con muchas opciones (sql/ddl/03_catalogos.sql) y las clínicas y órdenes de laboratorio que faltaban en consulta externa (sql/ddl/04_laboratorio_y_clinicas.sql). En las filas de llave foránea se indica la acción referencial ante DELETE y UPDATE y su justificación de negocio. Debajo se detallan sus restricciones CHECK, disparadores e índices.

Tabla personal (nueva)

Supertipo con los datos comunes de todo el personal del hospital; sus subtipos son medico y enfermero.

Atributo	Tipo de dato	Llave	Nulos	Por defecto	Descripción	DELETE	UPDATE	Justificación
id_personal	integer	PK	No	identity	Identificador de la persona; es también la llave de su subtipo.			
tipo_personal	character varying(9)	—	No		Discriminador: indica si la persona es medico o enfermero.			
nombres	character varying(100)	—	No		Nombres de la persona.			
apellidos	character varying(100)	—	No		Apellidos de la persona.			
dpi	character(13)	UK	No		Documento Personal de Identificación (CUI de 13 dígitos).			
telefono	character varying(9)	—	Sí		Teléfono de contacto.			
direccion	character varying(250)	—	No		Dirección de residencia.			
id_municipio	integer	FK	No		Municipio de residencia.	RESTRICT	CASCADE	DELETE RESTRICT: un municipio de catálogo no se borra mientras viva personal en él. UPDATE CASCADE: la llave es identity; si cambiara, la persona sigue a su municipio.
area	character varying(6)	—	No		Área de residencia: Urbana o Rural.			
id_hospital	integer	FK	No		Hospital donde trabaja.	RESTRICT	CASCADE	DELETE RESTRICT: no se elimina un hospital que todavía tiene personal; primero se reasigna. UPDATE CASCADE: si cambiara la llave del hospital, su personal la sigue.
fecha_contratacion	date	—	Sí		Fecha de contratación; nula para los médicos externos, que no están contratados.			
salario	numeric(10,2)	—	Sí		Salario mensual en quetzales; nulo si no está contratado. Se audita cada cambio.			
nivel_acceso	character varying(13)	—	No	'consulta'	Nivel de acceso al sistema: consulta, operativo, supervisor o administrador. Se audita cada cambio.			

Restricciones UNIQUE compuestas

Restricción	Columnas	Para qué sirve
uq_personal_tipo	id_personal, tipo_personal	Permite que cada subtipo referencie la pareja (id, tipo) y no el de otro tipo, osea que verifica que esa pareja id, tipo sea unica

Restricciones CHECK

Restricción	Expresión	Qué valida
ck_personal_area	`CHECK (((area) = ANY ((ARRAY['Urbana', 'Rural']))))`	Valida si la persona vive en zona urbana o rural
ck_personal_contratacion	`CHECK ((fecha_contratacion >= '1970-01-01'))`	Valida que la fecha de contratacion sea razonable
ck_personal_contratado	`CHECK (((fecha_contratacion IS NULL) = (salario IS NULL)))`	Valida que solo el personal contratado tenga salario (los externos no)
ck_personal_dpi	`CHECK ((length(dpi) = 13))`	Valida que el DPI (CUI) tenga 13 caracteres
ck_personal_nivel_acceso	`CHECK (((nivel_acceso) = ANY ((ARRAY['consulta', 'operativo', 'supervisor', 'administrador']))))`	Valida el nivel de acceso al sistema de informacion del hospital
ck_personal_salario	`CHECK ((salario >= (0)))`	Valida que el salario no sea negativo
ck_personal_telefono	`CHECK (((length((telefono)) >= 8) AND (length((telefono)) <= 9)))`	Valida el telefono: 8 digitos, o 9 si lleva guion (5551-2031)
ck_personal_tipo	`CHECK (((tipo_personal) = ANY ((ARRAY['medico', 'enfermero']))))`	Valida el discriminador: a que subtipo pertenece la persona

Disparadores

Disparador	Cuándo se ejecuta	Función	Propósito
trg_auditar_personal	AFTER UPDATE	fn_auditar_personal	Cuando cambia el salario o el nivel de acceso de alguien del personal

Índices

Índice	Columnas
personal_pkey	`(id_personal)`
uq_personal_dpi	`(dpi)`
uq_personal_tipo	`(id_personal, tipo_personal)`

Tabla auditoria_personal (nueva)

Bitácora automática de los cambios de salario, nivel de acceso y especialidad del personal.

Atributo	Tipo de dato	Llave	Nulos	Por defecto	Descripción	DELETE	UPDATE	Justificación
id_auditoria	integer	PK	No	identity	Identificador del registro de la bitácora (lo genera la base).			
fecha_hora	timestamp with time zone	—	No	now()	Fecha y hora exacta del cambio (now()).			
usuario	character varying(63)	—	No		Usuario de la base de datos que hizo el cambio (current_user).			
tabla	character varying(30)	—	No		Tabla donde ocurrió el cambio: personal o medico.			
id_registro	integer	—	No		Identificador de la persona cuyo dato cambió (id_personal = id_medico).			
campo	character varying(30)	—	No		Campo que cambió: salario, nivel_acceso o id_especialidad.			
valor_anterior	text	—	Sí		Valor antes del cambio.			
valor_nuevo	text	—	Sí		Valor después del cambio.			

Índices

Índice	Columnas
auditoria_personal_pkey	`(id_auditoria)`

Tabla clinica (nueva)

Clínicas (consultorios) de la unidad de Consulta Externa de cada hospital.

Atributo	Tipo de dato	Llave	Nulos	Por defecto	Descripción	DELETE	UPDATE	Justificación
id_clinica	integer	PK	No	identity	Identificador de la clínica.			
id_unidad	integer	FK	No		Referencia a unidad_medica.	RESTRICT	CASCADE	DELETE RESTRICT: no se borra una unidad que tiene clínicas con citas. UPDATE CASCADE: la clínica sigue a su unidad.
numero	character varying(10)	—	No		Número o código de la clínica dentro de su unidad.			

Restricciones UNIQUE compuestas

Restricción	Columnas	Para qué sirve
uq_clinica_numero	id_unidad, numero	No se repite el numero de clinica dentro de la misma unidad

Índices

Índice	Columnas
clinica_pkey	`(id_clinica)`
uq_clinica_numero	`(id_unidad, numero)`

Tabla orden_laboratorio (nueva)

Orden de laboratorio que el médico entrega en una consulta.

Atributo	Tipo de dato	Llave	Nulos	Por defecto	Descripción	DELETE	UPDATE	Justificación
id_orden	integer	PK	No	identity	Identificador de la orden.			
id_consulta	integer	FK	No		Referencia a consulta.	RESTRICT	CASCADE	DELETE RESTRICT: la orden es parte de la historia clínica de la consulta. UPDATE CASCADE: la orden sigue a su consulta.
fecha	date	—	No		Fecha de la orden.			
indicaciones	text	—	Sí		Indicaciones para el paciente.			

Índices

Índice	Columnas
orden_laboratorio_pkey	`(id_orden)`

Tabla orden_examen (nueva)

Exámenes que se piden en cada orden de laboratorio.

Atributo	Tipo de dato	Llave	Nulos	Por defecto	Descripción	DELETE	UPDATE	Justificación
id_orden	integer	PK/FK	No		Referencia a orden_laboratorio.	CASCADE	CASCADE	DELETE CASCADE: los exámenes pedidos no existen sin su orden. UPDATE CASCADE: siguen a su orden.
id_examen	integer	PK/FK	No		Referencia a examen_laboratorio.	RESTRICT	CASCADE	DELETE RESTRICT: no se borra un examen del catálogo que ya fue ordenado. UPDATE CASCADE: si cambia el id, las órdenes lo siguen.

Índices

Índice	Columnas
orden_examen_pkey	`(id_orden, id_examen)`

Tabla examen_laboratorio (nueva)

Catálogo de exámenes de laboratorio que se pueden ordenar.

Atributo	Tipo de dato	Llave	Nulos	Por defecto	Descripción	DELETE	UPDATE	Justificación
id_examen	integer	PK	No	identity	Identificador del examen.			
nombre	character varying(100)	UK	No		Nombre del examen.			
descripcion	character varying(200)	—	Sí		Qué mide el examen.			
costo	numeric(10,2)	—	No		Costo del examen en quetzales.			

Restricciones CHECK

Restricción	Expresión	Qué valida
ck_examen_costo	`CHECK ((costo >= (0)))`	El costo del examen no puede ser negativo

Índices

Índice	Columnas
examen_laboratorio_pkey	`(id_examen)`
uq_examen_nombre	`(nombre)`

Tabla forma_farmaceutica (nueva)

Catálogo de formas en que se entrega un medicamento.

Atributo	Tipo de dato	Llave	Nulos	Por defecto	Descripción	DELETE	UPDATE	Justificación
codigo	character varying(12)	PK	No		Código de la forma farmacéutica.			
descripcion	character varying(100)	—	No		Descripción de la forma farmacéutica.			

Índices

Índice	Columnas
forma_farmaceutica_pkey	`(codigo)`

Tabla funcion_instrumento (nueva)

Catálogo de funciones de un instrumento quirúrgico.

Atributo	Tipo de dato	Llave	Nulos	Por defecto	Descripción	DELETE	UPDATE	Justificación
codigo	character varying(12)	PK	No		Código de la función.			
descripcion	character varying(100)	—	No		Descripción de la función.			

Índices

Índice	Columnas
funcion_instrumento_pkey	`(codigo)`

Tabla funcion_equipo (nueva)

Catálogo de funciones de un equipo médico.

Atributo	Tipo de dato	Llave	Nulos	Por defecto	Descripción	DELETE	UPDATE	Justificación
codigo	character varying(14)	PK	No		Código de la función.			
descripcion	character varying(100)	—	No		Descripción de la función.			

Índices

Índice	Columnas
funcion_equipo_pkey	`(codigo)`

Tabla clasificacion_asa (nueva)

Catálogo de la clasificación ASA del riesgo anestésico.

Atributo	Tipo de dato	Llave	Nulos	Por defecto	Descripción	DELETE	UPDATE	Justificación
codigo	character varying(3)	PK	No		Clase ASA (I a VI).			
descripcion	character varying(100)	—	No		Significado de la clase.			

Índices

Índice	Columnas
clasificacion_asa_pkey	`(codigo)`

Tabla tipo_medico (nueva)

Catálogo de modalidades del médico dentro del hospital.

Atributo	Tipo de dato	Llave	Nulos	Por defecto	Descripción	DELETE	UPDATE	Justificación
codigo	character varying(12)	PK	No		Código de la modalidad.			
descripcion	character varying(100)	—	No		Descripción de la modalidad.			

Índices

Índice	Columnas
tipo_medico_pkey	`(codigo)`

Tabla medio_asignacion (nueva)

Catálogo de medios por los que se asigna una cita.

Atributo	Tipo de dato	Llave	Nulos	Por defecto	Descripción	DELETE	UPDATE	Justificación
codigo	character varying(20)	PK	No		Código del medio.			
descripcion	character varying(100)	—	No		Descripción del medio.			

Índices

Índice	Columnas
medio_asignacion_pkey	`(codigo)`

Tabla medico (modificada)

Subtipo de personal: médicos de la cadena (los externos no están contratados, solo operan).

Atributo	Tipo de dato	Llave	Nulos	Por defecto	Descripción	DELETE	UPDATE	Justificación
id_medico	integer	PK/FK	No		Identificador del médico; es el mismo id_personal de su supertipo.	CASCADE	CASCADE	FK compuesta (id_medico, tipo_personal) hacia personal. DELETE CASCADE: el subtipo no existe sin su supertipo, al borrar la persona se borra su fila de médico (si tiene historia clínica, esas FK lo impiden). UPDATE CASCADE: el subtipo comparte la llave con el supertipo y debe seguirla; un cambio de tipo_personal en personal choca con el CHECK del subtipo, por lo que nadie puede cambiar de médico a enfermero.
nombres	character varying(100)	—	No		Copia de personal.nombres (se guarda igual en las dos tablas).			
apellidos	character varying(100)	—	No		Copia de personal.apellidos.			
dpi	character(13)	UK	No		Copia de personal.dpi.			
telefono	character varying(9)	—	Sí		Copia de personal.telefono.			
id_hospital	integer	FK	No		Copia de personal.id_hospital.	RESTRICT	CASCADE	RESTRICT: un hospital con médicos asignados no se elimina; primero se reasigna su personal.
id_municipio	integer	FK	No		Copia de personal.id_municipio.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.
area	character varying(6)	—	No		Copia de personal.area.			
id_especialidad	integer	FK	No		Especialidad del médico. Se audita cada cambio.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.
tipo_medico	character varying(12)	FK	No		Modalidad: general, especialista, residente, interno o externo.	RESTRICT	CASCADE	DELETE RESTRICT: no se borra una modalidad que tienen médicos. UPDATE CASCADE: si cambia el código, los médicos lo siguen.
costo_primera_consulta	numeric(10,2)	—	Sí		Costo de la primera consulta; base para reconsultas y referidos.			
horario_inicio	time without time zone	—	Sí		Inicio del horario de consulta externa (turno matutino).			
horario_fin	time without time zone	—	Sí		Fin del horario de consulta externa.			
tipo_personal	character varying(9)	FK	No	'medico'	Discriminador fijo en medico; junto con id_medico forma la FK hacia personal.	CASCADE	CASCADE	Parte de la FK compuesta hacia personal (ver id_medico): obliga a que el médico apunte a personal de tipo médico.
cargo	character varying(15)	—	No	'medico'	Cargo en la cadena de mando: director_medico, jefe_servicio o medico.			

Atributo	Tipo de dato	Llave	Nulos	Por defecto	Descripción	DELETE	UPDATE	Justificación
id_medico_superior	integer	FK	Sí		Superior directo dentro del mismo hospital (relación recursiva); nulo para el director.	SET NULL	CASCADE	Relación recursiva. DELETE SET NULL: si el jefe deja el hospital, sus subordinados quedan sin superior hasta que se les asigne otro, sin perder su historia ni bloquear la baja. UPDATE CASCADE: si cambiara la llave del jefe, sus subordinados lo siguen.

Las filas resaltadas son las columnas agregadas en el Proyecto 2 o que ahora apuntan a un catálogo.

Restricciones CHECK

Restricción	Expresión	Qué valida
ck_medico_area	`CHECK (((area) = ANY ((ARRAY['Urbana', 'Rural']))))`	Valida si el medico vive en zona urbana o rural
ck_medico_cargo	`CHECK (((cargo) = ANY ((ARRAY['director_medico', 'jefe_servicio', 'medico']))))`	Valida el cargo del medico dentro de la cadena de mando
ck_medico_costo	`CHECK ((costo_primera_consulta >= (0)))`	Valida que el costo de la primera consulta no sea negativo
ck_medico_director_sin_superior	`CHECK ((((cargo) <> 'director_medico') OR (id_medico_superior IS NULL)))`	Valida que el director medico no tenga superior: es la cima de la cadena
ck_medico_dpi	`CHECK ((dpi ~ '^[0-9]{13}$'))`	Valida que el DPI (CUI) tenga 13 digitos
ck_medico_externo	`CHECK ((((tipo_medico) <> 'externo') OR ((horario_inicio IS NULL) AND (costo_primera_consulta IS NULL))))`	Valida que un externo no tenga horario ni costo de consulta: solo opera
ck_medico_horario	`CHECK (((horario_inicio < horario_fin) AND (horario_fin <= '13:00:00'::time without time zone)))`	Valida que la consulta externa sea en turno matutino y termine despues de empezar
ck_medico_horario_completo	`CHECK (((horario_inicio IS NULL) = (horario_fin IS NULL)))`	Valida que el horario tenga inicio y fin, o ninguno
ck_medico_superior_propio	`CHECK ((id_medico_superior <> id_medico))`	Valida que nadie sea su propio superior
ck_medico_telefono	`CHECK (((telefono) ~ '^[0-9]{4}-?[0-9]{4}$'))`	Valida el telefono de 8 digitos (ej. 5551-2001)
ck_medico_tipo_personal	`CHECK (((tipo_personal) = 'medico'))`	Valida que la fila del subtipo medico apunte a personal de tipo medico

Disparadores

Disparador	Cuándo se ejecuta	Función	Propósito
trg_auditar_medico	AFTER UPDATE	fn_auditar_medico	Cuando cambia la especialidad de un medico

Índices

Índice	Columnas
medico_pkey	`(id_medico)`
uq_medico_dpi	`(dpi)`

Tabla enfermero (modificada)

Subtipo de personal: enfermeros registrados y practicantes de enfermería.

Atributo	Tipo de dato	Llave	Nulos	Por defecto	Descripción	DELETE	UPDATE	Justificación
id_enfermero	integer	PK/FK	No		Identificador del enfermero; es el mismo id_personal de su supertipo.	CASCADE	CASCADE	FK compuesta (id_enfermero, tipo_personal) hacia personal. DELETE CASCADE: el subtipo no existe sin su supertipo. UPDATE CASCADE: el subtipo comparte la llave con el supertipo y debe seguirla.
nombres	character varying(100)	—	No		Copia de personal.nombres (se guarda igual en las dos tablas).			
apellidos	character varying(100)	—	No		Copia de personal.apellidos.			
dpi	character(13)	UK	No		Copia de personal.dpi.			
telefono	character varying(9)	—	Sí		Copia de personal.telefono.			
id_hospital	integer	FK	No		Copia de personal.id_hospital.	RESTRICT	CASCADE	RESTRICT: un hospital con enfermeros asignados no se elimina; primero se reasigna su personal.
tipo	character varying(12)	—	No		registrado o practicante.			
tipo_personal	character varying(9)	FK	No	'enfermero'	Discriminador fijo en enfermero; junto con id_enfermero forma la FK hacia personal.	CASCADE	CASCADE	Parte de la FK compuesta hacia personal (ver id_enfermero): obliga a que apunte a personal de tipo enfermero.

Las filas resaltadas son las columnas agregadas en el Proyecto 2 o que ahora apuntan a un catálogo.

Restricciones CHECK

Restricción	Expresión	Qué valida
ck_enfermero_dpi	`CHECK ((dpi ~ '^[0-9]{13}$'))`	Valida que el DPI (CUI) tenga 13 digitos
ck_enfermero_telefono	`CHECK (((telefono) ~ '^[0-9]{4}-?[0-9]{4}$'))`	Valida el telefono de 8 digitos
ck_enfermero_tipo	`CHECK (((tipo) = ANY ((ARRAY['registrado', 'practicante']))))`	Valida si el enfermero tiene titulo o es practicante
ck_enfermero_tipo_personal	`CHECK (((tipo_personal) = 'enfermero'))`	Valida que la fila del subtipo enfermero apunte a personal de tipo enfermero

Índices

Índice	Columnas
enfermero_pkey	`(id_enfermero)`
uq_enfermero_dpi	`(dpi)`

Tabla cita (modificada)

Citas programadas de consulta externa.

Atributo	Tipo de dato	Llave	Nulos	Por defecto	Descripción	DELETE	UPDATE	Justificación
id_cita	integer	PK	No	identity				
id_paciente	integer	FK	No		Referencia a paciente.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el paciente dejaría la cita sin referencia, por eso se impide.
id_medico	integer	FK	No		Referencia a medico.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el médico dejaría la cita sin referencia, por eso se impide.
fecha_hora	timestamp with time zone	—	No		Fecha y hora de la cita.			
numero_clinica	character varying(10)	—	No		Número de clínica escrito en la cita (se conserva por compatibilidad; usar id_clinica).			
medio_asignacion	character varying(20)	FK	No		Medio por el que se asignó: teléfono, correo, recepción general o del médico.	RESTRICT	CASCADE	DELETE RESTRICT: no se borra un medio con el que se asignaron citas. UPDATE CASCADE: si cambia el código, las citas lo siguen.
estado	character varying(12)	—	No	'programada'	programada, realizada, reprogramada o cancelada.			
es_reconsulta	boolean	—	No	false	Indica si es reconsulta (cuesta 75% de la primera consulta).			
es_referido	boolean	—	No	false	Indica si el paciente fue referido por una institución (cuesta 80%).			
cancelada_sin_aviso	boolean	—	No	false	Indica si se canceló sin avisar (sobrecargo de 10% en la siguiente cita).			
costo_base	numeric(10,2)	—	No		Costo de primera consulta del médico al momento de agendar.			
costo_final	numeric(10,2)	—	No		Costo cobrado después de aplicar las reglas de descuento o sobrecargo.			
id_clinica	integer	FK	No		Clínica donde se atiende la cita.	RESTRICT	CASCADE	DELETE RESTRICT: no se borra una clínica donde ya hubo citas, es parte de la historia. UPDATE CASCADE: la cita sigue a su clínica.

Las filas resaltadas son las columnas agregadas en el Proyecto 2 o que ahora apuntan a un catálogo.

Restricciones CHECK

Restricción	Expresión	Qué valida
ck_cita_costos	`CHECK (((costo_base >= (0)) AND (costo_final >= (0))))`	Valida que los costos no sean negativos
ck_cita_estado	`CHECK (((estado) = ANY ((ARRAY['programada', 'realizada', 'reprogramada', 'cancelada']))))`	Valida en que situacion se encuentra la cita
ck_cita_sin_aviso	`CHECK (((NOT cancelada_sin_aviso) OR ((estado) = 'cancelada')))`	Valida que solo una cita cancelada pueda marcarse como cancelada sin aviso

Índices

Índice	Columnas
cita_pkey	(id_cita)

Tabla medicamento (modificada)

Medicamentos que se pueden recetar.

Atributo	Tipo de dato	Llave	Nulos	Por defecto	Descripción	DELETE	UPDATE	Justificación
id_medicamento	integer	PK	No	identity				
nombre_generico	character varying(80)	—	No		Nombre genérico (principio activo).			
concentracion	character varying(30)	—	No		Concentración por unidad (ej. 50 mg).			
forma_farmaceutica	character varying(12)	FK	No		Presentación: tableta, cápsula, jarabe, inyectable...	RESTRICT	CASCADE	DELETE RESTRICT: no se borra una forma farmacéutica que usa algún medicamento. UPDATE CASCADE: si cambia el código, los medicamentos lo siguen.

Las filas resaltadas son las columnas agregadas en el Proyecto 2 o que ahora apuntan a un catálogo.

Restricciones UNIQUE compuestas

Restricción	Columnas	Para qué sirve
uq_medicamento	nombre_generico, concentracion, forma_farmaceutica	El mismo medicamento puede venir en varias presentaciones

Índices

Índice	Columnas
medicamento_pkey	(id_medicamento)
uq_medicamento	(nombre_generico, concentracion, forma_farmaceutica)

Tabla instrumento (modificada)

Instrumental médico y quirúrgico.

Atributo	Tipo de dato	Llave	Nulos	Por defecto	Descripción	DELETE	UPDATE	Justificación
id_instrumento	integer	PK	No	identity				
nombre	character varying(120)	UK	No		Nombre del instrumento.			
descripcion	text	—	Sí		Descripción del instrumento.			
tipo	character varying(10)	—	No		medico, quirurgico u otro.			
funcion	character varying(12)	FK	No		corte, contencion, hemostatica, retractor, accesorio, implante u otro.	RESTRICT	CASCADE	DELETE RESTRICT: no se borra una función que tiene instrumentos. UPDATE CASCADE: si cambia el código, los instrumentos lo siguen.
estado	character varying(16)	—	No	'disponible'	disponible, en_uso o en_mantenimiento.			

Las filas resaltadas son las columnas agregadas en el Proyecto 2 o que ahora apuntan a un catálogo.

Restricciones CHECK

Restricción	Expresión	Qué valida
ck_instrumento_estado	`CHECK (((estado) = ANY ((ARRAY['disponible', 'en_uso', 'en_mantenimiento']))))`	Valida si el instrumento esta disponible, en uso o en mantenimiento
ck_instrumento_tipo	`CHECK (((tipo) = ANY ((ARRAY['medico', 'quirurgico', 'otro']))))`	Valida si el instrumento es medico, quirurgico u otro

Índices

Índice	Columnas
instrumento_pkey	`(id_instrumento)`
uq_instrumento_nombre	`(nombre)`

Tabla equipo (modificada)

Equipos médicos y quirúrgicos.

Atributo	Tipo de dato	Llave	Nulos	Por defecto	Descripción	DELETE	UPDATE	Justificación
id_equipo	integer	PK	No	identity				
nombre	character varying(120)	UK	No		Nombre del equipo.			
descripcion	text	—	Sí		Descripción del equipo.			
tipo	character varying(10)	—	No		medico, quirurgico u otro.			
funcion	character varying(14)	FK	No		exploracion, diagnostico, tratamiento, rehabilitacion u otro.	RESTRICT	CASCADE	DELETE RESTRICT: no se borra una función que tiene equipos. UPDATE CASCADE: si cambia el código, los equipos lo siguen.
estado	character varying(16)	—	No	'disponible'	disponible, en_uso o en_mantenimiento.			

Las filas resaltadas son las columnas agregadas en el Proyecto 2 o que ahora apuntan a un catálogo.

Restricciones CHECK

Restricción	Expresión	Qué valida
ck_equipo_estado	`CHECK (((estado) = ANY ((ARRAY['disponible', 'en_uso', 'en_mantenimiento']))))`	Valida si el equipo esta disponible, en uso o en mantenimiento
ck_equipo_tipo	`CHECK (((tipo) = ANY ((ARRAY['medico', 'quirurgico', 'otro']))))`	Valida si el equipo es medico, quirurgico u otro

Índices

Índice	Columnas
equipo_pkey	`(id_equipo)`
uq_equipo_nombre	`(nombre)`

Tabla chequeo_preanestesico (modificada)

Evaluación del riesgo anestésico (ASA) y plan de anestesia.

Atributo	Tipo de dato	Llave	Nulos	Por defecto	Descripción	DELETE	UPDATE	Justificación
id_chequeo	integer	PK	No	identity				
id_agendamiento	integer	FK/UK	No		Referencia a agendamiento_quirurgico.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el agendamiento dejaría el chequeo preanestésico sin referencia, por eso se impide.
clasificacion_asa	character varying(3)	FK	No		Clasificación ASA del riesgo anestésico (I a VI).	RESTRICT	CASCADE	DELETE RESTRICT: la clase ASA es parte de la historia clínica del paciente. UPDATE CASCADE: si cambia el código, los chequeos lo siguen.
id_medico_clasifica	integer	FK	No		Referencia a medico.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el médico dejaría el chequeo sin referencia, por eso se impide.
plan_anestesia	text	—	No		Plan de anestesia.			
id_medico_anestesista	integer	FK	No		Referencia a medico.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el anestesista dejaría el chequeo sin referencia, por eso se impide.

Las filas resaltadas son las columnas agregadas en el Proyecto 2 o que ahora apuntan a un catálogo.

Índices

Índice	Columnas
chequeo_preanestesico_pkey	`(id_chequeo)`
uq_chequeo_agendamiento	`(id_agendamiento)`

4. Objetivos

Para cada objetivo se muestra el criterio usado, la sentencia (archivo sql/consultas/objetivo_XX.sql) y la captura de su resultado, ejecutado con el rol de solo lectura visor sobre enero a octubre de 2026.

Bloque A — Consultas de inteligencia de negocio

Objetivo 1. Hospitales con mayor demanda en consulta externa

demanda = citas no canceladas (reservaciones) + consultas realizadas + proximas citas de las recetas lapso: enero a octubre de 2026

```
select h.nombre as hospital,
       count(case when c.estado <> 'cancelada' then 1 end) as reservaciones,
       count(co.id_consulta) as consultas,
       count(r.fecha_proxima_cita) as proximas_consultas,
       count(case when c.estado <> 'cancelada' then 1 end) + count(co.id_consulta) + count(r.fecha_proxima_cita) as demanda_total
from cita c
inner join medico m on m.id_medico = c.id_medico   --a cada cita se busca su medico
inner join hospital h on h.id_hospital = m.id_hospital --a cada medico se busca su hospital
left join consulta co on co.id_cita = c.id_cita  --aqui  se busca a la cita su consulta, solo si la tuvo
left join receta r on r.id_consulta = co.id_consulta --a cada consulta se busca su receta, solo si la tuvo
where c.fecha_hora between '2026-01-01' and '2026-10-31 23:59:59' --aqui se filtra por el lapso de tiempo
group by h.nombre --aqui agurpo por nombre de hospital
order by demanda_total desc -- ordeno de manera descendente por demanda total
limit 3; --y solo muestro los 3 primeros hospitales con mas demanda
```

Objetivo 1 — Los 3 hospitales con mayor demanda en consulta externa

PostgreSQL 18 · base hospital · conectado con el rol de solo lectura: visor

```
select h.nombre as hospital,
       count(case when c.estado <> 'cancelada' then 1 end) as reservaciones,
       count(co.id_consulta) as consultas,
       count(r.fecha_proxima_cita) as proximas_consultas,
       count(case when c.estado <> 'cancelada' then 1 end) + count(co.id_consulta) + count(r.fecha_proxima_cita) as demanda_total
from cita c
inner join medico m on m.id_medico = c.id_medico   --a cada cita se busca su medico
inner join hospital h on h.id_hospital = m.id_hospital --a cada medico se busca su hospital
left join consulta co on co.id_cita = c.id_cita  --aqui  se busca a la cita su consulta, solo si la tuvo
left join receta r on r.id_consulta = co.id_consulta --a cada consulta se busca su receta, solo si la tuvo
where c.fecha_hora between '2026-01-01' and '2026-10-31 23:59:59' --aqui se filtra por el lapso de tiempo
group by h.nombre --aqui agurpo por nombre de hospital
order by demanda_total desc -- ordeno de manera descendente por demanda total
limit 3;
```

hospital	reservaciones	consultas	proximas_consultas	demanda_total
Hospital General de Occidente	77	66	29	172
Hospital Regional de San Marcos	44	40	16	100
Hospital Infantil de Retalhuleu	31	27	12	70

3 filas · 55 ms

```
--y solo muestro los 3 primeros hospitales con mas demanda;
```

0 filas · 62 ms

Objetivo 2. Médicos con menor calificación

se usa la calificacion (1 a 5) que el paciente le dio al medico en cada consulta lapso: enero a octubre de 2026

```
select m.nombres, m.apellidos,
       e.nombre as especialidad,
       h.nombre as hospital,
       round(avg(ca.puntuacion), 2) as calificacion_promedio,
       count(co.id_consulta) as consultas
from consulta co
inner join cita c on c.id_cita = co.id_cita
inner join medico m on m.id_medico = c.id_medico
inner join especialidad e on e.id_especialidad = m.id_especialidad
inner join hospital h on h.id_hospital = m.id_hospital
left join calificacion ca on ca.id_consulta = co.id_consulta
where c.fecha_hora between '2026-01-01' and '2026-10-31 23:59:59'
group by m.id_medico, m.nombres, m.apellidos, e.nombre, h.nombre
having count(ca.id_calificacion) > 0   --aqui solo se consideran los medicos que tengan al menos una calificacion
order by calificacion_promedio, consultas desc, m.apellidos
limit 5;
```

Objetivo 2 — Los 5 médicos de consulta externa con menor calificación

PostgreSQL 18 · base hospital · conectado con el rol de solo lectura: visor

```
select m.nombres, m.apellidos,
       e.nombre as especialidad,
       h.nombre as hospital,
       round(avg(ca.puntuacion), 2) as calificacion_promedio,
       count(co.id_consulta) as consultas
from consulta co
inner join cita c on c.id_cita = co.id_cita
inner join medico m on m.id_medico = c.id_medico
inner join especialidad e on e.id_especialidad = m.id_especialidad
inner join hospital h on h.id_hospital = m.id_hospital
left join calificacion ca on ca.id_consulta = co.id_consulta
where c.fecha_hora between '2026-01-01' and '2026-10-31 23:59:59'
group by m.id_medico, m.nombres, m.apellidos, e.nombre, h.nombre
having count(ca.id_calificacion) > 0   --aqui solo se consideran los medicos que tengan al menos una calificacion
order by calificacion_promedio, consultas desc, m.apellidos
limit 5;
```

nombres	apellidos	especialidad	hospital	calificacion_promedio	consultas
Andrés	Chinchilla Paz	Medicina General	Hospital General de Occidente	1.25	7
Elena	Vásquez Sotelo	Dermatología	Hospital General de Occidente	1.40	7
Marcos	Aguilar Salazar	Ginecología Oncológica	Hospital General de Occidente	1.67	6
Mónica	Pirir Cutzal	Psicología	Hospital de Occidente Sololá	1.67	6
Miguel	Barrios Donis	Medicina General	Hospital General de Occidente	2.00	7

5 filas · 64 ms

Objetivo 3. Mejores médicos de emergencias

lapso: enero a octubre de 2026

```
select m.nombres, m.apellidos,
       h.nombre as hospital,
       count(*) as turnos,
       count(*) * 12 as horas_laboradas, -- 12 horas por turno
       (select count(*)
        from egreso e
        inner join ingreso i on i.id_ingreso = e.id_ingreso
        where e.id_medico = m.id_medico
          and i.id_turno is not null      -- solo los ingresos de emergencias tienen turno
          and e.codigo_egreso = 'vivo'
          and e.sin_consentimiento = false
          and e.fecha_hora_egreso between '2026-01-01' and '2026-10-31 23:59:59') as egresos_vivos
from turno_medico tm
inner join turno_emergencia t on t.id_turno = tm.id_turno
inner join medico m on m.id_medico = tm.id_medico
inner join hospital h on h.id_hospital = m.id_hospital
where t.fecha between '2026-01-01' and '2026-10-31'
group by m.id_medico, m.nombres, m.apellidos, h.nombre
order by egresos_vivos desc, turnos desc
  limit 5;
```

Objetivo 3 — Los 5 mejores médicos de emergencias

PostgreSQL 18 · base hospital · conectado con el rol de solo lectura: visor

```
select m.nombres, m.apellidos,
       h.nombre as hospital,
       count(*) as turnos,
       count(*) * 12 as horas_laboradas, -- 12 horas por turno
       (select count(*)
        from egreso e
        inner join ingreso i on i.id_ingreso = e.id_ingreso
        where e.id_medico = m.id_medico
          and i.id_turno is not null      -- solo los ingresos de emergencias tienen turno
          and e.codigo_egreso = 'vivo'
          and e.sin_consentimiento = false
          and e.fecha_hora_egreso between '2026-01-01' and '2026-10-31 23:59:59') as egresos_vivos
from turno_medico tm
inner join turno_emergencia t on t.id_turno = tm.id_turno
inner join medico m on m.id_medico = tm.id_medico
inner join hospital h on h.id_hospital = m.id_hospital
where t.fecha between '2026-01-01' and '2026-10-31'
group by m.id_medico, m.nombres, m.apellidos, h.nombre
order by egresos_vivos desc, turnos desc
  limit 5;
```

nombres	apellidos	hospital	turnos	horas_laboradas	egresos_vivos
Lucía	Cumes Xajil	Hospital de Occidente Sololá	190	2280	12
Carmen	Ruiz Palacios	Hospital General de Occidente	160	1920	12
Diego	Yaxón Bocel	Hospital de Occidente Sololá	190	2280	11
Otto René	Velásquez Lima	Hospital Infantil de Retalhuleu	190	2280	11
Brenda	Sic Ixcot	Hospital Infantil de Retalhuleu	190	2280	11

5 filas · 64 ms

Objetivo 4. Hospitales más eficientes en emergencias

eficiente = tiene camilla libre cuando llega el paciente: mayor porcentaje de ingresos atendidos sin esperar turno y menor promedio de turnos de espera lapso: enero a octubre de 2026

```
select h.nombre as hospital,
       count(*) as ingresos,
       count(e.id_egreso) as egresos,
       count(case when i.turnos_espera = 0 then 1 end) as atendidos_sin_espera,
       round(100.0 * count(case when i.turnos_espera = 0 then 1 end) / count(*), 1) as porcentaje_sin_espera,
       round(avg(i.turnos_espera), 2) as promedio_turnos_espera
from ingreso i
inner join unidad_medica u on u.id_unidad = i.id_unidad
inner join hospital h on h.id_hospital = u.id_hospital
left join egreso e on e.id_ingreso = i.id_ingreso
where u.tipo_unidad = 'Emergencias'
  and i.fecha_hora_ingreso between '2026-01-01' and '2026-10-31 23:59:59'
group by h.nombre
order by porcentaje_sin_espera desc, promedio_turnos_espera, ingresos desc
limit 3;
```

Objetivo 4 — Los 3 hospitales más eficientes en emergencias

PostgreSQL 18 · base hospital · conectado con el rol de solo lectura: visor

```
select h.nombre as hospital,
       count(*) as ingresos,
       count(e.id_egreso) as egresos,
       count(case when i.turnos_espera = 0 then 1 end) as atendidos_sin_espera,
       round(100.0 * count(case when i.turnos_espera = 0 then 1 end) / count(*), 1) as porcentaje_sin_espera,
       round(avg(i.turnos_espera), 2) as promedio_turnos_espera
from ingreso i
inner join unidad_medica u on u.id_unidad = i.id_unidad
inner join hospital h on h.id_hospital = u.id_hospital
left join egreso e on e.id_ingreso = i.id_ingreso
where u.tipo_unidad = 'Emergencias'
  and i.fecha_hora_ingreso between '2026-01-01' and '2026-10-31 23:59:59'
group by h.nombre
order by porcentaje_sin_espera desc, promedio_turnos_espera, ingresos desc
limit 3;
```

hospital	ingresos	egresos	atendidos_sin_espera	porcentaje_sin_espera	promedio_turnos_espera
Hospital General de Occidente	35	30	28	80.0	0.37
Hospital Infantil de Retalhuleu	30	25	21	70.0	0.60
Hospital de Occidente Sololá	30	26	18	60.0	0.80

3 filas · 55 ms

Objetivo 5. Cirujanos más eficientes

primero los que tienen mas agendamientos aprobados y, entre ellos, los que pidieron menos insumos, instrumentos y equipos (cada uno por separado) lapso: enero a octubre de 2026

```
select m.nombres, m.apellidos,
       count(*) as agendamientos_aprobados,
       sum(coalesce(ri.cantidad, 0)) as insumos,
       sum(coalesce(rn.cantidad, 0)) as instrumentos,
       sum(coalesce(re.cantidad, 0)) as equipos
from agendamiento_quirurgico a
inner join medico m on m.id_medico = a.id_medico_cirujano
left join (select id_agendamiento, sum(cantidad) as cantidad
           from requerimiento_insumo group by id_agendamiento) ri on ri.id_agendamiento = a.id_agendamiento
left join (select id_agendamiento, sum(cantidad) as cantidad
           from requerimiento_instrumento group by id_agendamiento) rn on rn.id_agendamiento = a.id_agendamiento
left join (select id_agendamiento, sum(cantidad) as cantidad
           from requerimiento_equipo group by id_agendamiento) re on re.id_agendamiento = a.id_agendamiento
where a.estado_solicitud = 'aprobada'
  and a.fecha_hora_solicitud between '2026-01-01' and '2026-10-31 23:59:59'
group by m.id_medico, m.nombres, m.apellidos
order by agendamientos_aprobados desc, insumos, instrumentos, equipos
limit 3;
```

Objetivo 5 — Los 3 cirujanos más eficientes

PostgreSQL 18 · base hospital · conectado con el rol de solo lectura: visor

```
select m.nombres, m.apellidos,
       count(*) as agendamientos_aprobados,
       sum(coalesce(ri.cantidad, 0)) as insumos,
       sum(coalesce(rn.cantidad, 0)) as instrumentos,
       sum(coalesce(re.cantidad, 0)) as equipos
from agendamiento_quirurgico a
inner join medico m on m.id_medico = a.id_medico_cirujano
left join (select id_agendamiento, sum(cantidad) as cantidad
           from requerimiento_insumo group by id_agendamiento) ri on ri.id_agendamiento = a.id_agendamiento
left join (select id_agendamiento, sum(cantidad) as cantidad
           from requerimiento_instrumento group by id_agendamiento) rn on rn.id_agendamiento = a.id_agendamiento
left join (select id_agendamiento, sum(cantidad) as cantidad
           from requerimiento_equipo group by id_agendamiento) re on re.id_agendamiento = a.id_agendamiento
where a.estado_solicitud = 'aprobada'
  and a.fecha_hora_solicitud between '2026-01-01' and '2026-10-31 23:59:59'
group by m.id_medico, m.nombres, m.apellidos
order by agendamientos_aprobados desc, insumos, instrumentos, equipos
limit 3;
```

nombres	apellidos	agendamientos_aprobados	insumos	instrumentos	equipos
Alejandro	Ramírez Lemus	7	64	29	13
Patricia	López Mazariegos	5	32	18	5
Silvia	Vásquez Cotero	3	22	11	3

3 filas · 57 ms

Objetivo 6. Tipos de cirugía con mayores ingresos

solo cirugias "sin complicaciones" en las que todos los aspectos de la fase intraoperatoria salieron en exito o aceptables (ninguna falla ni no aceptable) lapso: enero a octubre de 2026

```
select tc.nombre as tipo_cirugia,
       e.nombre as especialidad,
       count(*) as cirugias,
       sum(f.total) as ingresos
from cirugia ci
inner join agendamiento_quirurgico a on a.id_agendamiento = ci.id_agendamiento
inner join tipo_cirugia tc on tc.id_tipo_cirugia = a.id_tipo_cirugia
inner join especialidad e on e.id_especialidad = tc.id_especialidad
inner join factura f on f.id_cirugia = ci.id_cirugia
where ci.condicion_evolucion = 'sin_complicaciones'
  and ci.fecha_hora_inicio between '2026-01-01' and '2026-10-31 23:59:59'
  and not exists (select 1
                  from registro_procedimiento rp
                  inner join aspecto_quirurgico aq on aq.id_aspecto = rp.id_aspecto
                  where rp.id_cirugia = ci.id_cirugia
                    and aq.etapa = 'intraoperatorio'
                    and rp.resultado not in ('exito', 'aceptable'))
group by tc.nombre, e.nombre
order by ingresos desc
limit 3;
```

Objetivo 6 — Los 3 tipos de cirugía con mayores ingresos

PostgreSQL 18 · base hospital · conectado con el rol de solo lectura: visor

```
select tc.nombre as tipo_cirugia,
       e.nombre as especialidad,
       count(*) as cirugias,
       sum(f.total) as ingresos
from cirugia ci
inner join agendamiento_quirurgico a on a.id_agendamiento = ci.id_agendamiento
inner join tipo_cirugia tc on tc.id_tipo_cirugia = a.id_tipo_cirugia
inner join especialidad e on e.id_especialidad = tc.id_especialidad
inner join factura f on f.id_cirugia = ci.id_cirugia
where ci.condicion_evolucion = 'sin_complicaciones'
  and ci.fecha_hora_inicio between '2026-01-01' and '2026-10-31 23:59:59'
  and not exists (select 1
                  from registro_procedimiento rp
                  inner join aspecto_quirurgico aq on aq.id_aspecto = rp.id_aspecto
                  where rp.id_cirugia = ci.id_cirugia
                    and aq.etapa = 'intraoperatorio'
                    and rp.resultado not in ('exito', 'aceptable'))
group by tc.nombre, e.nombre
order by ingresos desc
limit 3;
```

tipo_cirugia	especialidad	cirugias	ingresos
Hernioplastia inguinal	Cirugía General	3	25248.50
Apendicectomía abierta	Cirugía General	3	23265.00
Revascularización miocárdica	Cirugía Cardiovascular	1	12920.00

3 filas · 60 ms

Objetivo 7. Hospitales con mayores ingresos en hospitalización

con menos camillas asignadas y menor costo de servicio. Se juntan los tres criterios en el ingreso por camilla (lo facturado / camillas usadas) lapso: enero a octubre de 2026

```
select h.nombre as hospital,
       sum(f.total) as ingresos,
       count(distinct i.id_camilla) as camillas_asignadas,
       round(avg(s.costo), 2) as costo_promedio_servicio,
       round(sum(f.total) / count(distinct i.id_camilla), 2) as ingreso_por_camilla
from factura f
inner join ingreso i on i.id_ingreso = f.id_ingreso
inner join unidad_medica u on u.id_unidad = i.id_unidad
inner join hospital h on h.id_hospital = u.id_hospital
inner join servicio_atencion s on s.id_servicio = i.id_servicio
where u.tipo_unidad = 'Hospitalización'
  and f.fecha_emision between '2026-01-01' and '2026-10-31 23:59:59'
group by h.nombre
order by ingreso_por_camilla desc, costo_promedio_servicio
limit 3;
```

Objetivo 7 — Los 3 hospitales con mayores ingresos en hospitalización

PostgreSQL 18 · base hospital · conectado con el rol de solo lectura: visor

```
select h.nombre as hospital,
       sum(f.total) as ingresos,
       count(distinct i.id_camilla) as camillas_asignadas,
       round(avg(s.costo), 2) as costo_promedio_servicio,
       round(sum(f.total) / count(distinct i.id_camilla), 2) as ingreso_por_camilla
from factura f
inner join ingreso i on i.id_ingreso = f.id_ingreso
inner join unidad_medica u on u.id_unidad = i.id_unidad
inner join hospital h on h.id_hospital = u.id_hospital
inner join servicio_atencion s on s.id_servicio = i.id_servicio
where u.tipo_unidad = 'Hospitalización'
  and f.fecha_emision between '2026-01-01' and '2026-10-31 23:59:59'
group by h.nombre
order by ingreso_por_camilla desc, costo_promedio_servicio
limit 3;
```

hospital	ingresos	camillas_asignadas	costo_promedio_servicio	ingreso_por_camilla
Hospital General de Occidente	60845.00	11	1661.54	5531.36
Hospital Infantil de Retalhuleu	48140.00	9	1546.36	5348.89
Hospital de Occidente Sololá	36624.00	9	1320.00	4069.33

3 filas · 60 ms

Objetivo 8. Pagos por método de pago

en cada hospital, sumando los pagos de cada una de sus unidades. La unidad de cada factura sale de su origen: la clinica de la cita, la unidad del ingreso o el quirofano de la cirugia (cada factura tiene solo uno). lapso: enero a octubre de 2026

```
select h.nombre as hospital,
       u.tipo_unidad as unidad,
       sum(case when mp.nombre = 'Efectivo' then p.monto else 0 end) as efectivo,
       sum(case when mp.nombre = 'Tarjeta de crédito' then p.monto else 0 end) as tarjeta_credito,
       sum(case when mp.nombre = 'Tarjeta de débito' then p.monto else 0 end) as tarjeta_debito
from pago p
inner join metodo_pago mp on mp.id_metodo_pago = p.id_metodo_pago
inner join factura f on f.id_factura = p.id_factura
left join cita c on c.id_cita = f.id_cita
left join clinica cl on cl.id_clinica = c.id_clinica
left join ingreso i on i.id_ingreso = f.id_ingreso
left join cirugia ci on ci.id_cirugia = f.id_cirugia
left join quirofano q on q.id_quirofano = ci.id_quirofano
inner join unidad_medica u on u.id_unidad in (cl.id_unidad, i.id_unidad, q.id_unidad)
inner join hospital h on h.id_hospital = u.id_hospital
where p.fecha_pago between '2026-01-01' and '2026-10-31'
group by h.id_hospital, h.nombre, u.tipo_unidad
order by h.id_hospital, u.tipo_unidad;
```

Objetivo 8 — Pagos en efectivo, crédito y débito por hospital y unidad

PostgreSQL 18 · base hospital · conectado con el rol de solo lectura: visor

```
select h.nombre as hospital,
       u.tipo_unidad as unidad,
       sum(case when mp.nombre = 'Efectivo' then p.monto else 0 end) as efectivo,
       sum(case when mp.nombre = 'Tarjeta de crédito' then p.monto else 0 end) as tarjeta_credito,
       sum(case when mp.nombre = 'Tarjeta de débito' then p.monto else 0 end) as tarjeta_debito
from pago p
inner join metodo_pago mp on mp.id_metodo_pago = p.id_metodo_pago
inner join factura f on f.id_factura = p.id_factura
left join cita c on c.id_cita = f.id_cita
left join clinica cl on cl.id_clinica = c.id_clinica
left join ingreso i on i.id_ingreso = f.id_ingreso
left join cirugia ci on ci.id_cirugia = f.id_cirugia
left join quirofano q on q.id_quirofano = ci.id_quirofano
inner join unidad_medica u on u.id_unidad in (cl.id_unidad, i.id_unidad, q.id_unidad)
inner join hospital h on h.id_hospital = u.id_hospital
where p.fecha_pago between '2026-01-01' and '2026-10-31'
group by h.id_hospital, h.nombre, u.tipo_unidad
order by h.id_hospital, u.tipo_unidad;
```

hospital	unidad	efectivo	tarjeta_credito	tarjeta_debito
Hospital General de Occidente	Cirugía	16931.80	37625.24	14796.35
Hospital General de Occidente	Consulta Externa	2940.75	3511.25	2710.75
Hospital General de Occidente	Emergencias	9633.33	13433.33	9633.34
Hospital General de Occidente	Hospitalización	13238.13	11242.98	13429.67
Hospital Regional de San Marcos	Cirugía	10512.24	8744.64	23555.32
Hospital Regional de San Marcos	Consulta Externa	2440.00	1702.50	1981.25
Hospital Regional de San Marcos	Emergencias	7154.17	7593.33	5610.01
Hospital Regional de San Marcos	Hospitalización	6092.32	6909.17	12958.62
Hospital Infantil de Retalhuleu	Cirugía	16563.42	15600.64	15573.93
Hospital Infantil de Retalhuleu	Consulta Externa	1197.50	1563.75	1074.25
Hospital Infantil de Retalhuleu	Emergencias	6487.50	8557.50	7432.50
Hospital Infantil de Retalhuleu	Hospitalización	6516.35	7136.37	7756.63
Hospital de Occidente Huehuetenango	Cirugía	9148.91	11772.30	9224.79
Hospital de Occidente Huehuetenango	Consulta Externa	883.75	1091.50	1261.25
Hospital de Occidente Huehuetenango	Emergencias	5966.67	6686.67	6226.66
Hospital de Occidente Huehuetenango	Hospitalización	8021.85	9982.35	10436.12
Hospital de Occidente Sololá	Cirugía	3892.38	5543.74	3892.38
Hospital de Occidente Sololá	Consulta Externa	1191.75	891.00	1120.00
Hospital de Occidente Sololá	Emergencias	5412.50	6962.50	5287.50
Hospital de Occidente Sololá	Hospitalización	10005.63	7307.14	9449.32

20 filas · 60 ms

Objetivo 9. Hospitales con mayor ingreso por visita

visita = ficha de ingreso (emergencias, cirugia, hospitalizacion) o cita no cancelada. Las consultas se muestran pero no se suman porque ya estan contadas en su cita. El hospital de cada factura sale de su origen: la clinica de la cita, la unidad del ingreso o el quirofano de la cirugia (cada factura tiene solo uno). lapso: enero a octubre de 2026

```
select hospital, ingresos_fichas, citas, consultas,
       ingresos_fichas + citas as visitas,
       monto_facturado,
       round(monto_facturado / (ingresos_fichas + citas), 2) as ingreso_por_visita
from (select h.nombre as hospital,
             (select count(*)
              from ingreso i
              inner join unidad_medica u on u.id_unidad = i.id_unidad
              where u.id_hospital = h.id_hospital
                and i.fecha_hora_ingreso between '2026-01-01' and '2026-10-31 23:59:59') as ingresos_fichas,
             (select count(*)
              from cita c
              inner join medico m on m.id_medico = c.id_medico
              where m.id_hospital = h.id_hospital
                and c.estado <> 'cancelada'
                and c.fecha_hora between '2026-01-01' and '2026-10-31 23:59:59') as citas,
             (select count(*)
              from consulta co
              inner join cita c on c.id_cita = co.id_cita
              inner join medico m on m.id_medico = c.id_medico
              where m.id_hospital = h.id_hospital
                and c.fecha_hora between '2026-01-01' and '2026-10-31 23:59:59') as consultas,
             (select coalesce(sum(f.total), 0)
              from factura f
              left join cita c on c.id_cita = f.id_cita
              left join clinica cl on cl.id_clinica = c.id_clinica
              left join ingreso i on i.id_ingreso = f.id_ingreso
              left join cirugia ci on ci.id_cirugia = f.id_cirugia
              left join quirofano q on q.id_quirofano = ci.id_quirofano
              inner join unidad_medica u on u.id_unidad in (cl.id_unidad, i.id_unidad, q.id_unidad)
              where u.id_hospital = h.id_hospital
                and f.fecha_emision between '2026-01-01' and '2026-10-31 23:59:59') as monto_facturado
      from hospital h) datos
order by ingreso_por_visita desc
limit 3;
```

Objetivo 9 — Los 3 hospitales con mayor ingreso por visita

PostgreSQL 18 · base hospital · conectado con el rol de solo lectura: visor

```
select hospital, ingresos_fichas, citas, consultas,
       ingresos_fichas + citas as visitas,
       monto_facturado,
       round(monto_facturado / (ingresos_fichas + citas), 2) as ingreso_por_visita
from (select h.nombre as hospital,
             (select count(*)
              from ingreso i
              inner join unidad_medica u on u.id_unidad = i.id_unidad
              where u.id_hospital = h.id_hospital
                and i.fecha_hora_ingreso between '2026-01-01' and '2026-10-31 23:59:59') as ingresos_fichas,
             (select count(*)
              from cita c
              inner join medico m on m.id_medico = c.id_medico
              where m.id_hospital = h.id_hospital
                and c.estado <> 'cancelada'
                and c.fecha_hora between '2026-01-01' and '2026-10-31 23:59:59') as citas,
             (select count(*)
              from consulta co
              inner join cita c on c.id_cita = co.id_cita
              inner join medico m on m.id_medico = c.id_medico
              where m.id_hospital = h.id_hospital
                and c.fecha_hora between '2026-01-01' and '2026-10-31 23:59:59') as consultas,
             (select coalesce(sum(f.total), 0)
              from factura f
              left join cita c on c.id_cita = f.id_cita
              left join clinica cl on cl.id_clinica = c.id_clinica
              left join ingreso i on i.id_ingreso = f.id_ingreso
              left join cirugia ci on ci.id_cirugia = f.id_cirugia
              left join quirofano q on q.id_quirofano = ci.id_quirofano
              inner join unidad_medica u on u.id_unidad in (cl.id_unidad, i.id_unidad, q.id_unidad)
              where u.id_hospital = h.id_hospital
                and f.fecha_emision between '2026-01-01' and '2026-10-31 23:59:59') as monto_facturado
      from hospital h) datos
order by ingreso_por_visita desc
limit 3;
```

hospital	ingresos_fichas	citas	consultas	visitas	monto_facturado	ingreso_por_visita
Hospital Infantil de Retalhuleu	51	31	27	82	159605.75	1946.41
Hospital General de Occidente	61	77	66	138	213519.50	1547.24
Hospital Regional de San Marcos	52	44	40	96	147816.00	1539.75

3 filas · 69 ms

Objetivo 10. Fecha con más decesos

y la unidad medica donde se han registrado mas. Se resuelve con dos consultas. En cada una se cuentan los decesos por grupo y el HAVING se queda con el grupo que tiene el maximo de su hospital (si hay empate, salen todos los empatados). No lleva lapso: el objetivo pide la fecha historica.

```
select h.nombre as hospital,
       e.fecha_hora_egreso::date as fecha,
       count(*) as decesos
from egreso e
inner join ingreso i on i.id_ingreso = e.id_ingreso
inner join unidad_medica u on u.id_unidad = i.id_unidad
inner join hospital h on h.id_hospital = u.id_hospital
where e.codigo_egreso = 'muerto'
group by h.id_hospital, h.nombre, e.fecha_hora_egreso::date
having count(*) = (select max(total)
                   from (select count(*) as total
                         from egreso e2
                         inner join ingreso i2 on i2.id_ingreso = e2.id_ingreso
                         inner join unidad_medica u2 on u2.id_unidad = i2.id_unidad
                         where e2.codigo_egreso = 'muerto'
                           and u2.id_hospital = h.id_hospital
                         group by e2.fecha_hora_egreso::date) por_fecha)
order by decesos desc, hospital, fecha;

-- unidad medica con mas decesos de cada hospital
select h.nombre as hospital,
       u.tipo_unidad as unidad,
       count(*) as decesos
from egreso e
inner join ingreso i on i.id_ingreso = e.id_ingreso
inner join unidad_medica u on u.id_unidad = i.id_unidad
inner join hospital h on h.id_hospital = u.id_hospital
where e.codigo_egreso = 'muerto'
group by h.id_hospital, h.nombre, u.tipo_unidad
having count(*) = (select max(total)
                   from (select count(*) as total
                         from egreso e2
                         inner join ingreso i2 on i2.id_ingreso = e2.id_ingreso
                         inner join unidad_medica u2 on u2.id_unidad = i2.id_unidad
                         where e2.codigo_egreso = 'muerto'
                           and u2.id_hospital = h.id_hospital
                         group by u2.tipo_unidad) por_unidad)
order by decesos desc, hospital, unidad;
```

Objetivo 10 — Fecha con más decesos y unidad con más decesos por h

PostgreSQL 18 · base hospital · conectado con el rol de solo lectura: visor

```
select h.nombre as hospital,
       e.fecha_hora_egreso::date as fecha,
       count(*) as decesos
from egreso e
inner join ingreso i on i.id_ingreso = e.id_ingreso
inner join unidad_medica u on u.id_unidad = i.id_unidad
inner join hospital h on h.id_hospital = u.id_hospital
where e.codigo_egreso = 'muerto'
group by h.id_hospital, h.nombre, e.fecha_hora_egreso::date
having count(*) = (select max(total)
                   from (select count(*) as total
                         from egreso e2
                         inner join ingreso i2 on i2.id_ingreso = e2.id_ingreso
                         inner join unidad_medica u2 on u2.id_unidad = i2.id_unidad
                         where e2.codigo_egreso = 'muerto'
                           and u2.id_hospital = h.id_hospital
                         group by e2.fecha_hora_egreso::date) por_fecha)
order by decesos desc, hospital, fecha;
```

hospital	fecha	decesos
Hospital General de Occidente	2026-06-14	2
Hospital Regional de San Marcos	2026-03-21	2
Hospital de Occidente Huehuetenango	2026-02-23	1
Hospital de Occidente Huehuetenango	2026-07-24	1
Hospital de Occidente Sololá	2026-01-31	1
Hospital de Occidente Sololá	2026-07-20	1
Hospital Infantil de Retalhuleu	2026-04-15	1
Hospital Infantil de Retalhuleu	2026-08-16	1

8 filas · 58 ms

```
select h.nombre as hospital,
       u.tipo_unidad as unidad,
       count(*) as decesos
from egreso e
inner join ingreso i on i.id_ingreso = e.id_ingreso
inner join unidad_medica u on u.id_unidad = i.id_unidad
inner join hospital h on h.id_hospital = u.id_hospital
where e.codigo_egreso = 'muerto'
group by h.id_hospital, h.nombre, u.tipo_unidad
having count(*) = (select max(total)
                   from (select count(*) as total
                         from egreso e2
                         inner join ingreso i2 on i2.id_ingreso = e2.id_ingreso
                         inner join unidad_medica u2 on u2.id_unidad = i2.id_unidad
                         where e2.codigo_egreso = 'muerto'
                           and u2.id_hospital = h.id_hospital
                         group by u2.tipo_unidad) por_unidad)
order by decesos desc, hospital, unidad;
```

hospital	unidad	decesos
Hospital General de Occidente	Emergencias	3
Hospital Regional de San Marcos	Emergencias	3
Hospital de Occidente Huehuetenango	Emergencias	2
Hospital de Occidente Sololá	Emergencias	1
Hospital de Occidente Sololá	Hospitalización	1
Hospital Infantil de Retalhuleu	Emergencias	1
Hospital Infantil de Retalhuleu	Hospitalización	1

7 filas · 59 ms

Objetivo 11. Ingresos totales por hospital

lo facturado (servicios prestados), lo cobrado (cuotas pagadas) y lo pendiente. El hospital de cada factura sale de su origen: la clinica de la cita, la unidad del ingreso o el quirofano de la cirugia (cada factura tiene solo uno). lapso: enero a octubre de 2026

```
select hospital, facturas, total_facturado, total_cobrado,
       total_facturado - total_cobrado as pendiente_de_cobro
from (select h.nombre as hospital,
             (select count(*)
              from factura f
              left join cita c on c.id_cita = f.id_cita
              left join clinica cl on cl.id_clinica = c.id_clinica
              left join ingreso i on i.id_ingreso = f.id_ingreso
              left join cirugia ci on ci.id_cirugia = f.id_cirugia
              left join quirofano q on q.id_quirofano = ci.id_quirofano
              inner join unidad_medica u on u.id_unidad in (cl.id_unidad, i.id_unidad, q.id_unidad)
              where u.id_hospital = h.id_hospital
                and f.fecha_emision between '2026-01-01' and '2026-10-31 23:59:59') as facturas,
             (select coalesce(sum(f.total), 0)
              from factura f
              left join cita c on c.id_cita = f.id_cita
              left join clinica cl on cl.id_clinica = c.id_clinica
              left join ingreso i on i.id_ingreso = f.id_ingreso
              left join cirugia ci on ci.id_cirugia = f.id_cirugia
              left join quirofano q on q.id_quirofano = ci.id_quirofano
              inner join unidad_medica u on u.id_unidad in (cl.id_unidad, i.id_unidad, q.id_unidad)
              where u.id_hospital = h.id_hospital
                and f.fecha_emision between '2026-01-01' and '2026-10-31 23:59:59') as total_facturado,
             (select coalesce(sum(p.monto), 0)
              from pago p
              inner join factura f on f.id_factura = p.id_factura
              left join cita c on c.id_cita = f.id_cita
              left join clinica cl on cl.id_clinica = c.id_clinica
              left join ingreso i on i.id_ingreso = f.id_ingreso
              left join cirugia ci on ci.id_cirugia = f.id_cirugia
              left join quirofano q on q.id_quirofano = ci.id_quirofano
              inner join unidad_medica u on u.id_unidad in (cl.id_unidad, i.id_unidad, q.id_unidad)
              where u.id_hospital = h.id_hospital
                and p.fecha_pago between '2026-01-01' and '2026-10-31') as total_cobrado
      from hospital h) datos
order by total_facturado desc;
```

Objetivo 11 — Ingresos monetarios totales de cada hospital

PostgreSQL 18 · base hospital · conectado con el rol de solo lectura: visor

```
select hospital, facturas, total_facturado, total_cobrado,
       total_facturado - total_cobrado as pendiente_de_cobro
from (select h.nombre as hospital,
             (select count(*)
              from factura f
              left join cita c on c.id_cita = f.id_cita
              left join clinica cl on cl.id_clinica = c.id_clinica
              left join ingreso i on i.id_ingreso = f.id_ingreso
              left join cirugia ci on ci.id_cirugia = f.id_cirugia
              left join quirofano q on q.id_quirofano = ci.id_quirofano
              inner join unidad_medica u on u.id_unidad in (cl.id_unidad, i.id_unidad, q.id_unidad)
              where u.id_hospital = h.id_hospital
                and f.fecha_emision between '2026-01-01' and '2026-10-31 23:59:59') as facturas,
             (select coalesce(sum(f.total), 0)
              from factura f
              left join cita c on c.id_cita = f.id_cita
              left join clinica cl on cl.id_clinica = c.id_clinica
              left join ingreso i on i.id_ingreso = f.id_ingreso
              left join cirugia ci on ci.id_cirugia = f.id_cirugia
              left join quirofano q on q.id_quirofano = ci.id_quirofano
              inner join unidad_medica u on u.id_unidad in (cl.id_unidad, i.id_unidad, q.id_unidad)
              where u.id_hospital = h.id_hospital
                and f.fecha_emision between '2026-01-01' and '2026-10-31 23:59:59') as total_facturado,
             (select coalesce(sum(p.monto), 0)
              from pago p
              inner join factura f on f.id_factura = p.id_factura
              left join cita c on c.id_cita = f.id_cita
              left join clinica cl on cl.id_clinica = c.id_clinica
              left join ingreso i on i.id_ingreso = f.id_ingreso
              left join cirugia ci on ci.id_cirugia = f.id_cirugia
              left join quirofano q on q.id_quirofano = ci.id_quirofano
              inner join unidad_medica u on u.id_unidad in (cl.id_unidad, i.id_unidad, q.id_unidad)
              where u.id_hospital = h.id_hospital
                and p.fecha_pago between '2026-01-01' and '2026-10-31') as total_cobrado
      from hospital h) datos
order by total_facturado desc;
```

hospital	facturas	total_facturado	total_cobrado	pendiente_de_cobro
Hospital General de Occidente	120	213519.50	157525.35	55994.15
Hospital Infantil de Retalhuleu	72	159605.75	106041.59	53564.16
Hospital Regional de San Marcos	87	147816.00	122884.32	24931.68
Hospital de Occidente Huehuetenango	63	101871.50	84182.82	17688.68
Hospital de Occidente Sololá	61	75242.75	62418.34	12824.41

5 filas · 67 ms

Objetivo 12. Edad y sexo en emergencias por turno

de cada hospital, agrupados por turno (jornada diurna o nocturna). La edad es la que tenia el paciente el dia que ingreso
lapso: enero a octubre de 2026

```
select h.nombre as hospital,
       t.jornada,
       count(*) as ingresos,
       round(avg(extract(year from age(i.fecha_hora_ingreso::date, p.fecha_nacimiento))), 1) as edad_promedio,
       count(case when p.sexo = 'Masculino' then 1 end) as masculino,
       count(case when p.sexo = 'Femenino' then 1 end) as femenino
from ingreso i
inner join turno_emergencia t on t.id_turno = i.id_turno
inner join unidad_medica u on u.id_unidad = i.id_unidad
inner join hospital h on h.id_hospital = u.id_hospital
inner join paciente p on p.id_paciente = i.id_paciente
where i.fecha_hora_ingreso between '2026-01-01' and '2026-10-31 23:59:59'
group by h.id_hospital, h.nombre, t.jornada
order by h.id_hospital, t.jornada;
```

Objetivo 12 — Edad promedio y sexo en emergencias por hospital y turno

PostgreSQL 18 · base hospital · conectado con el rol de solo lectura: visor

```
select h.nombre as hospital,
       t.jornada,
       count(*) as ingresos,
       round(avg(extract(year from age(i.fecha_hora_ingreso::date, p.fecha_nacimiento))), 1) as edad_promedio,
       count(case when p.sexo = 'Masculino' then 1 end) as masculino,
       count(case when p.sexo = 'Femenino' then 1 end) as femenino
from ingreso i
inner join turno_emergencia t on t.id_turno = i.id_turno
inner join unidad_medica u on u.id_unidad = i.id_unidad
inner join hospital h on h.id_hospital = u.id_hospital
inner join paciente p on p.id_paciente = i.id_paciente
where i.fecha_hora_ingreso between '2026-01-01' and '2026-10-31 23:59:59'
group by h.id_hospital, h.nombre, t.jornada
order by h.id_hospital, t.jornada;
```

hospital	jornada	ingresos	edad_promedio	masculino	femenino
Hospital General de Occidente	diurno	16	41.9	8	8
Hospital General de Occidente	nocturno	19	38.7	9	10
Hospital Regional de San Marcos	diurno	17	38.5	9	8
Hospital Regional de San Marcos	nocturno	15	42.4	9	6
Hospital Infantil de Retalhuleu	diurno	15	42.6	7	8
Hospital Infantil de Retalhuleu	nocturno	15	39.7	6	9
Hospital de Occidente Huehuetenango	diurno	15	45.7	9	6
Hospital de Occidente Huehuetenango	nocturno	15	46.1	8	7
Hospital de Occidente Sololá	diurno	15	39.5	7	8
Hospital de Occidente Sololá	nocturno	15	38.5	7	8

10 filas · 56 ms

Objetivo 13. Reporte de citas

ordenado por fecha y agrupado por sexo del paciente y especialidad del medico. lapso: enero a octubre de 2026

```
select h.nombre as hospital,
       c.fecha_hora::date as fecha,
       e.nombre as especialidad,
       p.sexo,
       count(*) as citas
from cita c
inner join paciente p on p.id_paciente = c.id_paciente
inner join medico m on m.id_medico = c.id_medico
inner join especialidad e on e.id_especialidad = m.id_especialidad
inner join hospital h on h.id_hospital = m.id_hospital
where c.fecha_hora between '2026-01-01' and '2026-10-31 23:59:59'
group by h.id_hospital, h.nombre, c.fecha_hora::date, e.nombre, p.sexo
order by h.id_hospital, fecha, especialidad, p.sexo;
```

Objetivo 13 — Citas por hospital, fecha, especialidad y sexo (primer tr

PostgreSQL 18 · base hospital · conectado con el rol de solo lectura: visor

```
select h.nombre as hospital,
       c.fecha_hora::date as fecha,
       e.nombre as especialidad,
       p.sexo,
       count(*) as citas
from cita c
inner join paciente p on p.id_paciente = c.id_paciente
inner join medico m on m.id_medico = c.id_medico
inner join especialidad e on e.id_especialidad = m.id_especialidad
inner join hospital h on h.id_hospital = m.id_hospital
where c.fecha_hora between '2026-01-01' and '2026-10-31 23:59:59'
group by h.id_hospital, h.nombre, c.fecha_hora::date, e.nombre, p.sexo
order by h.id_hospital, fecha, especialidad, p.sexo;
```

hospital	fecha	especialidad	sexo	citas
Hospital General de Occidente	2026-01-08	Terapia del Lenguaje	Femenino	1
Hospital General de Occidente	2026-01-10	Medicina Física y Rehabilitación	Femenino	1
Hospital General de Occidente	2026-01-11	Fisioterapia	Femenino	1
Hospital General de Occidente	2026-01-16	Medicina General	Femenino	1
Hospital General de Occidente	2026-01-18	Dermatología	Femenino	1
Hospital General de Occidente	2026-01-23	Terapia del Lenguaje	Masculino	1
Hospital General de Occidente	2026-01-25	Cardiología	Masculino	1
Hospital General de Occidente	2026-01-31	Medicina General	Masculino	1
Hospital General de Occidente	2026-02-01	Medicina General	Masculino	1
Hospital General de Occidente	2026-02-06	Psicología	Femenino	1
Hospital General de Occidente	2026-02-07	Ginecología Oncológica	Femenino	1
Hospital General de Occidente	2026-02-09	Cardiología	Femenino	1
Hospital General de Occidente	2026-02-15	Pediatría	Femenino	1
Hospital General de Occidente	2026-02-22	Ginecología Oncológica	Masculino	1
Hospital General de Occidente	2026-02-23	Medicina General	Masculino	1
Hospital General de Occidente	2026-02-28	Medicina Física y Rehabilitación	Masculino	1
Hospital General de Occidente	2026-03-01	Fisioterapia	Masculino	1
Hospital General de Occidente	2026-03-08	Dermatología	Femenino	1
Hospital General de Occidente	2026-03-13	Terapia del Lenguaje	Femenino	1
Hospital General de Occidente	2026-03-16	Fisioterapia	Femenino	1
Hospital General de Occidente	2026-03-21	Medicina General	Masculino	1
Hospital General de Occidente	2026-03-22	Medicina General	Masculino	1
Hospital General de Occidente	2026-03-27	Psicología	Masculino	1
Hospital General de Occidente	2026-03-30	Cardiología	Masculino	1
Hospital General de Occidente	2026-04-06	Medicina General	Femenino	1

225 filas · 62 ms · se muestran las primeras 25

Objetivo 14. Promedio de cuotas por unidad

(segun el plan de cuotas de cada factura emitida) La unidad de cada factura sale de su origen: la clinica de la cita, la unidad del ingreso o el quirofano de la cirugia (cada factura tiene solo uno). lapso: enero a octubre de 2026

```
select h.nombre as hospital,
       u.tipo_unidad as unidad,
       count(*) as facturas,
       round(avg(f.numero_cuotas), 2) as promedio_cuotas,
       min(f.numero_cuotas) as minimo_cuotas,
       max(f.numero_cuotas) as maximo_cuotas
from factura f
left join cita c on c.id_cita = f.id_cita
left join clinica cl on cl.id_clinica = c.id_clinica
left join ingreso i on i.id_ingreso = f.id_ingreso
left join cirugia ci on ci.id_cirugia = f.id_cirugia
left join quirofano q on q.id_quirofano = ci.id_quirofano
inner join unidad_medica u on u.id_unidad in (cl.id_unidad, i.id_unidad, q.id_unidad)
inner join hospital h on h.id_hospital = u.id_hospital
where f.fecha_emision between '2026-01-01' and '2026-10-31 23:59:59'
group by h.id_hospital, h.nombre, u.tipo_unidad
order by h.id_hospital, u.tipo_unidad;
```

Objetivo 14 — Promedio de cuotas de pago por unidad en cada hospital

PostgreSQL 18 · base hospital · conectado con el rol de solo lectura: visor

```
select h.nombre as hospital,
       u.tipo_unidad as unidad,
       count(*) as facturas,
       round(avg(f.numero_cuotas), 2) as promedio_cuotas,
       min(f.numero_cuotas) as minimo_cuotas,
       max(f.numero_cuotas) as maximo_cuotas
from factura f
left join cita c on c.id_cita = f.id_cita
left join clinica cl on cl.id_clinica = c.id_clinica
left join ingreso i on i.id_ingreso = f.id_ingreso
left join cirugia ci on ci.id_cirugia = f.id_cirugia
left join quirofano q on q.id_quirofano = ci.id_quirofano
inner join unidad_medica u on u.id_unidad in (cl.id_unidad, i.id_unidad, q.id_unidad)
inner join hospital h on h.id_hospital = u.id_hospital
where f.fecha_emision between '2026-01-01' and '2026-10-31 23:59:59'
group by h.id_hospital, h.nombre, u.tipo_unidad
order by h.id_hospital, u.tipo_unidad;
```

hospital	unidad	facturas	promedio_cuotas	minimo_cuotas	maximo_cuotas
Hospital General de Occidente	Cirugía	12	6.92	1	12
Hospital General de Occidente	Consulta Externa	65	1.00	1	1
Hospital General de Occidente	Emergencias	30	2.50	1	4
Hospital General de Occidente	Hospitalización	13	6.46	2	12
Hospital Regional de San Marcos	Cirugía	8	5.75	1	12
Hospital Regional de San Marcos	Consulta Externa	40	1.00	1	1
Hospital Regional de San Marcos	Emergencias	28	2.57	1	4
Hospital Regional de San Marcos	Hospitalización	11	6.64	1	12
Hospital Infantil de Retalhuleu	Cirugía	9	8.56	6	12
Hospital Infantil de Retalhuleu	Consulta Externa	27	1.00	1	1
Hospital Infantil de Retalhuleu	Emergencias	25	2.52	1	4
Hospital Infantil de Retalhuleu	Hospitalización	11	6.27	1	12
Hospital de Occidente Huehuetenango	Cirugía	5	4.40	1	8
Hospital de Occidente Huehuetenango	Consulta Externa	22	1.00	1	1
Hospital de Occidente Huehuetenango	Emergencias	26	2.50	1	4
Hospital de Occidente Huehuetenango	Hospitalización	10	7.20	2	12
Hospital de Occidente Sololá	Cirugía	2	3.50	3	4
Hospital de Occidente Sololá	Consulta Externa	23	1.00	1	1
Hospital de Occidente Sololá	Emergencias	26	2.42	1	4
Hospital de Occidente Sololá	Hospitalización	10	6.40	1	12

20 filas · 62 ms

Bloque B — Extensión del modelo

Objetivo 15. Extensión del modelo

La extensión se entrega como migración en sql/ddl/01_personal.sql (supertipo PERSONAL y subtipos) y sql/ddl/02_jerarquia.sql (cadena de mando); el modelo y la estrategia están en la sección 2 y el diccionario en la sección 3. La captura muestra que cada persona tiene su subtipo y la cadena de mando del hospital 1.

Objetivo 15 — Extensión del modelo: supertipo PERSONAL y cadena de mando

PostgreSQL 18 · base hospital · conectado con el rol de solo lectura: visor

```
select p.tipo_personal, count(*) as personas, count(m.id_medico) as en_medico, count(e.id_enfermero) as en_enfermero
from personal p
left join medico m on m.id_medico = p.id_personal
left join enfermero e on e.id_enfermero = p.id_personal
group by p.tipo_personal;
```

tipo_personal	personas	en_medico	en_enfermero
medico	51	51	0
enfermero	20	0	20

2 filas · 47 ms

```
select p.id_personal, p.nombres || ' ' || p.apellidos as medico, m.cargo,
       s.nombres || ' ' || s.apellidos as superior_directo
from personal p
inner join medico m on m.id_medico = p.id_personal
left join medico s on s.id_medico = m.id_medico_superior
where p.id_hospital = 1 and m.tipo_medico <> 'externo'
order by m.cargo, p.id_personal;
```

id_personal	medico	cargo	superior_directo
1002	Roberto Castillo Méndez	director_medico	
1005	Miguel Barrios Donis	jefe_servicio	Roberto Castillo Méndez
1011	Marcos Aguilar Salazar	jefe_servicio	Roberto Castillo Méndez
1001	Carmen Ruiz Palacios	medico	Miguel Barrios Donis
1003	Andrea Soto Fuentes	medico	Marcos Aguilar Salazar
1004	Julia Paredes Ovalle	medico	Marcos Aguilar Salazar
1006	Alejandro Ramírez Lemus	medico	Miguel Barrios Donis
1008	Daniel Morales Peña	medico	Alejandro Ramírez Lemus
1010	Elena Vásquez Sotelo	medico	Marcos Aguilar Salazar
1013	Fernando López Arriaga	medico	Marcos Aguilar Salazar
1015	Andrés Chinchilla Paz	medico	Marcos Aguilar Salazar
1017	Ricardo Batz Coyoy	medico	Marcos Aguilar Salazar
1018	Silvia Gómez Tuch	medico	Ricardo Batz Coyoy

13 filas · 46 ms

Bloque C — Reportes sobre el personal médico

Objetivo 16. Cadena de supervisión

sin saber de antemano cuantos niveles hay entre ellos. Se usa una CTE recursiva: el caso base es el medico indicado (nivel 0) y la parte recursiva sube a su superior directo hasta llegar a quien ya no tiene superior. Para consultar otro medico solo se cambia el id del caso base.

```
with recursive cadena as (                          -- tabla temporal "cadena"; recursive = se usa a si misma para crecer

    -- CASO BASE: el medico que se quiere consultar
    select m.id_medico,                             -- el id del mesico
           m.id_medico_superior,                    -- el id de su jefe directo
           0 as nivel                               -- nivel 0 = el medico consultado
    from medico m                                   -- se buca en la tabla medico
    where m.id_medico = 1018                        -- el medico del que se parte (cambiar este id para otro medico)

    union all                                       -- junta el caso base con las filas que agrega la parte recursiva

    -- PARTE RECURSIVA: se repite sola, cada vuelta sube un nivel
    select m.id_medico,                             -- el id del jefe encontrado
           m.id_medico_superior,                    -- el jefe de ese jefe (para la siguiente vuelta)
           c.nivel + 1                              -- un nivel mas ariba que la fila anterior
    from medico m                                   -- se busca otra vez en medico
    inner join cadena c                             -- se une con lo que ya hay en "cadena"
            on m.id_medico = c.id_medico_superior   -- el medico buscado es el jefe de la ultima fila;
                                                    -- cuando llegs al director (superior null) ya no encuentra a nadie y termina
)
-- con la cadena armada se muestran los datos de cada medico
select c.nivel,                                     -- 0 = el consultado ... el mas alto = el director
       m.id_medico,                                 -- id del medico
       m.nombres,                                   -- nombres
       m.apellidos,                                 -- apellidos
       m.cargo,                                     -- medico, jefe_servicio o director_medico
       e.nombre as especialidad,                    -- nombre de su especialidad
       h.nombre as hospital                         -- nombre de su hospital
from cadena c                                       -- la tabla temporal que armo la parte de arriba
inner join medico m on m.id_medico = c.id_medico    -- se trae el nombre y cargo de cada medico de la cadena
inner join especialidad e on e.id_especialidad = m.id_especialidad   -- su especialidad
inner join hospital h on h.id_hospital = m.id_hospital               -- su hospital
order by c.nivel;                                   -- del medico consultado hasta el director

-- ============================================================
-- ejemplo 2: medico 1046 (hospital 4)
-- es la misma consulta: solo cambia el id del caso base
-- ============================================================

with recursive cadena as (                          -- taba temporl "cadena" que se usa a si misma
    -- caso base: el medico 1046 en el nivel 0
    select m.id_medico, m.id_medico_superior, 0 as nivel
    from medico m
    where m.id_medico = 1046                        -- unico cambio respecto al ejemplo 1
    union all                                       -- junta el caso base con lo que agrega la parte recursiva
    -- parte recursiva: trae al jefe de la ultima fila, un nivel mas arriba
    select m.id_medico, m.id_medico_superior, c.nivel + 1
    from medico m
    inner join cadena c on m.id_medico = c.id_medico_superior
)
select c.nivel, m.id_medico, m.nombres, m.apellidos, m.cargo,
       e.nombre as especialidad, h.nombre as hospital
from cadena c                                       -- la cadena ya armada
inner join medico m on m.id_medico = c.id_medico    -- nombre y cargo
inner join especialidad e on e.id_especialidad = m.id_especialidad   -- especialidad
inner join hospital h on h.id_hospital = m.id_hospital               -- hospital
order by c.nivel;                                   -- del medico hasta el director
```

Objetivo 16 — Cadena de supervisión hasta el director médico (CTE recursiva)

PostgreSQL 18 · base hospital · conectado con el rol de solo lectura: visor

```
with recursive cadena as (
    select m.id_medico, m.id_medico_superior, 0 as nivel
    from medico m
    where m.id_medico = 1018

    union all

    select m.id_medico, m.id_medico_superior, c.nivel + 1
    from medico m
    inner join cadena c on m.id_medico = c.id_medico_superior
)
select c.nivel, m.id_medico, m.nombres, m.apellidos, m.cargo,
       e.nombre as especialidad, h.nombre as hospital
from cadena c
inner join medico m on m.id_medico = c.id_medico                    -- nombre y cargo
inner join especialidad e on e.id_especialidad = m.id_especialidad  -- su especialidad
inner join hospital h on h.id_hospital = m.id_hospital              -- su hospital
order by c.nivel;
```

nivel	id_medico	nombres	apellidos	cargo	especialidad	hospital
0	1018	Silvia	Gómez Tuch	medico	Terapia del Lenguaje	Hospital General de Occidente
1	1017	Ricardo	Batz Coyoy	medico	Psicología	Hospital General de Occidente
2	1011	Marcos	Aguilar Salazar	jefe_servicio	Ginecología Oncológica	Hospital General de Occidente
3	1002	Roberto	Castillo Méndez	director_medico	Cardiología	Hospital General de Occidente

4 filas · 46 ms

```
-- del medico consultado (0) hasta el director (el nivel mas alto)

with recursive cadena as (
    select m.id_medico, m.id_medico_superior, 0 as nivel
    from medico m
    where m.id_medico = 1046
    union all
    select m.id_medico, m.id_medico_superior, c.nivel + 1
    from medico m
    inner join cadena c on m.id_medico = c.id_medico_superior
)
select c.nivel, m.id_medico, m.nombres, m.apellidos, m.cargo,
       e.nombre as especialidad, h.nombre as hospital
from cadena c
inner join medico m on m.id_medico = c.id_medico
inner join especialidad e on e.id_especialidad = m.id_especialidad
inner join hospital h on h.id_hospital = m.id_hospital
order by c.nivel;
```

nivel	id_medico	nombres	apellidos	cargo	especialidad	hospital
0	1046	Kevin	Domingo Pascual	medico	Anestesiología	Hospital de Occidente Huehuetenango
1	1042	Héctor Manuel	Gómez Ordóñez	medico	Medicina General	Hospital de Occidente Huehuetenango
2	1041	Ana Gabriela	Recinos Mérida	jefe_servicio	Medicina General	Hospital de Occidente Huehuetenango
3	1043	Carolina	Herrera Villatoro	director_medico	Cardiología	Hospital de Occidente Huehuetenango

4 filas · 46 ms

Objetivo 17. Ranking por hospital y global

con su posicion en su hospital y en toda la cadena. La posicion 1 es la nota mas baja. Hay dos formas que dan lo mismo, para ver cual es mas rapida (ver rendimiento/). lapso: enero a octubre de 2026

```
with promedios as (                                     -- tabla temporal con el promedio de cada medico
    select m.id_medico,                                 -- id del medico
           m.nombres || ' ' || m.apellidos as medico,   -- nombre completo (|| junta los textos)
           h.nombre as hospital,                        -- su hospital
           avg(ca.puntuacion) as promedio               -- su nota promedio (de 1 a 5)
    from calificacion ca                                -- las notas que dieron los pacientes
    inner join consulta co on co.id_consulta = ca.id_consulta   -- la consulta que se califico
    inner join cita c on c.id_cita = co.id_cita                 -- su cita, para la fecha y el medico
    inner join medico m on m.id_medico = c.id_medico            -- el medico que atendio
    inner join hospital h on h.id_hospital = m.id_hospital      -- el hospital del medico
    where c.fecha_hora between '2026-01-01' and '2026-10-31 23:59:59'   -- solo el lapso
    group by m.id_medico, m.nombres, m.apellidos, h.nombre      -- una fila por medico
)
select medico,                                          -- nombre
       hospital,                                        -- hospital
       round(promedio, 2) as calificacion_promedio,     -- promedio con 2 decimales
       rank() over (partition by hospital order by promedio) as posicion_en_hospital,   -- puesto contando solo su hospital
       rank() over (order by promedio) as posicion_en_cadena                            -- puesto contando a todos los medicos
from promedios                                          -- sale de la tabla temporal
order by posicion_en_cadena, hospital, medico;          -- del peor al mejor

-- Forma B: subconsultas correlacionadas (la posicion es 1 + cuantos medicos tienen menor promedio)
with promedios as (                                     -- la misma tabla temporal de la forma A
    select m.id_medico, m.nombres || ' ' || m.apellidos as medico, h.nombre as hospital,   -- id, nombre y hospital
           avg(ca.puntuacion) as promedio               -- nota promedio
    from calificacion ca                                -- las notas
    inner join consulta co on co.id_consulta = ca.id_consulta   -- su consulta
    inner join cita c on c.id_cita = co.id_cita                 -- su cita
    inner join medico m on m.id_medico = c.id_medico            -- el medico
    inner join hospital h on h.id_hospital = m.id_hospital      -- su hospital
    where c.fecha_hora between '2026-01-01' and '2026-10-31 23:59:59'   -- solo el lapso
    group by m.id_medico, m.nombres, m.apellidos, h.nombre      -- una fila por medico
)
-- p1 = el medico al que se le busca el puesto, p2 = los demas medicos con los que se compara.
-- puesto = 1 + cuantos van delante (los que tienen menos nota), como el lugar en una fila:
-- si nadie va delante eres el 1, si van 2 delante eres el 3
select p1.medico,                                       -- el medico al que se le busca el puesto
       p1.hospital,                                     -- su hospital
       round(p1.promedio, 2) as calificacion_promedio,  -- su promedio con 2 decimales
       1 + (select count(*) from promedios p2           -- 1 + los que van delante = su puesto en el hospital
            where p2.hospital = p1.hospital             -- el where no ordena, elige a quien contar: solo los de su hospital
              and p2.promedio < p1.promedio) as posicion_en_hospital,   -- y que tengan menos nota que el
       1 + (select count(*) from promedios p2           -- 1 + los que van delante = su puesto en la cadena
            where p2.promedio < p1.promedio) as posicion_en_cadena      -- cuenta a todos los que tienen menos nota (de cualquier
hospital)
from promedios p1                                       -- se hace para cada medico de la lista
order by posicion_en_cadena, hospital, medico;          -- este si ordena: del puesto 1 (peor nota) al ultimo
```

Objetivo 17 — Ranking por hospital y en toda la cadena (forma A y forma B)

PostgreSQL 18 · base hospital · conectado con el rol de solo lectura: visor

```
with promedios as (
    select m.id_medico, m.nombres || ' ' || m.apellidos as medico, h.nombre as hospital,
           avg(ca.puntuacion) as promedio
    from calificacion ca
    inner join consulta co on co.id_consulta = ca.id_consulta
    inner join cita c on c.id_cita = co.id_cita
    inner join medico m on m.id_medico = c.id_medico
    inner join hospital h on h.id_hospital = m.id_hospital
    where c.fecha_hora between '2026-01-01' and '2026-10-31 23:59:59'
    group by m.id_medico, m.nombres, m.apellidos, h.nombre
)
select medico, hospital, round(promedio, 2) as calificacion_promedio,
       rank() over (partition by hospital order by promedio) as posicion_en_hospital,
       rank() over (order by promedio) as posicion_en_cadena
from promedios
order by posicion_en_cadena, hospital, medico;
```

medico	hospital	calificacion_promedio	posicion_en_hospital	posicion_en_cadena
Andrés Chinchilla Paz	Hospital General de Occidente	1.25	1	1
Elena Vásquez Sotelo	Hospital General de Occidente	1.40	2	2
Mónica Pirir Cutzal	Hospital de Occidente Sololá	1.67	1	3
Marcos Aguilar Salazar	Hospital General de Occidente	1.67	3	3
Ana Gabriela Recinos Mérida	Hospital de Occidente Huehuetenango	2.00	1	5
Miguel Barrios Donis	Hospital General de Occidente	2.00	4	5
Otto René Velásquez Lima	Hospital Infantil de Retalhuleu	2.00	1	5
Paola Sandoval Itzep	Hospital Infantil de Retalhuleu	2.00	1	5
Carmen Ruiz Palacios	Hospital General de Occidente	2.20	5	9
Rosa María Fuentes Galindo	Hospital Regional de San Marcos	2.33	1	10
Lucía Cumes Xajil	Hospital de Occidente Sololá	2.50	2	11
Ricardo Batz Coyoy	Hospital General de Occidente	2.80	6	12
Gabriela Nájera Ruiz	Hospital Regional de San Marcos	2.80	2	12
Héctor Manuel Gómez Ordóñez	Hospital de Occidente Huehuetenango	3.00	2	14
Esteban Orozco Bámaca	Hospital Regional de San Marcos	3.25	3	15
Carolina Herrera Villatoro	Hospital de Occidente Huehuetenango	3.50	3	16
Brenda Sic Ixcot	Hospital Infantil de Retalhuleu	3.50	3	16
Roberto Castillo Méndez	Hospital General de Occidente	3.67	7	18
Leticia Morales Chávez	Hospital Regional de San Marcos	3.75	4	19
Diego Yaxón Bocel	Hospital de Occidente Sololá	4.00	3	20
Andrea Soto Fuentes	Hospital General de Occidente	4.00	8	20
Fernando López Arriaga	Hospital General de Occidente	4.00	8	20
Mario Cabrera Monterroso	Hospital Infantil de Retalhuleu	4.25	4	23
Gloria Tax Chiquitó	Hospital Infantil de Retalhuleu	4.33	5	24
Silvia Gómez Tuch	Hospital General de Occidente	4.50	10	25

31 filas · 49 ms · se muestran las primeras 25

```
with promedios as (
    select m.id_medico, m.nombres || ' ' || m.apellidos as medico, h.nombre as hospital,
           avg(ca.puntuacion) as promedio
    from calificacion ca
    inner join consulta co on co.id_consulta = ca.id_consulta
    inner join cita c on c.id_cita = co.id_cita
    inner join medico m on m.id_medico = c.id_medico
    inner join hospital h on h.id_hospital = m.id_hospital
    where c.fecha_hora between '2026-01-01' and '2026-10-31 23:59:59'
    group by m.id_medico, m.nombres, m.apellidos, h.nombre
)
select p1.medico, p1.hospital, round(p1.promedio, 2) as calificacion_promedio,
       1 + (select count(*) from promedios p2
            where p2.hospital = p1.hospital and p2.promedio < p1.promedio) as posicion_en_hospital,
       1 + (select count(*) from promedios p2
            where p2.promedio < p1.promedio) as posicion_en_cadena
from promedios p1
order by posicion_en_cadena, hospital, medico;
```

medico	hospital	calificacion_promedio	posicion_en_hospital	posicion_en_cadena
Andrés Chinchilla Paz	Hospital General de Occidente	1.25	1	1
Elena Vásquez Sotelo	Hospital General de Occidente	1.40	2	2
Mónica Pirir Cutzal	Hospital de Occidente Sololá	1.67	1	3
Marcos Aguilar Salazar	Hospital General de Occidente	1.67	3	3
Ana Gabriela Recinos Mérida	Hospital de Occidente Huehuetenango	2.00	1	5
Miguel Barrios Donis	Hospital General de Occidente	2.00	4	5
Otto René Velásquez Lima	Hospital Infantil de Retalhuleu	2.00	1	5
Paola Sandoval Itzep	Hospital Infantil de Retalhuleu	2.00	1	5
Carmen Ruiz Palacios	Hospital General de Occidente	2.20	5	9
Rosa María Fuentes Galindo	Hospital Regional de San Marcos	2.33	1	10
Lucía Cumes Xajil	Hospital de Occidente Sololá	2.50	2	11
Ricardo Batz Coyoy	Hospital General de Occidente	2.80	6	12
Gabriela Nájera Ruiz	Hospital Regional de San Marcos	2.80	2	12
Héctor Manuel Gómez Ordóñez	Hospital de Occidente Huehuetenango	3.00	2	14
Esteban Orozco Bámaca	Hospital Regional de San Marcos	3.25	3	15
Carolina Herrera Villatoro	Hospital de Occidente Huehuetenango	3.50	3	16
Brenda Sic Ixcot	Hospital Infantil de Retalhuleu	3.50	3	16
Roberto Castillo Méndez	Hospital General de Occidente	3.67	7	18
Leticia Morales Chávez	Hospital Regional de San Marcos	3.75	4	19
Diego Yaxón Bocel	Hospital de Occidente Sololá	4.00	3	20
Andrea Soto Fuentes	Hospital General de Occidente	4.00	8	20
Fernando López Arriaga	Hospital General de Occidente	4.00	8	20
Mario Cabrera Monterroso	Hospital Infantil de Retalhuleu	4.25	4	23
Gloria Tax Chiquitó	Hospital Infantil de Retalhuleu	4.33	5	24
Silvia Gómez Tuch	Hospital General de Occidente	4.50	10	25

31 filas · 48 ms · se muestran las primeras 25

Comparación de rendimiento (sql/consultas/rendimiento/medir_objetivo_17.sh)

Médicos calificados	Calificaciones	A: ventana (ms)	B: correlacionada (ms)	B / A
31	112	0.6	0.9	1.4x
531	10112	13.7	113.7	8.3x
2031	40112	62.3	1344.1	21.6x
5031	100112	168.8	5707.1	33.8x

Con pocos médicos las dos formas tardan lo mismo, pero al crecer los datos la forma A (funciones de ventana) crece casi en proporción a los registros, mientras que la forma B (subconsultas correlacionadas) crece con el cuadrado del número de médicos: para cada médico vuelve a recorrer a todos los demás, dos veces (hospital y cadena). La forma A ordena una sola vez cada partición, así que es la que se recomienda para un reporte que se ejecuta con frecuencia sobre un volumen

creciente.

Bloque D — Programa de fidelidad

Objetivo 18. Pacientes del programa de fidelidad

Emergencias, Cirugia y Hospitalizacion) para invitarlos al programa de fidelidad. lapso: enero a octubre de 2026

```
select p.id_paciente,
       p.numero_expediente,
       p.nombres,
       p.apellidos,
       p.sexo
from paciente p
inner join ingreso i on i.id_paciente = p.id_paciente           -- sus fichas de ingreso (emergencias, cirugia, hospitalizacion)
inner join unidad_medica u on u.id_unidad = i.id_unidad         -- para saber en que unidad fue cada ingreso
where i.fecha_hora_ingreso between '2026-01-01' and '2026-10-31 23:59:59'
  and p.id_paciente in (select c.id_paciente                    -- y que tambien haya tenido una consulta (consulta externa)
                        from consulta co
                        inner join cita c on c.id_cita = co.id_cita
                        where c.fecha_hora between '2026-01-01' and '2026-10-31 23:59:59')
group by p.id_paciente, p.numero_expediente, p.nombres, p.apellidos, p.sexo   -- una fila por paciente
having count(distinct u.tipo_unidad) = 3                        -- que sus ingresos sean de las 3 unidades distintas
order by p.id_paciente;
```

Objetivo 18 — Pacientes atendidos en las 4 unidades (división relacion

PostgreSQL 18 · base hospital · conectado con el rol de solo lectura: visor

```
select p.id_paciente,
       p.numero_expediente,
       p.nombres || ' ' || p.apellidos as paciente,
       extract(year from age(p.fecha_nacimiento))::int as edad,
       p.sexo
from paciente p
inner join ingreso i on i.id_paciente = p.id_paciente
inner join unidad_medica u on u.id_unidad = i.id_unidad
where i.fecha_hora_ingreso between '2026-01-01' and '2026-10-31 23:59:59'
  and p.id_paciente in (select c.id_paciente
                        from consulta co
                        inner join cita c on c.id_cita = co.id_cita
                        where c.fecha_hora between '2026-01-01' and '2026-10-31 23:59:59')
group by p.id_paciente, p.numero_expediente, p.nombres, p.apellidos, p.fecha_nacimiento, p.sexo
having count(distinct u.tipo_unidad) = 3
order by p.id_paciente;
```

id_paciente	numero_expediente	paciente	edad	sexo
2	HG1-0002	Jorge Luis Ramírez Sosa	36	Masculino
3	HG1-0003	Pedro Gómez Aguilar	51	Masculino
5	HG1-0005	Carlos De León Ortiz	44	Masculino
9	HG5-0009	Andrés Cruz Sic	58	Masculino
11	HG2-0011	Héctor Fuentes Chávez	34	Masculino
12	HG3-0012	Claudia Vásquez Paz	21	Femenino
18	HG4-0018	Carmen Xicará Ruiz	31	Femenino
19	HG5-0019	Luis Fernando Batz Rojas	18	Masculino
22	HG3-0022	Gabriela Pérez Tum	65	Femenino
23	HG4-0023	Pedro Morales Chávez	52	Masculino
24	HG5-0024	Verónica Hernández Paz	40	Femenino
25	HG1-0025	Ricardo Castillo Lima	28	Masculino
26	HG2-0026	Julia Ramírez Ixcot	15	Femenino
27	HG3-0027	Fernando Gómez Pop	3	Masculino
28	HG4-0028	Daniela Méndez Barrios	74	Femenino
35	HG1-0035	Diego Tzul Chávez	71	Masculino
37	HG3-0037	Jorge Ajcá Lima	46	Masculino
38	HG4-0038	Paola Xicará Ixcot	34	Femenino
40	HG1-0040	Marta García Barrios	9	Femenino
41	HG2-0041	Héctor López Chan	80	Masculino
43	HG4-0043	Rafael Morales Rojas	55	Masculino
44	HG5-0044	Elena Hernández Soto	43	Femenino
52	HG3-0052	Gabriela Vásquez Barrios	28	Femenino
53	HG4-0053	Pedro Juárez Chan	15	Masculino
54	HG5-0054	Verónica Sosa Ruiz	3	Femenino

30 filas · 49 ms · se muestran las primeras 25

Bloque E — Revisión del reporte de cirugías

Objetivo 19. Revisión del reporte de cirugías

Aqui reviso el reporte de cirugias que nos dieron y propongo como separarlo en tablas. El reporte viene todo en una sola tabla: Reporte_Cirugia (Id_cirugia, Nombre_cirujano, Especialidad_cirujano, Nombre_paciente, Codigo_diagnostico, Descripcion_diagnostico, Nombre_insumo_1, Cantidad_insumo_1, Nombre_insumo_2, Cantidad_insumo_2, Fecha) Que problemas tiene (una frase cada uno): * 1FN: insumo_1 e insumo_2 son un grupo repetitivo, asi que no caben mas de 2 insumos y si usa uno queda vacio el otro. * 2FN: al poner un insumo por fila la llave pasa a ser (cirugia, insumo), pero el cirujano, paciente, diagnostico y fecha solo dependen de la cirugia (media llave). * 3FN: la especialidad depende del cirujano y la descripcion depende del codigo del diagnostico, no de la cirugia. * Datos repetidos: la especialidad y la descripcion se vuelven a escribir en cada cirugia. * Al actualizar: si un cirujano cambia de especialidad hay que cambiar todas sus filas, y si falta una queda contradictorio. * Al insertar: no puedo guardar un cirujano, diagnostico o insumo nuevo hasta que salga en una cirugia. * Al borrar: si borro la unica cirugia de un diagnostico tambien pierdo su descripcion. * Identificador: usa nombres para identificar a las personas, y dos se pueden llamar igual. Mi propueta en 3FN son las 7 tablas de abajo. Es solo la propuesta: no se corre sobre la base del hospital porque ahi ya existen tablas con estos nombre.

Sentencias CREATE TABLE de la propuesta en 3FN:

```
create table especialidad (
    id_especialidad integer generated always as identity primary key,
    nombre varchar(80) not null,
    constraint uq_especialidad_nombre unique (nombre)
);

-- cirujanos: la especialidad va aqui porque depende del cirujano, no de la cirugia
create table cirujano (
    id_cirujano integer generated always as identity primary key,
    nombres varchar(100) not null,
    apellidos varchar(100) not null,
    id_especialidad integer not null references especialidad(id_especialidad)
        on delete restrict on update cascade
);

-- los pacientes
create table paciente (
    id_paciente integer generated always as identity primary key,
    nombres varchar(100) not null,
    apellidos varchar(100) not null
);

-- diagnosticos: la descripcion va aqui porque depende del codigo, no de la cirugia
create table diagnostico (
    codigo_cie10 varchar(7) primary key,
    descripcion varchar(200) not null,
    -- el codigo CIE-10 tiene que medir de 3 a 7 (de K35 a K35.81)
    constraint ck_diagnostico_codigo check (length(codigo_cie10) between 3 and 7)
);

-- los insumos, cada uno una sola vez
create table insumo (
    id_insumo integer generated always as identity primary key,
    nombre varchar(120) not null,
    constraint uq_insumo_nombre unique (nombre)
);

-- cada cirugia: quien opera, a quien, por que diagnostico y cuando
create table cirugia (
    id_cirugia integer primary key,
    id_cirujano integer not null references cirujano(id_cirujano)
        on delete restrict on update cascade,
    id_paciente integer not null references paciente(id_paciente)
        on delete restrict on update cascade,
    codigo_cie10 varchar(7) not null references diagnostico(codigo_cie10)
        on delete restrict on update cascade,
    fecha date not null
);

-- los insumos de cada cirugia, ya sin limite de 2 (tabla intermedia N:M)
create table cirugia_insumo (
    id_cirugia integer not null references cirugia(id_cirugia)
        on delete cascade on update cascade,
    id_insumo integer not null references insumo(id_insumo)
        on delete restrict on update cascade,
```

```
    cantidad smallint not null,
    primary key (id_cirugia, id_insumo),
    -- por lo menos tiene que usar 1
    constraint ck_cirugia_insumo_cantidad check (cantidad > 0)
);
```

Bloque F — Trazabilidad de cambios sensibles

Objetivo 20. Trazabilidad de cambios sensibles

Mecanismo en sql/auditoria/01_auditoria.sql: dos disparadores AFTER UPDATE, uno sobre personal (salario y nivel de acceso) y otro sobre medico (especialidad), anotan en la bitácora auditoria_personal una fila por cada campo que cambia, con quién lo cambió (current_user), el campo, el valor anterior (old), el valor nuevo (new) y la fecha y hora (now()). Así el registro es automático y no depende de la aplicación ni del usuario. La prueba (sql/auditoria/02_prueba.sql) hace 3 cambios: un aumento de salario, un cambio de especialidad y un cambio de salario y nivel de acceso en la misma sentencia; un UPDATE que deja el mismo valor no se anota.

Objetivo 20 — Bitácora automática de cambios sensibles del personal

PostgreSQL 18 · base hospital · conectado con el rol de solo lectura: visor

```
select id_auditoria, fecha_hora, usuario_bd, usuario_aplicacion, tabla, id_registro,
       campo, valor_anterior, valor_nuevo
from auditoria_personal
order by id_auditoria;
```

id_auditoria	fecha_hora	usuario_bd	usuario_aplicacion	tabla	id_registro	campo	valor_anterior	valor_nuevo
1	2026-10-20 18:13:17.681033-06	admin	rrhh.lperez	personal	1001	salario	13000.00	14500.00
2	2026-10-20 18:13:17.684541-06	admin	rrhh.lperez	medico	1015	id_especialidad	309 - Odontología General	307 - Medicina General
3	2026-10-20 18:13:17.685358-06	admin	direccion.hgo	personal	1004	salario	18500.00	22000.00
4	2026-10-20 18:13:17.685423-06	admin	direccion.hgo	personal	1004	nivel_acceso	operativo	supervisor

4 filas · 47 ms

5. ORM: objetivos 8, 11 y 18 con JPA

Se implementaron con Java 21, JPA (Hibernate 6.6) y el driver JDBC de PostgreSQL, en el proyecto Maven orm/. Las entidades son de solo lectura (@Immutable) y mapean solo las columnas usadas; Factura se mapea a la tabla factura con sus tres orígenes posibles (cita, ingreso y cirugía); la unidad y el hospital de cada factura se obtienen uniendo el origen con su clínica, su unidad o su quirófano. La conexión usa el rol visor y la contraseña se lee de la variable de entorno HOSPITAL_DB_PASSWORD. Todas las consultas JPQL usan parámetros con nombre (:desde, :hasta, :efectivo…): Hibernate los envía en sentencias preparadas, nunca concatenados al texto, por lo que no hay riesgo de inyección SQL.

Igualdad de resultados

La clase Comparar ejecuta cada objetivo con JPA y con la consulta SQL de su archivo (sql/consultas/objetivo_XX.sql, cuyas fechas se envían como parámetros enlazados :desde y :hasta) y compara fila por fila:

```
Objetivo 8   ORM:  20 filas | SQL:  20 filas -> iguales
Objetivo 11  ORM:   5 filas | SQL:   5 filas -> iguales
Objetivo 18  ORM:  30 filas | SQL:  30 filas -> iguales
```

Consultas JPQL (ConsultasOrm.java)

```
package gt.cunoc.hospital.orm;

import jakarta.persistence.EntityManager;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.OffsetDateTime;
import java.time.ZoneId;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Objetivos 8, 11 y 18 escritos con JPA (JPQL), equivalentes a las consultas SQL de
 * sql/consultas/objetivo_08.sql, objetivo_11.sql y objetivo_18.sql.
 *
 * Todos los valores que vienen de afuera (fechas, nombres de metodo de pago) se pasan como
 * parametros con nombre (:desde, :hasta, ...). Hibernate los envia en una sentencia preparada,
 * nunca concatenados al texto de la consulta, por lo que no hay riesgo de inyeccion SQL.
 */
public class ConsultasOrm {

    private static final ZoneId GUATEMALA = ZoneId.of("America/Guatemala");

    private final EntityManager em;

    public ConsultasOrm(EntityManager em) {
        this.em = em;
    }

    /** Fila del objetivo 8: pagos por metodo en una unidad de un hospital. */
    public record PagosPorMetodo(String hospital, String unidad, BigDecimal efectivo,
                                 BigDecimal tarjetaCredito, BigDecimal tarjetaDebito) {
    }

    /** Fila del objetivo 11: ingresos de un hospital. */
    public record IngresosHospital(String hospital, long facturas, BigDecimal totalFacturado,
                                   BigDecimal totalCobrado, BigDecimal pendienteDeCobro) {
    }

    /** Fila del objetivo 18: paciente atendido en las 4 unidades. */
    public record PacienteFiel(int idPaciente, String numeroExpediente, String nombres, String apellidos, String sexo) {
    }

    /** Objetivo 8: total pagado en efectivo, credito y debito por hospital y unidad. */
    public List<PagosPorMetodo> objetivo08(LocalDate desde, LocalDate hasta) {
        List<Object[]> filas = em.createQuery("""
                select h.nombre, u.tipoUnidad,
                       sum(case when m.nombre = :efectivo then p.monto else 0 end),
                       sum(case when m.nombre = :credito then p.monto else 0 end),
                       sum(case when m.nombre = :debito then p.monto else 0 end)
                  from Pago p
                  join p.metodoPago m
                  join p.factura f
                  left join f.cita c
```

```
              left join c.clinica cl
              left join f.ingreso i
              left join f.cirugia ci
              left join ci.quirofano q
              join UnidadMedica u on u.id in (cl.unidad.id, i.unidad.id, q.unidad.id)
              join u.hospital h
             where p.fechaPago between :desde and :hasta
             group by h.id, h.nombre, u.tipoUnidad
             order by h.id, u.tipoUnidad
            """, Object[].class)
            .setParameter("efectivo", "Efectivo")
            .setParameter("credito", "Tarjeta de crédito")
            .setParameter("debito", "Tarjeta de débito")
            .setParameter("desde", desde)
            .setParameter("hasta", hasta)
            .getResultList();
    List<PagosPorMetodo> resultado = new ArrayList<>();
    for (Object[] f : filas) {
        resultado.add(new PagosPorMetodo((String) f[0], (String) f[1], decimal(f[2]), decimal(f[3]), decimal(f[4])));
    }
    return resultado;
}

/** Objetivo 11: total facturado y cobrado por hospital. */
public List<IngresosHospital> objetivo11(LocalDate desde, LocalDate hasta) {
    Map<Integer, Object[]> facturado = new HashMap<>();
    for (Object[] f : em.createQuery("""
            select u.hospital.id, count(f), sum(f.total)
              from Factura f
              left join f.cita c
              left join c.clinica cl
              left join f.ingreso i
              left join f.cirugia ci
              left join ci.quirofano q
              join UnidadMedica u on u.id in (cl.unidad.id, i.unidad.id, q.unidad.id)
             where f.fechaEmision >= :desde and f.fechaEmision < :hasta
             group by u.hospital.id
            """, Object[].class)
            .setParameter("desde", inicioDelDia(desde))
            .setParameter("hasta", inicioDelDia(hasta.plusDays(1)))
            .getResultList()) {
        facturado.put((Integer) f[0], f);
    }
    Map<Integer, BigDecimal> cobrado = new HashMap<>();
    for (Object[] f : em.createQuery("""
            select u.hospital.id, sum(p.monto)
              from Pago p
              join p.factura f
              left join f.cita c
              left join c.clinica cl
              left join f.ingreso i
              left join f.cirugia ci
              left join ci.quirofano q
              join UnidadMedica u on u.id in (cl.unidad.id, i.unidad.id, q.unidad.id)
             where p.fechaPago between :desde and :hasta
             group by u.hospital.id
            """, Object[].class)
            .setParameter("desde", desde)
            .setParameter("hasta", hasta)
            .getResultList()) {
        cobrado.put((Integer) f[0], decimal(f[1]));
    }
    List<IngresosHospital> resultado = new ArrayList<>();
    for (Object[] h : em.createQuery("select h.id, h.nombre from Hospital h", Object[].class).getResultList()) {
        Object[] f = facturado.get((Integer) h[0]);
        long facturas = f == null ? 0 : ((Number) f[1]).longValue();
        BigDecimal total = f == null ? BigDecimal.ZERO : decimal(f[2]);
        BigDecimal pagado = cobrado.getOrDefault((Integer) h[0], BigDecimal.ZERO);
        resultado.add(new IngresosHospital((String) h[1], facturas, total, pagado, total.subtract(pagado)));
    }
    resultado.sort(Comparator.comparing(IngresosHospital::totalFacturado).reversed());
    return resultado;
}

/**
 * Objetivo 18: division relacional. Pacientes con consulta externa en el lapso y para los
 * que no existe una unidad con ficha de ingreso (emergencias, cirugia, hospitalizacion)
 * en la que no hayan sido atendidos.
 */
public List<PacienteFiel> objetivo18(LocalDate desde, LocalDate hasta) {
    OffsetDateTime inicio = inicioDelDia(desde);
    OffsetDateTime fin = inicioDelDia(hasta.plusDays(1));
    List<Object[]> filas = em.createQuery("""
            select p.id, p.numeroExpediente, p.nombres, p.apellidos, p.sexo
```

```
                 from Paciente p
                where exists (select co.id from Consulta co
                              where co.cita.paciente = p
                                and co.cita.fechaHora >= :inicio and co.cita.fechaHora < :fin)
                  and not exists (select u.id from UnidadMedica u
                                  where u.tipoUnidad <> :consultaExterna
                                    and not exists (select i.id from Ingreso i
                                                    where i.paciente = p
                                                      and i.unidad.tipoUnidad = u.tipoUnidad
                                                      and i.fechaHoraIngreso >= :inicio
                                                      and i.fechaHoraIngreso < :fin))
                order by p.id
               """, Object[].class)
               .setParameter("inicio", inicio)
               .setParameter("fin", fin)
               .setParameter("consultaExterna", "Consulta Externa")
               .getResultList();
        List<PacienteFiel> resultado = new ArrayList<>();
        for (Object[] f : filas) {
            resultado.add(new PacienteFiel((Integer) f[0], (String) f[1], (String) f[2], (String) f[3], (String) f[4]));
        }
        return resultado;
    }

    private static OffsetDateTime inicioDelDia(LocalDate fecha) {
        return fecha.atStartOfDay(GUATEMALA).toOffsetDateTime();
    }

    private static BigDecimal decimal(Object valor) {
        if (valor == null) {
            return BigDecimal.ZERO;
        }
        return valor instanceof BigDecimal b ? b : new BigDecimal(valor.toString());
    }
}
```

Sentencia preparada enviada por Hibernate (objetivo 8)

Extracto de la salida con hibernate.show_sql: los valores aparecen como «?».

```
Hibernate:
    select
        h1_0.nombre,
        um1_0.tipo_unidad,
        sum(case
            when mp1_0.nombre=?
                then p1_0.monto
            else 0
    end),
    sum(case
        when mp1_0.nombre=?
            then p1_0.monto
        else 0
end),
sum(case
    when mp1_0.nombre=?
        then p1_0.monto
    else 0
end)
from
    pago p1_0
join
    metodo_pago mp1_0
        on mp1_0.id_metodo_pago=p1_0.id_metodo_pago
join
    factura f1_0
        on f1_0.id_factura=p1_0.id_factura
left join
    cita c1_0
        on c1_0.id_cita=f1_0.id_cita
left join
    clinica c2_0
        on c2_0.id_clinica=c1_0.id_clinica
left join
    ingreso i1_0
        on i1_0.id_ingreso=f1_0.id_ingreso
left join
    cirugia c3_0
        on c3_0.id_cirugia=f1_0.id_cirugia
left join
    quirofano q1_0
        on q1_0.id_quirofano=c3_0.id_quirofano
join
    unidad_medica um1_0
```

```
        on um1_0.id_unidad in (c2_0.id_unidad, i1_0.id_unidad, q1_0.id_unidad)
join
    hospital h1_0
        on h1_0.id_hospital=um1_0.id_hospital
where
    p1_0.fecha_pago between ? and ?
group by
    h1_0.id_hospital,
    h1_0.nombre,
    um1_0.tipo_unidad
order by
    h1_0.id_hospital,
    um1_0.tipo_unidad
Hospital General de Occidente        Cirugía              16931.80         37625.24        14796.35
Hospital General de Occidente        Consulta Externa      2940.75          3511.25         2710.75
Hospital General de Occidente        Emergencias           9633.33         13433.33         9633.34
Hospital General de Occidente        Hospitalización      13238.13         11242.98        13429.67
Hospital Regional de San Marcos      Cirugía              10512.24          8744.64        23555.32
Hospital Regional de San Marcos      Consulta Externa      2440.00          1702.50         1981.25
Hospital Regional de San Marcos      Emergencias           7154.17          7593.33         5610.01
Hospital Regional de San Marcos      Hospitalización       6092.32          6909.17        12958.62
Hospital Infantil de Retalhuleu      Cirugía              16563.42         15600.64        15573.93
Hospital Infantil de Retalhuleu      Consulta Externa      1197.50          1563.75         1074.25
Hospital Infantil de Retalhuleu      Emergencias           6487.50          8557.50         7432.50
Hospital Infantil de Retalhuleu      Hospitalización       6516.35          7136.37         7756.63
Hospital de Occidente Huehuetenango  Cirugía              9148.91         11772.30         9224.79
Hospital de Occidente Huehuetenango  Consulta Externa       883.75          1091.50         1261.25
Hospital de Occidente Huehuetenango  Emergencias           5966.67          6686.67         6226.66
Hospital de Occidente Huehuetenango  Hospitalización       8021.85          9982.35        10436.12
Hospital de Occidente Sololá         Cirugía              3892.38          5543.74         3892.38
Hospital de Occidente Sololá         Consulta Externa      1191.75           891.00         1120.00
Hospital de Occidente Sololá         Emergencias           5412.50          6962.50         5287.50
Hospital de Occidente Sololá         Hospitalización      10005.63          7307.14         9449.32

Objetivo 11 - ingresos por hospital, del 2026-01-01 al 2026-10-31
Hospital                              Facturas        Facturado        Cobrado      Pendiente
```

Anexo A. Diccionario de datos completo

Todas las tablas del esquema final (Proyecto 1 extendido por el Proyecto 2), en orden alfabético.

Tabla agendamiento_quirurgico

Solicitud de cirugía que el cirujano envía al comité médico.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_agendamiento	integer	PK	No				
id_paciente	integer	FK	No	Referencia a paciente.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el paciente dejaría la solicitud sin referencia, por eso se impide.
id_medico_cirujano	integer	FK	No	Referencia a medico.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el cirujano dejaría la solicitud sin referencia, por eso se impide.
id_tipo_cirugia	integer	FK	No	Referencia a tipo_cirugia.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.
id_tipo_anestesia	integer	FK	No	Referencia a tipo_anestesia.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.
caracter	character varying(10)	—	No	urgente o programado.			
resumen_clinico	text	—	No	Resumen de la historia clínica que justifica la cirugía.			
tiempo_estimado_min	smallint	—	No	Tiempo estimado de la intervención en minutos.			
fecha_hora_solicitud	timestamp with time zone	—	No	Fecha y hora en que se envió la solicitud.			
estado_solicitud	character varying(9)	—	No	Decisión del comité: pendiente, aprobada o rechazada.			
motivo_rechazo	text	—	Sí	Razones del rechazo (base de cirugías rechazadas).			
id_quirofano_solicitado	integer	FK	Sí	Referencia a quirofano.	RESTRICT	CASCADE	RESTRICT: no se borra un quirófano con solicitudes; se marca en mantenimiento.
fecha_hora_agendada	timestamp with time zone	—	Sí	Fecha y hora asignadas al aprobarse.			

Tabla aspecto_quirurgico

Aspectos que el enfermero verifica en cada etapa de la cirugía.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_aspecto	integer	PK	No				
etapa	character varying(15)	—	No	preoperatorio, intraoperatorio o postoperatorio.			
seccion	character varying(40)	—	No	Sección de la etapa (Planificación, Entrada, Pausa quirúrgica...).			
nombre	character varying(150)	—	No	Aspecto que se verifica.			
tipo_evaluacion	character varying(13)	—	No	exito_falla o aceptabilidad.			

Tabla auditoria_personal

Bitácora automática de los cambios de salario, nivel de acceso y especialidad del personal.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_auditoria	integer	PK	No	Identificador del registro de la bitácora (lo genera la base).			
fecha_hora	timestamp with time zone	—	No	Fecha y hora exacta del cambio (now()).			
usuario	character varying(63)	—	No	Usuario de la base de datos que hizo el cambio (current_user).			
tabla	character varying(30)	—	No	Tabla donde ocurrió el cambio: personal o medico.			
id_registro	integer	—	No	Identificador de la persona cuyo dato cambió (id_personal = id_medico).			
campo	character varying(30)	—	No	Campo que cambió: salario, nivel_acceso o id_especialidad.			
valor_anterior	text	—	Sí	Valor antes del cambio.			
valor_nuevo	text	—	Sí	Valor después del cambio.			

Tabla calificacion

Calificación de la calidad del servicio de un hospital, médico, enfermero o encargado.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_calificacion	integer	PK	No				
id_paciente	integer	FK	No	Referencia a paciente.	RESTRICT	CASCADE	RESTRICT: se conserva quién emitió cada calificación.
fecha	date	—	No	Fecha de la calificación.			
puntuacion	smallint	—	No	Puntuación de 1 (peor) a 5 (mejor).			
comentario	text	—	Sí	Comentario opcional.			
id_hospital	integer	FK	Sí	Referencia a hospital.	RESTRICT	CASCADE	RESTRICT: la calificación forma parte del historial de calidad del servicio.
id_medico	integer	FK	Sí	Referencia a medico.	RESTRICT	CASCADE	RESTRICT: la calificación forma parte del historial de calidad del servicio.
id_enfermero	integer	FK	Sí	Referencia a enfermero.	RESTRICT	CASCADE	RESTRICT: la calificación forma parte del historial de calidad del servicio.
id_encargado	integer	FK	Sí	Referencia a encargado.	RESTRICT	CASCADE	RESTRICT: la calificación forma parte del historial de calidad del servicio.
id_consulta	integer	FK	Sí	Referencia a consulta.	RESTRICT	CASCADE	RESTRICT: la calificación forma parte del historial de calidad del servicio.

Tabla camilla

Camillas de las unidades de Emergencias y Hospitalización.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_camilla	integer	PK	No				
id_unidad	integer	FK	No	Referencia a unidad_medica.	CASCADE	CASCADE	CASCADE: la camilla es un detalle que no existe sin su unidad; al borrar el unidad se borran sus detalles.
numero	smallint	—	No	Número de la camilla dentro de su unidad.			
disponible	boolean	—	No	Indica si la camilla está libre.			

Tabla chequeo_preanestesico

Evaluación del riesgo anestésico (ASA) y plan de anestesia.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_chequeo	integer	PK	No				
id_agendamiento	integer	FK/UK	No	Referencia a agendamiento_quirurgico.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el agendamiento dejaría el chequeo preanestésico sin referencia, por eso se impide.
clasificacion_asa	character varying(3)	FK	No	Clasificación ASA del riesgo anestésico (I a VI).	RESTRICT	CASCADE	DELETE RESTRICT: la clase ASA es parte de la historia clínica del paciente. UPDATE CASCADE: si cambia el código, los chequeos lo siguen.
id_medico_clasifica	integer	FK	No	Referencia a medico.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el médico dejaría el chequeo sin referencia, por eso se impide.
plan_anestesia	text	—	No	Plan de anestesia.			
id_medico_anestesista	integer	FK	No	Referencia a medico.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el anestesista dejaría el chequeo sin referencia, por eso se impide.

Tabla cirugia

Cirugía realizada a partir de un agendamiento aprobado.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_cirugia	integer	PK	No				
id_agendamiento	integer	FK/UK	No	Referencia a agendamiento_quirurgico.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el agendamiento dejaría la cirugía sin referencia, por eso se impide.
id_quirofano	integer	FK	No	Referencia a quirofano.	RESTRICT	CASCADE	RESTRICT: no se borra un quirófano donde se operó; se marca en mantenimiento.
id_ingreso	integer	FK	Sí	Referencia a ingreso.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el ingreso dejaría la cirugía sin referencia, por eso se impide.
fecha_hora_inicio	timestamp with time zone	—	No	Inicio de la cirugía.			
fecha_hora_fin	timestamp with time zone	—	Sí	Fin de la cirugía.			
estado	character varying(15)	—	No	preoperatorio, intraoperatorio, postoperatorio o finalizada.			
condicion_evolucion	character varying(21)	—	Sí	sin_complicaciones, complicaciones o complicaciones_graves.			

Tabla cirugia_insumo

Insumos usados en la cirugía.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_cirugia	integer	PK/FK	No	Referencia a cirugia.	CASCADE	CASCADE	CASCADE: el consumo de insumos es un detalle que no existe sin su cirugía; al borrar el cirugía se borran sus detalles.
id_insumo	integer	PK/FK	No	Referencia a insumo.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.
cantidad	smallint	—	No	Cantidad usada.			
costo_unitario	numeric(10,2)	—	No	Costo por unidad al momento de usarlo.			

Tabla cirugia_personal

Médicos y enfermeros que participan en la cirugía.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_cirugia_personal	integer	PK	No				
id_cirugia	integer	FK	No	Referencia a cirugia.	CASCADE	CASCADE	CASCADE: la participación del personal es un detalle que no existe sin su cirugía; al borrar el cirugía se borran sus detalles.
id_medico	integer	FK	Sí	Referencia a medico.	RESTRICT	CASCADE	RESTRICT: no se borra a un médico que participó en cirugías.
id_enfermero	integer	FK	Sí	Referencia a enfermero.	RESTRICT	CASCADE	RESTRICT: no se borra a un enfermero que participó en cirugías.
id_rol	integer	FK	No	Referencia a rol_quirurgico.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.

Tabla cita

Citas programadas de consulta externa.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_cita	integer	PK	No				
id_paciente	integer	FK	No	Referencia a paciente.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el paciente dejaría la cita sin referencia, por eso se impide.
id_medico	integer	FK	No	Referencia a medico.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el médico dejaría la cita sin referencia, por eso se impide.
fecha_hora	timestamp with time zone	—	No	Fecha y hora de la cita.			
numero_clinica	character varying(10)	—	No	Número de clínica escrito en la cita (se conserva por compatibilidad; usar id_clinica).			
medio_asignacion	character varying(20)	FK	No	Medio por el que se asignó: teléfono, correo, recepción general o del médico.	RESTRICT	CASCADE	DELETE RESTRICT: no se borra un medio con el que se asignaron citas. UPDATE CASCADE: si cambia el código, las citas lo siguen.
estado	character varying(12)	—	No	programada, realizada, reprogramada o cancelada.			
es_reconsulta	boolean	—	No	Indica si es reconsulta (cuesta 75% de la primera consulta).			
es_referido	boolean	—	No	Indica si el paciente fue referido por una institución (cuesta 80%).			
cancelada_sin_aviso	boolean	—	No	Indica si se canceló sin avisar (sobrecargo de 10% en la siguiente cita).			
costo_base	numeric(10,2)	—	No	Costo de primera consulta del médico al momento de agendar.			
costo_final	numeric(10,2)	—	No	Costo cobrado después de aplicar las reglas de descuento o sobrecargo.			
id_clinica	integer	FK	No	Clínica donde se atiende la cita.	RESTRICT	CASCADE	DELETE RESTRICT: no se borra una clínica donde ya hubo citas, es parte de la historia. UPDATE CASCADE: la cita sigue a su clínica.

Tabla clasificacion_asa

Catálogo de la clasificación ASA del riesgo anestésico.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
codigo	character varying(3)	PK	No	Clase ASA (I a VI).			
descripcion	character varying(100)	—	No	Significado de la clase.			

Tabla clinica

Clínicas (consultorios) de la unidad de Consulta Externa de cada hospital.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_clinica	integer	PK	No	Identificador de la clínica.			
id_unidad	integer	FK	No	Referencia a unidad_medica.	RESTRICT	CASCADE	DELETE RESTRICT: no se borra una unidad que tiene clínicas con citas. UPDATE CASCADE: la clínica sigue a su unidad.
numero	character varying(10)	—	No	Número o código de la clínica dentro de su unidad.			

Tabla consentimiento_informado

Consentimiento informado firmado para la cirugía.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_consentimiento	integer	PK	No				
id_agendamiento	integer	FK/UK	No	Referencia a agendamiento_quirurgico.	RESTRICT	CASCADE	RESTRICT: el consentimiento firmado es un documento legal que se debe conservar.
id_medico	integer	FK	No	Referencia a medico.	RESTRICT	CASCADE	RESTRICT: el consentimiento firmado es un documento legal; no se borra al médico que lo firmó.
procedimiento_nombre	character varying(150)	—	No	Nombre específico del procedimiento.			
procedimiento_objetivo	text	—	No	Objetivo del procedimiento.			
procedimiento_caracteristicas	text	—	No	Características del procedimiento.			
procedimiento_riesgos	text	—	No	Posibles riesgos del procedimiento.			
medico_firmo	boolean	—	No	Indica si el médico firmó.			
firmante_nombre	character varying(150)	—	No	Nombre de quien firma por el paciente.			
firmante_relacion	character varying(20)	—	No	paciente, familiar, tutor o representante_legal.			
firmante_firmo	boolean	—	No	Indica si el paciente o su representante firmó.			
fecha_obtencion	date	—	No	Fecha en que se obtuvo el consentimiento.			

Tabla consulta

Consulta realizada en una cita.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_consulta	integer	PK	No				
id_cita	integer	FK/UK	No	Referencia a cita.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el cita dejaría la consulta sin referencia, por eso se impide.
id_diagnostico	integer	FK	No	Referencia a diagnostico.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.
otros_datos	text	—	Sí	Otros datos de interés clínico.			
notas_observaciones	text	—	Sí	Notas u observaciones del médico.			

Tabla departamento

Departamentos de Guatemala.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_departamento	integer	PK	No				
nombre	character varying(80)	UK	No	Nombre del departamento.			

Tabla diagnostico

Diagnósticos de la Clasificación Internacional de Enfermedades (CIE-10).

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_diagnostico	integer	PK	No				
codigo_cie10	character varying(7)	UK	No	Código CIE-10 del diagnóstico.			
descripcion	character varying(200)	—	No	Descripción oficial del diagnóstico.			

Tabla documentacion_etapa

Documentación de cada etapa de la cirugía enviada a secretaría.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_cirugia	integer	PK/FK	No	Referencia a cirugia.	CASCADE	CASCADE	CASCADE: la documentación de la etapa es un detalle que no existe sin su cirugía; al borrar el cirugía se borran sus detalles.
etapa	character varying(15)	PK	No	Etapa documentada.			
id_enfermero	integer	FK	No	Referencia a enfermero.	RESTRICT	CASCADE	RESTRICT: se debe conservar qué enfermero envió la documentación.
fecha_envio	date	—	No	Fecha de envío a secretaría.			

Tabla egreso

Ficha de egreso que cierra un ingreso.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_egreso	integer	PK	No				
id_ingreso	integer	FK/UK	No	Referencia a ingreso.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el ingreso dejaría el egreso sin referencia, por eso se impide.
id_medico	integer	FK	No	Referencia a medico.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el médico dejaría el egreso sin referencia, por eso se impide.
id_diagnostico_principal	integer	FK	No	Referencia a diagnostico.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.
fecha_hora_egreso	timestamp with time zone	—	No	Fecha y hora del egreso.			
motivo_egreso	text	—	No	Motivo del egreso.			
codigo_egreso	character varying(8)	—	No	Condición de salida: vivo, muerto, embarazo o parto.			
sin_consentimiento	boolean	—	No	Indica si el paciente salió sin consentimiento médico.			
motivo_sin_consentimiento	text	—	Sí	Motivo del egreso sin consentimiento.			
codigo_traslado	character varying(20)	—	Sí	Código de traslado, si el egreso fue por traslado.			
referido_otro_hospital	boolean	—	No	Indica si fue referido a otro hospital o establecimiento.			

Tabla egreso_diagnostico

Diagnósticos secundarios de cada egreso.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_egreso	integer	PK/FK	No	Referencia a egreso.	CASCADE	CASCADE	CASCADE: el diagnóstico secundario es un detalle que no existe sin su egreso; al borrar el egreso se borran sus detalles.
id_diagnostico	integer	PK/FK	No	Referencia a diagnostico.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.

Tabla encargado

Personas responsables de un paciente.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_encargado	integer	PK	No				
nombres	character varying(100)	—	No	Nombres del encargado.			
apellidos	character varying(100)	—	No	Apellidos del encargado.			
dpi	character(13)	UK	No	CUI de 13 dígitos.			
telefono	character varying(9)	—	Sí	Teléfono de contacto.			
id_municipio	integer	FK	No	Referencia a municipio.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.
area	character varying(6)	—	No	Área de residencia: Urbana o Rural.			

Tabla enfermero

Subtipo de personal: enfermeros registrados y practicantes de enfermería.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_enfermero	integer	PK/FK	No	Identificador del enfermero; es el mismo id_personal de su supertipo.	CASCADE	CASCADE	FK compuesta (id_enfermero, tipo_personal) hacia personal. DELETE CASCADE: el subtipo no existe sin su supertipo. UPDATE CASCADE: el subtipo comparte la llave con el supertipo y debe seguirla.
nombres	character varying(100)	—	No	Copia de personal.nombres (se guarda igual en las dos tablas).			
apellidos	character varying(100)	—	No	Copia de personal.apellidos.			
dpi	character(13)	UK	No	Copia de personal.dpi.			
telefono	character varying(9)	—	Sí	Copia de personal.telefono.			
id_hospital	integer	FK	No	Copia de personal.id_hospital.	RESTRICT	CASCADE	RESTRICT: un hospital con enfermeros asignados no se elimina; primero se reasigna su personal.
tipo	character varying(12)	—	No	registrado o practicante.			
tipo_personal	character varying(9)	FK	No	Discriminador fijo en enfermero; junto con id_enfermero forma la FK hacia personal.	CASCADE	CASCADE	Parte de la FK compuesta hacia personal (ver id_enfermero): obliga a que apunte a personal de tipo enfermero.

Tabla equipo

Equipos médicos y quirúrgicos.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_equipo	integer	PK	No				
nombre	character varying(120)	UK	No	Nombre del equipo.			
descripcion	text	—	Sí	Descripción del equipo.			
tipo	character varying(10)	—	No	medico, quirurgico u otro.			
funcion	character varying(14)	FK	No	exploracion, diagnostico, tratamiento, rehabilitacion u otro.	RESTRICT	CASCADE	DELETE RESTRICT: no se borra una función que tiene equipos. UPDATE CASCADE: si cambia el código, los equipos lo siguen.
estado	character varying(16)	—	No	disponible, en_uso o en_mantenimiento.			

Tabla especialidad

Especialidades médicas clínicas y quirúrgicas.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_especialidad	integer	PK	No				
nombre	character varying(80)	UK	No	Nombre de la especialidad.			
tipo	character varying(10)	—	No	clinica (consulta) o quirurgica (cirugía).			

Tabla estado_civil

Catálogo de estados civiles.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_estado_civil	integer	PK	No				
nombre	character varying(20)	UK	No	Estado civil.			

Tabla examen_laboratorio

Catálogo de exámenes de laboratorio que se pueden ordenar.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_examen	integer	PK	No	Identificador del examen.			
nombre	character varying(100)	UK	No	Nombre del examen.			
descripcion	character varying(200)	—	Sí	Qué mide el examen.			
costo	numeric(10,2)	—	No	Costo del examen en quetzales.			

Tabla factura

Factura emitida al terminar una consulta, emergencia, hospitalización o cirugía.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_factura	integer	PK	No				
id_cita	integer	FK/UK	Sí	Referencia a cita.	RESTRICT	CASCADE	RESTRICT: es información financiera que se debe conservar para auditoría.
id_ingreso	integer	FK/UK	Sí	Referencia a ingreso.	RESTRICT	CASCADE	RESTRICT: es información financiera que se debe conservar para auditoría.
id_cirugia	integer	FK/UK	Sí	Referencia a cirugia.	RESTRICT	CASCADE	RESTRICT: es información financiera que se debe conservar para auditoría.
fecha_emision	timestamp with time zone	—	No	Fecha y hora de emisión.			
descripcion	text	—	No	Descripción breve de los servicios facturados.			
total	numeric(10,2)	—	No	Monto total en quetzales.			
numero_cuotas	smallint	—	No	Número de cuotas mensuales del plan de pago (1 a 12).			

Tabla forma_farmaceutica

Catálogo de formas en que se entrega un medicamento.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
codigo	character varying(12)	PK	No	Código de la forma farmacéutica.			
descripcion	character varying(100)	—	No	Descripción de la forma farmacéutica.			

Tabla funcion_equipo

Catálogo de funciones de un equipo médico.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
codigo	character varying(14)	PK	No	Código de la función.			
descripcion	character varying(100)	—	No	Descripción de la función.			

Tabla funcion_instrumento

Catálogo de funciones de un instrumento quirúrgico.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
codigo	character varying(12)	PK	No	Código de la función.			
descripcion	character varying(100)	—	No	Descripción de la función.			

Tabla historia_clinica

Interrogatorio y exploración física del paciente antes de la cirugía.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_historia_clinica	integer	PK	No				

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_agendamiento	integer	FK/UK	No	Referencia a agendamiento_quirurgico.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el agendamiento dejaría la historia clínica sin referencia, por eso se impide.
id_medico	integer	FK	No	Referencia a medico.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el médico dejaría la historia clínica sin referencia, por eso se impide.
fecha	date	—	No	Fecha de elaboración.			
fuente_interrogatorio	character varying(9)	—	No	directo (del paciente) o indirecto (de un familiar o amigo).			
religion	character varying(40)	—	Sí	Religión del paciente.			
ocupacion	character varying(60)	—	Sí	Ocupación del paciente.			
lugar_nacimiento	character varying(80)	—	Sí	Lugar de nacimiento.			
lugar_residencia	character varying(80)	—	Sí	Lugar de residencia.			
antecedentes_heredofamiliares	text	—	No	Antecedentes heredofamiliares.			
antecedentes_personales_no_patologicos	text	—	No	Antecedentes personales no patológicos.			
antecedentes_patologicos	text	—	No	Antecedentes patológicos.			
padecimiento_actual	text	—	No	Padecimiento actual.			
interrogatorio_aparatos_sistemas	text	—	Sí	Interrogatorio por aparatos y sistemas.			
sintomas_generales_terapeutica	text	—	Sí	Síntomas generales y terapéutica empleada.			
estudios_previos	text	—	Sí	Estudios previos.			
signos_vitales	text	—	Sí	Signos vitales.			
exploracion_general	text	—	Sí	Exploración general.			
exploracion_sistematizada	text	—	Sí	Exploración de cabeza, cuello, tórax, abdomen, extremidades, columna y cavidades.			

Tabla hospital

Hospitales de la cadena Hospitales de Occidente.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_hospital	integer	PK	No				
nombre	character varying(120)	UK	No	Nombre del hospital.			
informacion_especifica	text	—	Sí	Información específica del hospital (nivel, capacidad, especialización).			
direccion	character varying(250)	—	No	Dirección del hospital.			
id_municipio	integer	FK	No	Referencia a municipio.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.

Tabla hospitalizacion_insumo

Insumos usados durante un internamiento.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_ingreso	integer	PK/FK	No	Referencia a ingreso.	CASCADE	CASCADE	CASCADE: el consumo de insumos es un detalle que no existe sin su ingreso; al borrar el ingreso se borran sus detalles.
id_insumo	integer	PK/FK	No	Referencia a insumo.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.
cantidad	smallint	—	No	Cantidad usada durante el internamiento.			
costo_unitario	numeric(10,2)	—	No	Costo por unidad al momento de usarlo.			

Tabla ingreso

Ficha de ingreso a Emergencias, Cirugía u Hospitalización.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_ingreso	integer	PK	No				
id_paciente	integer	FK	No	Referencia a paciente.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el paciente dejaría el ingreso sin referencia, por eso se impide.
id_unidad	integer	FK	No	Referencia a unidad_medica.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el unidad dejaría el ingreso sin referencia, por eso se impide.
id_servicio	integer	FK	Sí	Referencia a servicio_atencion.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.
id_medico_encargado	integer	FK	No	Referencia a medico.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el médico encargado dejaría el ingreso sin referencia, por eso se impide.
id_camilla	integer	FK	Sí	Referencia a camilla.	RESTRICT	CASCADE	RESTRICT: no se borra una camilla que aparece en ingresos; se marca como no disponible.
id_quirofano	integer	FK	Sí	Referencia a quirofano.	RESTRICT	CASCADE	RESTRICT: no se borra un quirófano que aparece en ingresos; se marca en mantenimiento.
id_turno	integer	FK	Sí	Referencia a turno_emergencia.	RESTRICT	CASCADE	RESTRICT: el turno es parte del registro de cómo se atendió el ingreso.
fecha_hora_ingreso	timestamp with time zone	—	No	Fecha y hora del ingreso.			
motivo_ingreso	text	—	No	Motivo por el que ingresa el paciente.			
diagnostico_presuntivo	text	—	Sí	Diagnóstico presuntivo de ingreso.			
prioridad_triage	smallint	—	Sí	Prioridad de atención en emergencias: 1 la más urgente, 5 la menos.			
turnos_espera	smallint	—	Sí	Turnos que esperó por falta de camilla libre (solo emergencias).			

Tabla institucion_externa

Hospitales y establecimientos fuera de la cadena que reciben traslados.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_institucion	integer	PK	No				
nombre	character varying(120)	UK	No	Nombre de la institución.			
direccion	character varying(250)	—	No	Dirección de la institución.			
id_municipio	integer	FK	No	Referencia a municipio.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.

Tabla instrumento

Instrumental médico y quirúrgico.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_instrumento	integer	PK	No				
nombre	character varying(120)	UK	No	Nombre del instrumento.			
descripcion	text	—	Sí	Descripción del instrumento.			
tipo	character varying(10)	—	No	medico, quirurgico u otro.			
funcion	character varying(12)	FK	No	corte, contencion, hemostatica, retractor, accesorio, implante u otro.	RESTRICT	CASCADE	DELETE RESTRICT: no se borra una función que tiene instrumentos. UPDATE CASCADE: si cambia el código, los instrumentos lo siguen.
estado	character varying(16)	—	No	disponible, en_uso o en_mantenimiento.			

Tabla insumo

Insumos médicos y quirúrgicos con su inventario.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_insumo	integer	PK	No				
nombre	character varying(120)	UK	No	Nombre del insumo.			
descripcion	text	—	Sí	Descripción del insumo.			
material	character varying(80)	—	Sí	Material del que está hecho.			
tipo	character varying(10)	—	No	medico, quirurgico u otro.			
unidad_medida	character varying(20)	—	No	Unidad en que se despacha (caja, unidad, bolsa...).			
costo_unitario	numeric(10,2)	—	No	Costo actual por unidad.			
stock_minimo	integer	—	No	Existencia mínima antes de reabastecer.			
stock_actual	integer	—	No	Existencia actual en inventario.			

Tabla medicamento

Medicamentos que se pueden recetar.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_medicamento	integer	PK	No				
nombre_generico	character varying(80)	—	No	Nombre genérico (principio activo).			
concentracion	character varying(30)	—	No	Concentración por unidad (ej. 50 mg).			
forma_farmaceutica	character varying(12)	FK	No	Presentación: tableta, cápsula, jarabe, inyectable...	RESTRICT	CASCADE	DELETE RESTRICT: no se borra una forma farmacéutica que usa algún medicamento. UPDATE CASCADE: si cambia el código, los medicamentos lo siguen.

Tabla medico

Subtipo de personal: médicos de la cadena (los externos no están contratados, solo operan).

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_medico	integer	PK/FK	No	Identificador del médico; es el mismo id_personal de su supertipo.	CASCADE	CASCADE	FK compuesta (id_medico, tipo_personal) hacia personal. DELETE CASCADE: el subtipo no existe sin su supertipo, al borrar la persona se borra su fila de médico (si tiene historia clínica, esas FK lo impiden). UPDATE CASCADE: el subtipo comparte la llave con el supertipo y debe seguirla; un cambio de tipo_personal en personal choca con el CHECK del subtipo, por lo que nadie puede cambiar de médico a enfermero.
nombres	character varying(100)	—	No	Copia de personal.nombres (se guarda igual en las dos tablas).			
apellidos	character varying(100)	—	No	Copia de personal.apellidos.			
dpi	character(13)	UK	No	Copia de personal.dpi.			
telefono	character varying(9)	—	Sí	Copia de personal.telefono.			
id_hospital	integer	FK	No	Copia de personal.id_hospital.	RESTRICT	CASCADE	RESTRICT: un hospital con médicos asignados no se elimina; primero se reasigna su personal.
id_municipio	integer	FK	No	Copia de personal.id_municipio.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.
area	character varying(6)	—	No	Copia de personal.area.			
id_especialidad	integer	FK	No	Especialidad del médico. Se audita cada cambio.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.
tipo_medico	character varying(12)	FK	No	Modalidad: general, especialista, residente, interno o externo.	RESTRICT	CASCADE	DELETE RESTRICT: no se borra una modalidad que tienen médicos. UPDATE CASCADE: si cambia el código, los médicos lo siguen.
costo_primera_consulta	numeric(10,2)	—	Sí	Costo de la primera consulta; base para reconsultas y referidos.			
horario_inicio	time without time zone	—	Sí	Inicio del horario de consulta externa (turno matutino).			
horario_fin	time without time zone	—	Sí	Fin del horario de consulta externa.			
tipo_personal	character varying(9)	FK	No	Discriminador fijo en medico; junto con id_medico forma la FK hacia personal.	CASCADE	CASCADE	Parte de la FK compuesta hacia personal (ver id_medico): obliga a que el médico apunte a personal de tipo médico.
cargo	character varying(15)	—	No	Cargo en la cadena de mando: director_medico, jefe_servicio o medico.			
id_medico_superior	integer	FK	Sí	Superior directo dentro del mismo hospital (relación recursiva); nulo para el director.	SET NULL	CASCADE	Relación recursiva. DELETE SET NULL: si el jefe deja el hospital, sus subordinados quedan sin superior hasta que se les asigne otro, sin perder su historia ni bloquear la baja. UPDATE CASCADE: si cambiara la llave del jefe, sus subordinados lo siguen.

Tabla medio_asignacion

Catálogo de medios por los que se asigna una cita.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
codigo	character varying(20)	PK	No	Código del medio.			
descripcion	character varying(100)	—	No	Descripción del medio.			

Tabla metodo_pago

Métodos de pago aceptados.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_metodo_pago	integer	PK	No				
nombre	character varying(30)	UK	No	Método de pago.			

Tabla municipio

Municipios de cada departamento.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_municipio	integer	PK	No				
id_departamento	integer	FK	No	Referencia a departamento.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.
nombre	character varying(80)	—	No	Nombre del municipio.			

Tabla orden_examen

Exámenes que se piden en cada orden de laboratorio.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_orden	integer	PK/FK	No	Referencia a orden_laboratorio.	CASCADE	CASCADE	DELETE CASCADE: los exámenes pedidos no existen sin su orden. UPDATE CASCADE: siguen a su orden.
id_examen	integer	PK/FK	No	Referencia a examen_laboratorio.	RESTRICT	CASCADE	DELETE RESTRICT: no se borra un examen del catálogo que ya fue ordenado. UPDATE CASCADE: si cambia el id, las órdenes lo siguen.

Tabla orden_laboratorio

Orden de laboratorio que el médico entrega en una consulta.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_orden	integer	PK	No	Identificador de la orden.			
id_consulta	integer	FK	No	Referencia a consulta.	RESTRICT	CASCADE	DELETE RESTRICT: la orden es parte de la historia clínica de la consulta. UPDATE CASCADE: la orden sigue a su consulta.
fecha	date	—	No	Fecha de la orden.			
indicaciones	text	—	Sí	Indicaciones para el paciente.			

Tabla paciente

Pacientes atendidos en la red.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_paciente	integer	PK	No				
numero_expediente	character varying(20)	UK	No	Número de expediente clínico del paciente.			
nombres	character varying(100)	—	No	Nombres del paciente.			
apellidos	character varying(100)	—	No	Apellidos del paciente.			
fecha_nacimiento	date	—	No	Fecha de nacimiento; de ella se calcula la edad.			
sexo	character varying(9)	—	No	Masculino o Femenino.			
dpi	character(13)	UK	Sí	CUI de 13 dígitos; opcional para menores de edad.			
telefono	character varying(9)	—	Sí	Teléfono de contacto.			
id_municipio	integer	FK	No	Referencia a municipio.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.
area	character varying(6)	—	No	Área de residencia: Urbana o Rural.			
id_estado_civil	integer	FK	Sí	Referencia a estado_civil.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.
numero_seguro_social	character varying(30)	—	Sí	Número de afiliación al seguro social, si tiene.			

Tabla paciente_encargado

Encargados de cada paciente y su parentesco.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_paciente	integer	PK/FK	No	Referencia a paciente.	CASCADE	CASCADE	CASCADE: la relación no tiene sentido sin el paciente; al borrarlo se quitan sus vínculos.
id_encargado	integer	PK/FK	No	Referencia a encargado.	CASCADE	CASCADE	CASCADE: la relación no tiene sentido sin el encargado; al borrarlo solo se quita el vínculo.
id_parentesco	integer	FK	No	Referencia a parentesco.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.

Tabla pago

Pago de una cuota de la factura.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_pago	integer	PK	No				
id_factura	integer	FK	No	Referencia a factura.	RESTRICT	CASCADE	RESTRICT: es información financiera que se debe conservar para auditoría. Una factura con pagos no se puede borrar.
numero_cuota	smallint	—	No	Número de cuota que se paga.			
monto	numeric(10,2)	—	No	Monto pagado.			
fecha_pago	date	—	No	Fecha del pago.			
id_metodo_pago	integer	FK	No	Referencia a metodo_pago.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.

Tabla parentesco

Catálogo de parentescos entre paciente y encargado.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_parentesco	integer	PK	No				
nombre	character varying(20)	UK	No	Parentesco o relación con el paciente.			

Tabla personal

Supertipo con los datos comunes de todo el personal del hospital; sus subtipos son medico y enfermero.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_personal	integer	PK	No	Identificador de la persona; es también la llave de su subtipo.			
tipo_personal	character varying(9)	—	No	Discriminador: indica si la persona es medico o enfermero.			
nombres	character varying(100)	—	No	Nombres de la persona.			
apellidos	character varying(100)	—	No	Apellidos de la persona.			
dpi	character(13)	UK	No	Documento Personal de Identificación (CUI de 13 dígitos).			
telefono	character varying(9)	—	Sí	Teléfono de contacto.			
direccion	character varying(250)	—	No	Dirección de residencia.			
id_municipio	integer	FK	No	Municipio de residencia.	RESTRICT	CASCADE	DELETE RESTRICT: un municipio de catálogo no se borra mientras viva personal en él. UPDATE CASCADE: la llave es identity; si cambiara, la persona sigue a su municipio.
area	character varying(6)	—	No	Área de residencia: Urbana o Rural.			
id_hospital	integer	FK	No	Hospital donde trabaja.	RESTRICT	CASCADE	DELETE RESTRICT: no se elimina un hospital que todavía tiene personal; primero se reasigna. UPDATE CASCADE: si cambiara la llave del hospital, su personal la sigue.
fecha_contratacion	date	—	Sí	Fecha de contratación; nula para los médicos externos, que no están contratados.			
salario	numeric(10,2)	—	Sí	Salario mensual en quetzales; nulo si no está contratado. Se audita cada cambio.			
nivel_acceso	character varying(13)	—	No	Nivel de acceso al sistema: consulta, operativo, supervisor o administrador. Se audita cada cambio.			

Tabla quirofano

Quirófanos de la unidad de Cirugía.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_quirofano	integer	PK	No				
id_unidad	integer	FK	No	Referencia a unidad_medica.	CASCADE	CASCADE	CASCADE: el quirófano es un detalle que no existe sin su unidad; al borrar el unidad se borran sus detalles.
numero	smallint	—	No	Número del quirófano dentro de la unidad de cirugía.			
estado	character varying(16)	—	No	Estado: disponible, ocupado o en_mantenimiento.			

Tabla receta

Receta entregada al final de la consulta.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_receta	integer	PK	No				
id_consulta	integer	FK/UK	No	Referencia a consulta.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el consulta dejaría la receta sin referencia, por eso se impide.
fecha_proxima_cita	date	—	Sí	Fecha sugerida para la próxima cita de seguimiento.			

Tabla receta_medicamento

Medicamentos prescritos en cada receta.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_receta	integer	PK/FK	No	Referencia a receta.	CASCADE	CASCADE	CASCADE: cada medicamento recetado es un detalle que no existe sin su receta; al borrar el receta se borran sus detalles.
id_medicamento	integer	PK/FK	No	Referencia a medicamento.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.
dosis	character varying(80)	—	No	Dosis indicada.			
duracion	character varying(40)	—	No	Duración del tratamiento.			

Tabla registro_procedimiento

Resultado de cada aspecto verificado durante la cirugía.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_registro	integer	PK	No				
id_cirugia	integer	FK	No	Referencia a cirugia.	CASCADE	CASCADE	CASCADE: el registro de cada aspecto es un detalle que no existe sin su cirugía; al borrar el cirugía se borran sus detalles.
id_aspecto	integer	FK	No	Referencia a aspecto_quirurgico.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.
resultado	character varying(22)	—	No	exito/falla, o aceptable/medianamente_aceptable/no_aceptable.			
descripcion	text	—	Sí	Descripción de lo observado.			
fecha_hora_registro	timestamp with time zone	—	No	Hora en que se registró.			
id_enfermero	integer	FK	No	Referencia a enfermero.	RESTRICT	CASCADE	RESTRICT: se debe conservar quién registró cada aspecto de la cirugía.

Tabla requerimiento_equipo

Equipos que solicita el cirujano en el agendamiento.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_agendamiento	integer	PK/FK	No	Referencia a agendamiento_quirurgico.	CASCADE	CASCADE	CASCADE: el requerimiento es un detalle que no existe sin su agendamiento; al borrar el agendamiento se borran sus detalles.
id_equipo	integer	PK/FK	No	Referencia a equipo.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.
cantidad	smallint	—	No	Cantidad solicitada.			

Tabla requerimiento_instrumento

Instrumental que solicita el cirujano en el agendamiento.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_agendamiento	integer	PK/FK	No	Referencia a agendamiento_quirurgico.	CASCADE	CASCADE	CASCADE: el requerimiento es un detalle que no existe sin su agendamiento; al borrar el agendamiento se borran sus detalles.
id_instrumento	integer	PK/FK	No	Referencia a instrumento.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.
cantidad	smallint	—	No	Cantidad solicitada.			

Tabla requerimiento_insumo

Insumos que solicita el cirujano en el agendamiento.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_agendamiento	integer	PK/FK	No	Referencia a agendamiento_quirurgico.	CASCADE	CASCADE	CASCADE: el requerimiento es un detalle que no existe sin su agendamiento; al borrar el agendamiento se borran sus detalles.
id_insumo	integer	PK/FK	No	Referencia a insumo.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.
cantidad	smallint	—	No	Cantidad solicitada.			

Tabla rol_quirurgico

Rol que cumple cada persona del equipo quirúrgico.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_rol	integer	PK	No				
nombre	character varying(40)	UK	No	Nombre del rol.			
tipo_personal	character varying(9)	—	No	Quién cumple el rol: medico o enfermero.			

Tabla servicio_atencion

Tipo de atención médica que ofrece cada unidad, con su costo.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_servicio	integer	PK	No				
id_unidad	integer	FK	No	Referencia a unidad_medica.	CASCADE	CASCADE	CASCADE: el servicio es un detalle que no existe sin su unidad; al borrar el unidad se borran sus detalles.
nombre	character varying(80)	—	No	Nombre del servicio de atención.			
costo	numeric(10,2)	—	No	Costo del servicio en quetzales.			

Tabla tipo_anestesia

Catálogo de tipos de anestesia.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_tipo_anestesia	integer	PK	No				
nombre	character varying(40)	UK	No	Tipo de anestesia.			

Tabla tipo_cirugia

Procedimientos quirúrgicos que ofrece la red.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_tipo_cirugia	integer	PK	No				
nombre	character varying(120)	UK	No	Nombre del procedimiento quirúrgico.			
id_especialidad	integer	FK	No	Referencia a especialidad.	RESTRICT	CASCADE	RESTRICT: no se borra un valor de catálogo que está en uso; primero se reasignan las filas que lo usan.

Tabla tipo_medico

Catálogo de modalidades del médico dentro del hospital.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
codigo	character varying(12)	PK	No	Código de la modalidad.			
descripcion	character varying(100)	—	No	Descripción de la modalidad.			

Tabla traslado

Ficha de traslado de un paciente entre unidades o hacia fuera de la red.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_traslado	integer	PK	No				
fecha_hora	timestamp with time zone	—	No	Fecha y hora del traslado.			
id_medico_indica	integer	FK	No	Referencia a medico.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el médico dejaría el traslado sin referencia, por eso se impide.
id_consulta	integer	FK	Sí	Referencia a consulta.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el consulta dejaría el traslado sin referencia, por eso se impide.
id_ingreso	integer	FK	Sí	Referencia a ingreso.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el ingreso dejaría el traslado sin referencia, por eso se impide.
id_unidad_origen	integer	FK	No	Referencia a unidad_medica.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el unidad dejaría el traslado sin referencia, por eso se impide.
tipo_traslado	character varying(7)	—	No	interno (dentro de la red) o externo (fuera de la red).			
id_unidad_destino	integer	FK	Sí	Referencia a unidad_medica.	RESTRICT	CASCADE	RESTRICT: es historia clínica; borrar el unidad dejaría el traslado sin referencia, por eso se impide.
id_institucion_destino	integer	FK	Sí	Referencia a institucion_externa.	RESTRICT	CASCADE	RESTRICT: no se borra una institución que recibió traslados.

Tabla turno_emergencia

Turnos de 12 horas de la unidad de Emergencias.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_turno	integer	PK	No				
id_unidad	integer	FK	No	Referencia a unidad_medica.	RESTRICT	CASCADE	RESTRICT: los turnos son el historial de cobertura de emergencias y los usan los ingresos.
fecha	date	—	No	Fecha en que inicia el turno.			
jornada	character varying(8)	—	No	diurno (07:00-19:00) o nocturno (19:00-07:00).			

Tabla turno_medico

Médicos asignados a cada turno de emergencia.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_turno	integer	PK/FK	No	Referencia a turno_emergencia.	CASCADE	CASCADE	CASCADE: la asignación de médicos es un detalle que no existe sin su turno; al borrar el turno se borran sus detalles.
id_medico	integer	PK/FK	No	Referencia a medico.	RESTRICT	CASCADE	RESTRICT: no se borra un médico que aparece en el historial de turnos.

Tabla unidad_medica

Unidades de atención de cada hospital: Consulta Externa, Emergencias, Cirugía y Hospitalización.

Atributo	Tipo de dato	Llave	Nulos	Descripción	DELETE	UPDATE	Justificación
id_unidad	integer	PK	No				
id_hospital	integer	FK	No	Referencia a hospital.	CASCADE	CASCADE	CASCADE: la unidad es un detalle que no existe sin su hospital; al borrar el hospital se borran sus detalles. Si ya tiene fichas, esas FK bloquean el borrado.
tipo_unidad	character varying(20)	—	No	Tipo de unidad: Consulta Externa, Emergencias, Cirugía u Hospitalización.			
costo_dia_hospitalizacion	numeric(10,2)	—	Sí	Costo por día internado; solo aplica a Hospitalización.			

Anexo B. Sentencias SQL del Proyecto 2

Contenido completo de los scripts, en el orden en que se ejecutan: sql/ddl, sql/dml, sql/auditoria y sql/consultas.

sql/ddl/01_personal.sql

```
-- Proyecto 2 - BD Hospitales de Occidente
-- 01_personal.sql : supertipo PERSONAL con los subtipos MEDICO y ENFERMERO (Bloque B)
--
-- Estrategia: una tabla para el supertipo y una por subtipo, que comparten la
-- llave primaria (id_medico = id_enfermero = id_personal). PERSONAL guarda los
-- datos comunes y el discriminador tipo_personal.
--
-- Migracion no destructiva: medico y enfermero conservan sus columnas del
-- Proyecto 1 (nombres, DPI, hospital...). Al registrar personal nuevo esos
-- datos se guardan en PERSONAL y tambien en su subtipo.
--
-- Se ejecuta con el rol administrativo.

set role admin;

-- ============================================================
-- supertipo
-- ============================================================

-- datos que comparte todo el personal del hospital
create table personal (
    id_personal integer generated by default as identity primary key,
    tipo_personal varchar(9) not null,  --discriminador
    nombres varchar(100) not null,
    apellidos varchar(100) not null,
    dpi char(13) not null,
    telefono varchar(9),
    direccion varchar(250) not null,
    id_municipio integer not null references municipio(id_municipio)
        on delete restrict on update cascade,
    area varchar(6) not null,
    id_hospital integer not null references hospital(id_hospital)
        on delete restrict on update cascade,
    fecha_contratacion date,
    salario numeric(10,2),
    nivel_acceso varchar(13) not null default 'consulta',
    constraint uq_personal_dpi unique (dpi),
    -- permite que cada subtipo referencie la pareja (id, tipo) y no el de otro tipo, osea que verifica que esa pareja id, tipo
sea unica
    constraint uq_personal_tipo unique (id_personal, tipo_personal),
    -- valida el discriminador: a que subtipo pertenece la persona
    constraint ck_personal_tipo check (tipo_personal in ('medico','enfermero')),
    -- valida que el DPI (CUI) tenga 13 caracteres
    constraint ck_personal_dpi check (length(dpi) = 13),
    -- valida el telefono: 8 digitos, o 9 si lleva guion (5551-2031)
    constraint ck_personal_telefono check (length(telefono) between 8 and 9),
    -- valida si la persona vive en zona urbana o rural
    constraint ck_personal_area check (area in ('Urbana','Rural')),
    -- valida que la fecha de contratacion sea razonable
    constraint ck_personal_contratacion check (fecha_contratacion >= date '1970-01-01'),
    -- valida que el salario no sea negativo
    constraint ck_personal_salario check (salario >= 0),
    -- valida que solo el personal contratado tenga salario (los externos no)
    constraint ck_personal_contratado check ((fecha_contratacion is null) = (salario is null)),
    -- valida el nivel de acceso al sistema de informacion del hospital
    constraint ck_personal_nivel_acceso check (nivel_acceso in ('consulta','operativo','supervisor','administrador'))
);

-- ============================================================
-- se copia el personal existente al supertipo
-- ============================================================
-- direccion, fecha de contratacion, salario y nivel de acceso no existian en el
-- Proyecto 1: se cargan valores ficticios (la direccion queda sin registrar).

insert into personal (id_personal, tipo_personal, nombres, apellidos, dpi, telefono, direccion,
                      id_municipio, area, id_hospital, fecha_contratacion, salario, nivel_acceso)
select m.id_medico, 'medico', m.nombres, m.apellidos, m.dpi, m.telefono, 'Sin registrar',
       m.id_municipio, m.area, m.id_hospital,
       case when m.tipo_medico = 'externo' then null else date '2015-01-15' end,
       case m.tipo_medico when 'externo' then null
                          when 'especialista' then 18500.00
                          when 'general' then 13000.00
                          when 'residente' then 9500.00
                          else 6500.00 end,
       case when m.tipo_medico = 'externo' then 'consulta' else 'operativo' end
  from medico m;
```

```
insert into personal (id_personal, tipo_personal, nombres, apellidos, dpi, telefono, direccion,
                      id_municipio, area, id_hospital, fecha_contratacion, salario, nivel_acceso)
select e.id_enfermero, 'enfermero', e.nombres, e.apellidos, e.dpi, e.telefono, 'Sin registrar',
       h.id_municipio, 'Urbana', e.id_hospital,
       date '2016-03-01',
       case e.tipo when 'registrado' then 7200.00 else 3800.00 end,
       'operativo'
  from enfermero e
  join hospital h on h.id_hospital = e.id_hospital;

-- se copiaron ids hasta el 2005: el contador sigue en el 2006
--osea que el siguiente personal que se registre tendra id_personal = 2006
-- ya que todo el personal existente se copio al supertipo, y los ids de medico y enfermero se conservaron y eran del 1 al 2005,
entonces el siguiente id_personal que se genere sera 2006
alter table personal alter column id_personal restart with 2006;

-- ============================================================
-- subtipos: comparten la llave del supertipo
-- ============================================================

-- el id del subtipo ya no se genera solo: es el id_personal de la persona

--aqui quitamos la generacion automatica de id_medico porque ahora se genera en la tabla personal y se copia a medico y enfermero,
-- ya que el id_medico y id_enfermero son iguales al id_personal del supertipo
alter table medico alter column id_medico drop identity;
alter table enfermero alter column id_enfermero drop identity;


--aqui agregamos la columna tipo_personal a medico y enfermero, para que apunten al supertipo personal y asi poder hacer la
herencia de tablas
--se añade la restriccion de que el tipo_personal de medico sea 'medico' y el de enfermero sea 'enfermero', para que no se pueda
insertar un medico con tipo_personal = 'enfermero' o viceversa
alter table medico
    add column tipo_personal varchar(9) not null default 'medico',
    -- valida que la fila del subtipo medico apunte a personal de tipo medico
    add constraint ck_medico_tipo_personal check (tipo_personal = 'medico'), --aqui se valida que el tipo_personal de medico sea
'medico'
    add constraint fk_medico_personal foreign key (id_medico, tipo_personal)-- aqui se añade la restriccion de que el id_medico y
tipo_personal de medico apunten a personal de tipo medico
        references personal(id_personal, tipo_personal)
        on delete cascade on update cascade;

alter table enfermero
    add column tipo_personal varchar(9) not null default 'enfermero',
    -- valida que la fila del subtipo enfermero apunte a personal de tipo enfermero
    add constraint ck_enfermero_tipo_personal check (tipo_personal = 'enfermero'),
    add constraint fk_enfermero_personal foreign key (id_enfermero, tipo_personal)
        references personal(id_personal, tipo_personal)
        on delete cascade on update cascade;

reset role;
```

sql/ddl/02_jerarquia.sql

```
-- Proyecto 2 - BD Hospitales de Occidente
-- 02_jerarquia.sql : cadena de mando del personal medico (Bloque B, relacion recursiva)
--
-- Cada medico puede tener un superior directo dentro de su hospital (jefe de
-- servicio o director medico). El director de cada hospital es la cima de la
-- cadena: no tiene superior. Los medicos externos no estan contratados, por
-- lo que pueden quedar fuera de la cadena.
--
-- Se ejecuta con el rol administrativo.

set role admin;

alter table medico
    add column cargo varchar(15) not null default 'medico',
    add column id_medico_superior integer references medico(id_medico)
        on delete set null on update cascade,
    -- valida el cargo del medico dentro de la cadena de mando
    add constraint ck_medico_cargo check (cargo in ('director_medico','jefe_servicio','medico')),
    -- valida que nadie sea su propio superior
    add constraint ck_medico_superior_propio check (id_medico_superior <> id_medico),
    -- valida que el director medico no tenga superior: es la cima de la cadena
    add constraint ck_medico_director_sin_superior check (cargo <> 'director_medico' or id_medico_superior is null);

reset role;
```

sql/ddl/03_catalogos.sql

```
-- Proyecto 2 - BD Hospitales de Occidente
-- 03_catalogos.sql : catalogos para los CHECK que tenian muchas opciones
--
-- Cada catalogo usa como llave el mismo codigo que ya se guardaba en la
```

```
-- columna, asi los datos existentes no cambian: solo se quita el CHECK y se
-- pone una llave foranea hacia el catalogo.

set role admin;

-- formas en que se entrega un medicamento
create table forma_farmaceutica (
    codigo varchar(12) primary key,
    descripcion varchar(100) not null
);

insert into forma_farmaceutica values
('tableta', 'Tableta sólida para tomar por vía oral'),
('capsula', 'Cápsula para tomar por vía oral'),
('jarabe', 'Líquido espeso para tomar por vía oral'),
('suspension', 'Líquido con partículas que se agita antes de tomar'),
('inyectable', 'Solución para aplicar con jeringa'),
('crema', 'Uso tópico sobre la piel'),
('gotas', 'Gotas oftálmicas, óticas u orales'),
('inhalador', 'Dosis que se inhala por la boca');

-- funcion que cumple un instrumento en la cirugia
create table funcion_instrumento (
    codigo varchar(12) primary key,
    descripcion varchar(100) not null
);

insert into funcion_instrumento values
('corte', 'Corta o diseca tejido'),
('contencion', 'Sujeta o sostiene tejidos o agujas'),
('hemostatica', 'Detiene el sangrado'),
('retractor', 'Separa los bordes de la herida'),
('accesorio', 'Apoyo durante el procedimiento'),
('implante', 'Queda dentro del paciente'),
('otro', 'Otra función');

-- funcion que cumple un equipo medico
create table funcion_equipo (
    codigo varchar(14) primary key,
    descripcion varchar(100) not null
);

insert into funcion_equipo values
('exploracion', 'Permite revisar al paciente'),
('diagnostico', 'Ayuda a identificar la enfermedad'),
('tratamiento', 'Se usa para tratar al paciente'),
('rehabilitacion', 'Apoya la recuperación'),
('otro', 'Otra función');

-- clasificacion del riesgo anestesico de la ASA
create table clasificacion_asa (
    codigo varchar(3) primary key,
    descripcion varchar(100) not null
);

insert into clasificacion_asa values
('I', 'Paciente sano'),
('II', 'Enfermedad sistémica leve'),
('III', 'Enfermedad sistémica grave'),
('IV', 'Enfermedad grave que amenaza la vida'),
('V', 'Paciente moribundo que no sobrevivirá sin la operación'),
('VI', 'Muerte cerebral, donante de órganos');

-- modalidad del medico dentro del hospital
create table tipo_medico (
    codigo varchar(12) primary key,
    descripcion varchar(100) not null
);

insert into tipo_medico values
('general', 'Médico general contratado'),
('especialista', 'Médico especialista contratado'),
('residente', 'Médico residente en formación'),
('interno', 'Médico interno en formación'),
('externo', 'Cirujano de otra institución que solo llega a operar');

-- medio por el que se asigna una cita de consulta externa
create table medio_asignacion (
    codigo varchar(20) primary key,
    descripcion varchar(100) not null
);

insert into medio_asignacion values
('telefonico', 'Llamada al número de Hospitales de Occidente'),
```

```
('correo_institucional', 'Correo institucional'),
('recepcion_general', 'Recepción general del hospital'),
('recepcion_medico', 'Recepción del médico');

-- se cambia cada CHECK por la llave foranea a su catalogo
alter table medicamento drop constraint ck_medicamento_forma;
alter table medicamento add constraint fk_medicamento_forma
    foreign key (forma_farmaceutica) references forma_farmaceutica(codigo)
    on delete restrict on update cascade;

alter table instrumento drop constraint ck_instrumento_funcion;
alter table instrumento add constraint fk_instrumento_funcion
    foreign key (funcion) references funcion_instrumento(codigo)
    on delete restrict on update cascade;

alter table equipo drop constraint ck_equipo_funcion;
alter table equipo add constraint fk_equipo_funcion
    foreign key (funcion) references funcion_equipo(codigo)
    on delete restrict on update cascade;

alter table chequeo_preanestesico drop constraint ck_chequeo_asa;
alter table chequeo_preanestesico add constraint fk_chequeo_asa
    foreign key (clasificacion_asa) references clasificacion_asa(codigo)
    on delete restrict on update cascade;

alter table medico drop constraint ck_medico_tipo;
alter table medico add constraint fk_medico_tipo
    foreign key (tipo_medico) references tipo_medico(codigo)
    on delete restrict on update cascade;

alter table cita drop constraint ck_cita_medio;
alter table cita add constraint fk_cita_medio
    foreign key (medio_asignacion) references medio_asignacion(codigo)
    on delete restrict on update cascade;

reset role;
```

sql/ddl/04_laboratorio_y_clinicas.sql

```
-- Proyecto 2 - BD Hospitales de Occidente
-- 04_laboratorio_y_clinicas.sql : tablas que faltaban en consulta externa
--
-- El enunciado del Proyecto 1 pide registrar la clinica donde se atiende la
-- cita (hospital y numero de clinica) y las ordenes de laboratorio que el
-- medico da en la consulta. Se ejecuta sobre la base ya cargada: las clinicas
-- se crean a partir de las que ya aparecen en las citas.

set role admin;

-- clinicas (consultorios) de la unidad de consulta externa de cada hospital
create table clinica (
    id_clinica integer generated by default as identity primary key,
    id_unidad integer not null references unidad_medica(id_unidad)
        on delete restrict on update cascade,
    numero varchar(10) not null,
    -- no se repite el numero de clinica dentro de la misma unidad
    constraint uq_clinica_numero unique (id_unidad, numero)
);

-- una clinica por cada numero que ya se usaba en las citas, en la unidad de
-- consulta externa del hospital del medico
--osea que ponemos o agregamos a la tabla clinica los numeros de
-- clinica que ya se usaban en las citas, y los asociamos a la unidad de
-- consulta externa del hospital del medico que atendio la cita
insert into clinica (id_unidad, numero)
select distinct u.id_unidad, c.numero_clinica
from cita c
inner join medico m on m.id_medico = c.id_medico
inner join unidad_medica u on u.id_hospital = m.id_hospital and u.tipo_unidad = 'Consulta Externa'
order by u.id_unidad, c.numero_clinica;

-- cada cita apunta a su clinica --
--osea que ahora cada cita tendra un id_clinica que apunta a
--la clinica donde se atendio la cita, y ese id_clinica se obtiene de la
-- tabla clinica, buscando la clinica que tiene el mismo numero de clinica y pertenece a
--la unidad de consulta externa del hospital del medico que atendio la cita
alter table cita add column id_clinica integer references clinica(id_clinica)
    on delete restrict on update cascade;


--aqui pone a cada cita el id de su clinica correspondiente, buscando la clinica que
--tiene el mismo numero de clinica y pertenece a la unidad de consulta externa del
--hospital del medico que atendio la cita
update cita c
```

```
set id_clinica = (select cl.id_clinica
                  from clinica cl
                  inner join unidad_medica u on u.id_unidad = cl.id_unidad
                  inner join medico m on m.id_hospital = u.id_hospital
                  where m.id_medico = c.id_medico
                    and u.tipo_unidad = 'Consulta Externa'
                    and cl.numero = c.numero_clinica);

alter table cita alter column id_clinica set not null;--aqui ya pasa a ser obligatoria

-- catalogo de examenes de laboratorio que se pueden ordenar
create table examen_laboratorio (
    id_examen integer generated by default as identity primary key,
    nombre varchar(100) not null,
    descripcion varchar(200),
    costo numeric(10,2) not null,
    constraint uq_examen_nombre unique (nombre),
    -- el costo del examen no puede ser negativo
    constraint ck_examen_costo check (costo >= 0)
);

-- orden de laboratorio que el medico entrega en una consulta
create table orden_laboratorio (
    id_orden integer generated by default as identity primary key,
    id_consulta integer not null references consulta(id_consulta)
        on delete restrict on update cascade,
    fecha date not null,
    indicaciones text
);

-- examenes que lleva cada orden (una orden puede pedir varios examenes)
create table orden_examen (
    id_orden integer not null references orden_laboratorio(id_orden)
        on delete cascade on update cascade,
    id_examen integer not null references examen_laboratorio(id_examen)
        on delete restrict on update cascade,
    primary key (id_orden, id_examen)
);

reset role;
```

sql/dml/01_red_y_personal.sql

```
-- Proyecto 2 - BD Hospitales de Occidente
-- 01_red_y_personal.sql : ampliacion de la red de hospitales y del personal (datos FICTICIOS)
--
-- Se agregan los hospitales 4 y 5 para que los rankings "top 3" comparen
-- entre varios hospitales, y personal suficiente para turnos, consultas y
-- cirugias en todos ellos, con su cadena de mando.

set role admin;

-- ============================================================
-- red de hospitales
-- ============================================================

insert into municipio (id_municipio, id_departamento, nombre) values
(9, 8, 'Huehuetenango'), (10, 8, 'Chiantla'), (11, 19, 'Sololá'),
(12, 19, 'Panajachel'), (13, 17, 'Malacatán'), (14, 15, 'San Sebastián');

insert into hospital (id_hospital, nombre, informacion_especifica, direccion, id_municipio) values
(4, 'Hospital de Occidente Huehuetenango', 'Hospital general de segundo nivel con énfasis en atención materno-infantil.', '5a.
avenida 3-20, zona 1', 9),
(5, 'Hospital de Occidente Sololá', 'Hospital general de segundo nivel para la región del lago.', 'Calle principal 7-15, zona 2',
11);

insert into unidad_medica (id_unidad, id_hospital, tipo_unidad, costo_dia_hospitalizacion) values
(41, 4, 'Consulta Externa', null), (42, 4, 'Emergencias', null), (43, 4, 'Cirugía', null), (44, 4, 'Hospitalización', 480.00),
(51, 5, 'Consulta Externa', null), (52, 5, 'Emergencias', null), (53, 5, 'Cirugía', null), (54, 5, 'Hospitalización', 430.00);

-- los mismos servicios del hospital 1 con el costo propio de cada hospital (id + 300 y + 400)
insert into servicio_atencion (id_servicio, id_unidad, nombre, costo)
select s.id_servicio + 100 * (h.n - 1), s.id_unidad + 10 * (h.n - 1), s.nombre, round(s.costo * h.factor, 2)
  from servicio_atencion s
 cross join (values (4, 0.80), (5, 0.75)) as h(n, factor)
 where s.id_unidad in (12, 13, 14);

insert into camilla (id_camilla, id_unidad, numero)
select 300 + n, 44, n from generate_series(1, 14) n      -- hospital 4, hospitalizacion
union all
select 314 + n, 42, n from generate_series(1, 5) n       -- hospital 4, emergencias
union all
select 400 + n, 54, n from generate_series(1, 10) n      -- hospital 5, hospitalizacion
union all
```

```
select 410 + n, 52, n from generate_series(1, 4) n;      -- hospital 5, emergencias

insert into quirofano (id_quirofano, id_unidad, numero)
select 4 * (h - 1) + n, 10 * h + 3, n
  from generate_series(4, 5) h, generate_series(1, 4) n;

-- ============================================================
-- personal nuevo: primero el supertipo, luego su subtipo
-- ============================================================

insert into personal (id_personal, tipo_personal, nombres, apellidos, dpi, telefono, direccion, id_municipio, area, id_hospital,
fecha_contratacion, salario, nivel_acceso) values
-- hospital 2
(1031, 'medico', 'Rosa María', 'Fuentes Galindo', '2001123450131', '5551-2031', '3 calle 5-12, zona 1', 6, 'Urbana', 2, '2016-02-
01', 13000.00, 'operativo'),
(1032, 'medico', 'Esteban', 'Orozco Bámaca', '2001123450132', '5551-2032', '8 avenida 2-40, zona 3', 6, 'Rural', 2, '2019-07-15',
13000.00, 'operativo'),
(1033, 'medico', 'Leticia', 'Morales Chávez', '2001123450133', '5551-2033', '1 calle 9-03, zona 2', 6, 'Urbana', 2, '2011-03-10',
18500.00, 'operativo'),
(1034, 'medico', 'Iván', 'Coyoy Tzul', '2001123450134', '5551-2034', '6 calle 1-77, zona 4', 13, 'Urbana', 2, '2020-01-20', 18500.
00, 'operativo'),
(1035, 'medico', 'Marisol', 'Ajanel Pérez', '2001123450135', '5551-2035', '2 avenida 6-18, zona 1', 6, 'Urbana', 2, '2023-08-01',
9500.00, 'operativo'),
-- hospital 3
(1036, 'medico', 'Otto René', 'Velásquez Lima', '2001123450136', '5551-2036', '4 calle 3-30, zona 1', 8, 'Urbana', 3, '2014-05-
05', 13000.00, 'operativo'),
(1037, 'medico', 'Brenda', 'Sic Ixcot', '2001123450137', '5551-2037', 'Lotificación El Recreo, casa 14', 14, 'Rural', 3, '2021-11-
02', 13000.00, 'operativo'),
(1038, 'medico', 'Mario', 'Cabrera Monterroso', '2001123450138', '5551-2038', '7 avenida 0-55, zona 1', 8, 'Urbana', 3, '2010-09-
13', 18500.00, 'operativo'),
(1039, 'medico', 'Gloria', 'Tax Chiquitó', '2001123450139', '5551-2039', '5 calle 8-21, zona 2', 8, 'Urbana', 3, '2018-04-23',
18500.00, 'operativo'),
(1040, 'medico', 'Samuel', 'Barrios Juárez', '2001123450140', '5551-2040', '10 calle 2-09, zona 3', 8, 'Urbana', 3, '2017-06-12',
18500.00, 'operativo'),
-- hospital 4
(1041, 'medico', 'Ana Gabriela', 'Recinos Mérida', '2001123450141', '5551-2041', '2 calle 4-50, zona 1', 9, 'Urbana', 4, '2015-01-
12', 13000.00, 'operativo'),
(1042, 'medico', 'Héctor Manuel', 'Gómez Ordóñez', '2001123450142', '5551-2042', 'Aldea Las Lagunas, casa 3', 10, 'Rural', 4,
'2022-02-14', 13000.00, 'operativo'),
(1043, 'medico', 'Carolina', 'Herrera Villatoro', '2001123450143', '5551-2043', '6 avenida 1-15, zona 1', 9, 'Urbana', 4, '2009-
10-01', 18500.00, 'operativo'),
(1044, 'medico', 'Jorge Mario', 'Alvarado Castillo', '2001123450144', '5551-2044', '3 avenida 7-02, zona 5', 9, 'Urbana', 4,
'2016-08-08', 18500.00, 'operativo'),
(1045, 'medico', 'Patricia', 'López Mazariegos', '2001123450145', '5551-2045', '9 calle 3-44, zona 1', 9, 'Urbana', 4, '2013-03-
18', 18500.00, 'operativo'),
(1046, 'medico', 'Kevin', 'Domingo Pascual', '2001123450146', '5551-2046', '1 avenida 2-08, zona 2', 10, 'Rural', 4, '2025-01-06',
 6500.00, 'operativo'),
-- hospital 5
(1047, 'medico', 'Lucía', 'Cumes Xajil', '2001123450147', '5551-2047', 'Calle Santander 4-11', 12, 'Urbana', 5, '2014-07-21',
13000.00, 'operativo'),
(1048, 'medico', 'Diego', 'Yaxón Bocel', '2001123450148', '5551-2048', 'Cantón Pujujil, casa 8', 11, 'Rural', 5, '2020-09-28',
13000.00, 'operativo'),
(1049, 'medico', 'Sandra', 'Tuy Guarcax', '2001123450149', '5551-2049', '7 calle 2-33, zona 1', 11, 'Urbana', 5, '2008-02-04',
18500.00, 'operativo'),
(1050, 'medico', 'Alberto', 'Ramos Sajquiy', '2001123450150', '5551-2050', '3 avenida 5-60, zona 2', 11, 'Urbana', 5, '2012-12-
03', 18500.00, 'operativo'),
(1051, 'medico', 'Mónica', 'Pirir Cutzal', '2001123450151', '5551-2051', 'Calle del Lago 1-09', 12, 'Urbana', 5, '2019-05-13',
18500.00, 'operativo'),
-- enfermeros de todos los hospitales
(2006, 'enfermero', 'Rosa', 'Xicará Pop', '2001123450206', '5552-2006', '2 calle 3-10, zona 1', 1, 'Urbana', 1, '2017-04-03',
7200.00, 'operativo'),
(2007, 'enfermero', 'Julio', 'Chay Tum', '2001123450207', '5552-2007', 'Aldea Chichimecas, casa 2', 5, 'Rural', 1, '2024-06-17',
3800.00, 'operativo'),
(2008, 'enfermero', 'Mirna', 'Ochoa Bravo', '2001123450208', '5552-2008', '5 avenida 1-22, zona 3', 6, 'Urbana', 2, '2016-10-10',
7200.00, 'operativo'),
(2009, 'enfermero', 'Wilson', 'Escobar Ruiz', '2001123450209', '5552-2009', '4 calle 6-07, zona 2', 13, 'Rural', 2, '2021-03-01',
7200.00, 'operativo'),
(2010, 'enfermero', 'Yesenia', 'Cardona Aguilar', '2001123450210', '5552-2010', '1 calle 2-80, zona 1', 6, 'Urbana', 2, '2025-02-
03', 3800.00, 'operativo'),
(2011, 'enfermero', 'Ronaldo', 'Pineda Sosa', '2001123450211', '5552-2011', '6 calle 4-15, zona 1', 8, 'Urbana', 3, '2015-08-24',
7200.00, 'operativo'),
(2012, 'enfermero', 'Ingrid', 'Toc Chan', '2001123450212', '5552-2012', 'Colonia Las Flores, casa 9', 14, 'Rural', 3, '2019-01-
14', 7200.00, 'operativo'),
(2013, 'enfermero', 'Elmer', 'Yac Sontay', '2001123450213', '5552-2013', '3 avenida 8-30, zona 2', 8, 'Urbana', 3, '2024-09-09',
3800.00, 'operativo'),
(2014, 'enfermero', 'Dora', 'Matías Gaspar', '2001123450214', '5552-2014', '4 calle 1-40, zona 1', 9, 'Urbana', 4, '2014-11-17',
7200.00, 'operativo'),
(2015, 'enfermero', 'Fredy', 'Mendoza Hidalgo', '2001123450215', '5552-2015', 'Caserío El Calvario, casa 5', 10, 'Rural', 4,
'2020-05-04', 7200.00, 'operativo'),
(2016, 'enfermero', 'Sonia', 'Jiménez Sales', '2001123450216', '5552-2016', '2 avenida 3-03, zona 4', 9, 'Urbana', 4, '2025-03-
10', 3800.00, 'operativo'),
(2017, 'enfermero', 'Pablo', 'Sicajau Tzep', '2001123450217', '5552-2017', 'Calle Rancho Grande 2-14', 12, 'Urbana', 5, '2016-06-
06', 7200.00, 'operativo'),
(2018, 'enfermero', 'Claudia', 'Batz Sequén', '2001123450218', '5552-2018', 'Cantón Xajaxac, casa 1', 11, 'Rural', 5, '2018-08-
```

```
20', 7200.00, 'operativo'),
(2019, 'enfermero', 'Edwin', 'Coj Lacán', '2001123450219', '5552-2019', '1 calle 5-25, zona 1', 11, 'Urbana', 5, '2024-10-07',
3800.00, 'operativo'),
(2020, 'enfermero', 'Vilma', 'Ixcoy Boj', '2001123450220', '5552-2020', '9 avenida 4-12, zona 2', 1, 'Urbana', 1, '2018-02-26',
7200.00, 'operativo');

-- subtipo medico: los datos comunes (nombres, DPI, hospital...) se toman de personal
insert into medico (id_medico, nombres, apellidos, dpi, telefono, id_hospital, id_municipio, area,
                    id_especialidad, tipo_medico, costo_primera_consulta, horario_inicio, horario_fin)
select p.id_personal, p.nombres, p.apellidos, p.dpi, p.telefono, p.id_hospital, p.id_municipio, p.area,
       d.id_especialidad, d.tipo_medico, d.costo, d.inicio::time, d.fin::time
  from personal p
 inner join (values
(1031, 307, 'general', 170.00, '08:00', '12:00'),
(1032, 307, 'general', 160.00, '08:00', '12:00'),
(1033, 301, 'especialista', 230.00, '08:00', '11:30'),
(1034, 312, 'especialista', 190.00, '08:00', '12:00'),
(1035, 315, 'residente', null, null, null),
(1036, 307, 'general', 165.00, '08:00', '12:00'),
(1037, 307, 'general', 150.00, '08:00', '12:00'),
(1038, 312, 'especialista', 210.00, '08:00', '12:00'),
(1039, 302, 'especialista', 175.00, '09:00', '12:00'),
(1040, 326, 'especialista', null, null, null),
(1041, 307, 'general', 160.00, '08:00', '12:00'),
(1042, 307, 'general', 150.00, '08:00', '12:00'),
(1043, 301, 'especialista', 215.00, '08:00', '11:30'),
(1044, 304, 'especialista', 205.00, '09:00', '12:00'),
(1045, 315, 'especialista', null, null, null),
(1046, 318, 'interno', null, null, null),
(1047, 307, 'general', 150.00, '08:00', '12:00'),
(1048, 307, 'general', 145.00, '08:00', '12:00'),
(1049, 312, 'especialista', 200.00, '08:00', '12:00'),
(1050, 324, 'especialista', null, null, null),
(1051, 311, 'especialista', 160.00, '09:00', '12:00')
       ) as d(id_medico, id_especialidad, tipo_medico, costo, inicio, fin) on d.id_medico = p.id_personal;

-- subtipo enfermero: tambien con sus datos comunes tomados de personal
insert into enfermero (id_enfermero, nombres, apellidos, dpi, telefono, id_hospital, tipo)
select id_personal, nombres, apellidos, dpi, telefono, id_hospital,
       case when salario < 5000 then 'practicante' else 'registrado' end
  from personal
 where tipo_personal = 'enfermero'
   and id_personal not in (select id_enfermero from enfermero);

-- ============================================================
-- cadena de mando: director -> jefes de servicio -> medicos
-- (de arriba hacia abajo, para que cada superior ya exista)
-- ============================================================

update medico set cargo = 'director_medico' where id_medico in (1002, 1033, 1038, 1043, 1049);
update medico set cargo = 'jefe_servicio' where id_medico in (1005, 1011, 1031, 1012, 1036, 1009, 1041, 1045, 1047, 1050);

update medico m set id_medico_superior = s.superior
  from (values
        -- hospital 1: dos servicios; en consulta externa la cadena llega a 4 niveles
        (1005, 1002), (1011, 1002),
        (1001, 1005), (1006, 1005), (1008, 1006),
        (1003, 1011), (1004, 1011), (1010, 1011), (1013, 1011), (1015, 1011), (1017, 1011), (1018, 1017),
        -- hospital 2
        (1031, 1033), (1012, 1033), (1032, 1031), (1035, 1031), (1014, 1012), (1019, 1012), (1034, 1012),
        -- hospital 3
        (1036, 1038), (1009, 1038), (1037, 1036), (1040, 1036), (1016, 1009), (1039, 1009),
        -- hospital 4
        (1041, 1043), (1045, 1043), (1042, 1041), (1046, 1042), (1044, 1045),
        -- hospital 5
        (1047, 1049), (1050, 1049), (1048, 1047), (1051, 1050)
       ) as s(medico, superior)
 where m.id_medico = s.medico;

-- nivel de acceso y salario segun el cargo
update personal p
   set nivel_acceso = case m.cargo when 'director_medico' then 'administrador' else 'supervisor' end,
       salario = case m.cargo when 'director_medico' then 28000.00 else 22000.00 end
  from medico m
 where m.id_medico = p.id_personal and m.cargo in ('director_medico','jefe_servicio');

-- el ultimo id usado es el 2020: el contador sigue en el 2021
alter table personal alter column id_personal restart with 2021;

reset role;
```

sql/dml/02_pacientes.sql

```
-- Proyecto 2 - BD Hospitales de Occidente
```

```
-- 02_pacientes.sql : ampliacion de pacientes y encargados (datos FICTICIOS)
--
-- Los pacientes 8 a 67 se generan combinando listas de nombres y apellidos
-- ficticios con formulas fijas, para que la carga sea reproducible (siempre
-- produce los mismos datos) y cubra ambos sexos, todas las edades, zonas
-- urbanas y rurales y los municipios de los 5 hospitales.

set role admin;

insert into paciente (id_paciente, numero_expediente, nombres, apellidos, fecha_nacimiento, sexo, dpi,
                      telefono, id_municipio, area, id_estado_civil, numero_seguro_social)
select n,
       format('HG%s-%s', 1 + n % 5, lpad(n::text, 4, '0')),
       case when n % 2 = 0
            then (array['María José','Ana Belén','Lucía','Carmen','Rosa Elena','Andrea','Sofía','Gabriela','Paola','Verónica',
                        'Marta','Julia','Claudia','Daniela','Elena'])[1 + n % 15]
            else (array['José Luis','Carlos','Miguel','Juan Pablo','Luis Fernando','Diego','Mario','Jorge','Pedro','Andrés',
                        'Ricardo','Héctor','Fernando','Rafael','Óscar'])[1 + n % 15] end,
       (array['García','López','Pérez','Morales','Hernández','Castillo','Ramírez','Gómez','Méndez','Cruz','Ortiz','Fuentes',
              'Vásquez','Juárez','Sosa','Tzul','Coyoy','Ajcá','Xicará','Batz'])[1 + n % 20] || ' ' ||
       (array['Paz','Rojas','Ixcot','Sic','Barrios','Chávez','Ruiz','Lima','Soto','Pop','Tum','Chan'])[1 + (n * 7) % 12],
       fecha_nac,
       case when n % 2 = 0 then 'Femenino' else 'Masculino' end,
       case when extract(year from age(fecha_nac)) >= 18 then '41' || lpad(n::text, 11, '0') end,
       format('55%s-%s', lpad((10 + n % 90)::text, 2, '0'), lpad((1000 + n * 37 % 9000)::text, 4, '0')),
       (array[1, 2, 3, 4, 5, 6, 13, 8, 14, 9, 10, 11, 12])[1 + n % 13],
       case when n % 3 = 0 then 'Rural' else 'Urbana' end,
       case when extract(year from age(fecha_nac)) < 18 then 1 else 1 + n % 5 end,
       case when n % 4 = 0 then 'IGSS-' || (50000 + n * 113) end
  from (select n, date '1940-03-01' + (n * 4517) % 30500 as fecha_nac
          from generate_series(8, 67) n) g;

insert into encargado (id_encargado, nombres, apellidos, dpi, telefono, id_municipio, area)
select 300 + n,
       (array['Marta','Juan','Lidia','Rodrigo','Elsa','Pedro','Olga','Mynor','Silvia','Erick'])[1 + n % 10],
       (array['Pérez','Coyoy','Ruiz','Batz','Sosa','Castillo','Ixcot','Fuentes','Lima','Morales'])[1 + (n * 3) % 10] || ' ' ||
       (array['García','Tzul','Paz','Chan','Rojas'])[1 + n % 5],
       '42' || lpad(n::text, 11, '0'),
       format('56%s-%s', lpad((10 + n)::text, 2, '0'), lpad((2000 + n * 41)::text, 4, '0')),
       (array[1, 6, 8, 9, 11])[1 + n % 5],
       case when n % 4 = 0 then 'Rural' else 'Urbana' end
  from generate_series(4, 23) n;

-- cada menor de edad tiene uno o dos encargados (madre o padre); algunos adultos, un hermano
insert into paciente_encargado (id_paciente, id_encargado, id_parentesco)
select p.id_paciente, 304 + (p.id_paciente % 20), case when p.id_paciente % 2 = 0 then 1 else 2 end
  from paciente p
 where p.id_paciente >= 8 and extract(year from age(p.fecha_nacimiento)) < 18
union all
select p.id_paciente, 304 + ((p.id_paciente + 7) % 20), case when p.id_paciente % 2 = 0 then 2 else 1 end
  from paciente p
 where p.id_paciente >= 8 and extract(year from age(p.fecha_nacimiento)) < 12
union all
select p.id_paciente, 304 + ((p.id_paciente + 3) % 20), 3
  from paciente p
 where p.id_paciente >= 8 and extract(year from age(p.fecha_nacimiento)) >= 65;

select setval(pg_get_serial_sequence('paciente', 'id_paciente'), (select max(id_paciente) from paciente));
select setval(pg_get_serial_sequence('encargado', 'id_encargado'), (select max(id_encargado) from encargado));

reset role;
```

sql/dml/03_consulta_externa.sql

```
-- Proyecto 2 - BD Hospitales de Occidente
-- 03_consulta_externa.sql : citas, consultas, recetas y calificaciones de enero a octubre de 2026 (datos FICTICIOS)
--
-- Las filas se generan con formulas fijas sobre generate_series, asi que la
-- carga siempre produce los mismos datos. Rangos de id: citas 101-320; la
-- consulta y la receta usan el mismo id que su cita.

set role admin;

-- diagnosticos frecuentes en consulta externa (CIE-10)
insert into diagnostico (id_diagnostico, codigo_cie10, descripcion) values
(12, 'J06.9', 'Infección aguda de las vías respiratorias superiores, no especificada'),
(13, 'K29.7', 'Gastritis, no especificada'),
(14, 'M54.5', 'Lumbago no especificado'),
(15, 'L20.9', 'Dermatitis atópica, no especificada'),
(16, 'H10.9', 'Conjuntivitis, no especificada'),
(17, 'F41.1', 'Trastorno de ansiedad generalizada'),
(18, 'Z00.0', 'Examen médico general'),
(19, 'N39.0', 'Infección de vías urinarias, sitio no especificado');
-- ============================================================
```

```
-- citas: 220 repartidas entre los medicos con horario de consulta externa
-- ============================================================
-- estado: las futuras quedan programadas; de las pasadas 1 de cada 10 se
-- cancela y 1 de cada 10 se reprograma. Costo: reconsulta 75%, referido 80%.

insert into cita (id_cita, id_paciente, id_medico, fecha_hora, numero_clinica, medio_asignacion, estado,
                  es_reconsulta, es_referido, cancelada_sin_aviso, costo_base, costo_final)
select 100 + g.n,
       1 + (g.n * 7) % 67,
       m.id_medico,
       f.fecha_hora,
       'C-' || (100 + m.id_medico % 100),
       (array['telefonico','correo_institucional','recepcion_general','recepcion_medico'])[1 + g.n % 4],
       e.estado,
       g.n % 3 = 0,
       g.n % 3 <> 0 and g.n % 7 = 0,
       e.estado = 'cancelada' and g.n % 4 = 0,
       m.costo_primera_consulta,
       round(m.costo_primera_consulta * case when g.n % 3 = 0 then 0.75 when g.n % 7 = 0 then 0.80 else 1 end, 2)
  from generate_series(1, 220) g(n)
 cross join lateral (
        select id_medico, costo_primera_consulta, horario_inicio
          from medico
         where horario_inicio is not null
         order by id_medico
        offset (g.n * 5) % (select count(*) from medico where horario_inicio is not null)
         limit 1) m
 cross join lateral (
        select ((date '2026-01-05' + (g.n * 13) % 290) + m.horario_inicio + (g.n % 5) * interval '30 minutes')
               at time zone 'America/Guatemala' as fecha_hora) f
 cross join lateral (
        select case when f.fecha_hora > timestamptz '2026-10-20 23:59-06' then 'programada'
                    when g.n % 10 = 0 then 'cancelada'
                    when g.n % 10 = 5 then 'reprogramada'
                    else 'realizada' end as estado) e;

-- ============================================================
-- consultas, recetas y medicamentos de las citas realizadas
-- ============================================================

insert into consulta (id_consulta, id_cita, id_diagnostico, otros_datos, notas_observaciones)
select c.id_cita, c.id_cita, 1 + c.id_cita % 19,
       case when c.es_referido then 'Paciente referido por centro de salud.' end,
       case when c.es_reconsulta then 'Reconsulta de seguimiento; evolución favorable.'
            else 'Primera consulta; se indica tratamiento y control.' end
  from cita c
 where c.id_cita > 100 and c.estado = 'realizada';

-- 6 de cada 7 consultas terminan con receta; la mitad agenda proxima cita a 30 dias
insert into receta (id_receta, id_consulta, fecha_proxima_cita)
select co.id_consulta, co.id_consulta,
       case when co.id_consulta % 2 = 0 then ci.fecha_hora::date + 30 end
  from consulta co
  join cita ci on ci.id_cita = co.id_cita
 where co.id_consulta > 100 and co.id_consulta % 7 <> 0;

insert into receta_medicamento (id_receta, id_medicamento, dosis, duracion)
select r.id_receta, 1 + r.id_receta % 10, '1 cada 12 horas', (7 * (1 + r.id_receta % 4)) || ' días'
  from receta r where r.id_receta > 100
union all
select r.id_receta, 1 + (r.id_receta + 3) % 10, '1 cada 8 horas si hay dolor', '5 días'
  from receta r where r.id_receta > 100 and r.id_receta % 3 = 0;

-- ============================================================
-- calificaciones
-- ============================================================

-- 7 de cada 10 pacientes califican al medico que los atendio. Cada medico
-- tiene un nivel base distinto (1 a 5) para que el ranking tenga diferencias.
insert into calificacion (id_paciente, fecha, puntuacion, comentario, id_medico, id_consulta)
select ci.id_paciente, ci.fecha_hora::date,
       least(5, greatest(1, 1 + ci.id_medico % 5 + (co.id_consulta % 3) - 1)),
       case co.id_consulta % 4 when 0 then 'Muy buena explicación del tratamiento.'
                               when 1 then 'Tuve que esperar bastante.' end,
       ci.id_medico, co.id_consulta
  from consulta co
  join cita ci on ci.id_cita = co.id_cita
 where co.id_consulta > 100 and co.id_consulta % 10 < 7;

-- calificaciones a los hospitales, enfermeros y encargados
insert into calificacion (id_paciente, fecha, puntuacion, comentario, id_hospital)
select 1 + (n * 3) % 67, date '2026-01-10' + (n * 7) % 280,
       least(5, greatest(1, 2 + (n * 7 + n / 5) % 4)), null, 1 + n % 5
  from generate_series(1, 40) n;
```

```
insert into calificacion (id_paciente, fecha, puntuacion, comentario, id_enfermero)
select 1 + (n * 5) % 67, date '2026-02-01' + (n * 11) % 250,
       2 + n % 4, case when n % 3 = 0 then 'Atención amable y oportuna.' end,
       (select id_enfermero from enfermero order by id_enfermero offset n % 20 limit 1)
  from generate_series(1, 20) n;

insert into calificacion (id_paciente, fecha, puntuacion, comentario, id_encargado)
select pe.id_paciente, date '2026-03-01' + pe.id_encargado % 200, 3 + pe.id_encargado % 3,
       'Acompañó al paciente en todos los trámites.', pe.id_encargado
  from paciente_encargado pe
 where pe.id_paciente between 8 and 30 and pe.id_parentesco in (1, 2);

reset role;
```

sql/dml/04_emergencias_hospitalizacion.sql

```
-- Proyecto 2 - BD Hospitales de Occidente
-- 04_emergencias_hospitalizacion.sql : turnos, ingresos, egresos y traslados de enero a octubre de 2026 (datos FICTICIOS)
--
-- Rangos de id: turnos 1000-1949; ingresos a emergencias 1001-1155 y a
-- hospitalizacion 2001-2060; el egreso usa el mismo id que su ingreso.

set role admin;

-- ============================================================
-- turnos de emergencia: cada 3 dias, jornada diurna y nocturna, en los 5 hospitales
-- ============================================================
-- id = 1000 + dia*10 + (hospital-1)*2 + (0 diurno | 1 nocturno)

insert into turno_emergencia (id_turno, id_unidad, fecha, jornada)
select 1000 + d * 10 + (h - 1) * 2 + j, 10 * h + 2, date '2026-01-05' + 3 * d,
       case j when 0 then 'diurno' else 'nocturno' end
  from generate_series(0, 94) d, generate_series(1, 5) h, generate_series(0, 1) j
 -- se omiten los turnos que ya existian desde el Proyecto 1
 where not exists (select 1 from turno_emergencia x
                   where x.id_unidad = 10 * h + 2 and x.fecha = date '2026-01-05' + 3 * d
                     and x.jornada = case j when 0 then 'diurno' else 'nocturno' end);

-- medicos de emergencia de cada hospital; donde hay 3, cada turno lo cubren 2
-- y la rotacion hace que unos acumulen mas turnos que otros
with candidato(id_hospital, posicion, id_medico) as (
    values (1, 0, 1001), (1, 1, 1005), (1, 2, 1006),
           (2, 0, 1031), (2, 1, 1032), (2, 2, 1035),
           (3, 0, 1036), (3, 1, 1037),
           (4, 0, 1041), (4, 1, 1042), (4, 2, 1046),
           (5, 0, 1047), (5, 1, 1048)
), turno as (
    select t.id_turno, u.id_hospital, (t.id_turno - 1000) / 10 + (t.id_turno % 2) as rotacion
      from turno_emergencia t join unidad_medica u on u.id_unidad = t.id_unidad
     where t.id_turno >= 1000
)
insert into turno_medico (id_turno, id_medico)
select t.id_turno, c.id_medico
  from turno t
  join candidato c on c.id_hospital = t.id_hospital
 where (select count(*) from candidato x where x.id_hospital = t.id_hospital) = 2
    or c.posicion <> case when t.rotacion % 6 = 0 then 0 when t.rotacion % 6 in (1, 2) then 1 else 2 end;

-- ============================================================
-- ingresos a emergencias (150), dentro de un turno de su hospital
-- ============================================================
-- la espera por falta de camilla depende de cada hospital: unos casi siempre
-- tienen camilla libre al llegar el paciente y otros no

insert into ingreso (id_ingreso, id_paciente, id_unidad, id_servicio, id_medico_encargado, id_camilla, id_turno,
                     fecha_hora_ingreso, motivo_ingreso, diagnostico_presuntivo, prioridad_triage, turnos_espera)
select 1000 + g.n, 1 + (g.n * 11) % 67, 10 * g.h + 2, (g.h - 1) * 100 + 1 + g.n % 7,
       (select id_medico from turno_medico tm where tm.id_turno = g.id_turno order by id_medico
         offset g.n % (select count(*) from turno_medico x where x.id_turno = g.id_turno) limit 1),
       (array[25, 113, 211, 315, 411])[g.h] + g.n % (array[6, 4, 4, 5, 4])[g.h],
       g.id_turno,
       ((t.fecha + case t.jornada when 'diurno' then time '07:00' else time '19:00' end)
         at time zone 'America/Guatemala') + ((g.n * 37) % 700) * interval '1 minute',
       (array['Dolor torácico.', 'Fiebre alta y deshidratación.', 'Traumatismo por caída.', 'Dificultad respiratoria.',
              'Dolor abdominal agudo.', 'Herida cortante en extremidad.', 'Crisis hipertensiva.'])[1 + g.n % 7],
       (array['Síndrome coronario en estudio.', 'Cuadro viral.', 'Contusión.', 'Crisis asmática.',
              'Abdomen agudo en estudio.', 'Herida superficial.', 'Hipertensión descompensada.'])[1 + g.n % 7],
       1 + (g.n * 3) % 5,
       case when ((g.n / 5) * 7) % 10 < (array[8, 5, 7, 4, 6])[g.h] then 0 else 1 + (g.n / 5) % 3 end
  from (select n, 1 + n % 5 as h, 1000 + ((n * 7) % 95) * 10 + (n % 5) * 2 + n % 2 as id_turno
          from generate_series(1, 150) n) g
  join turno_emergencia t on t.id_turno = g.id_turno;

-- egresos de 6 de cada 7 ingresos; algunos fallecen y otros salen sin consentimiento medico
```

```
insert into egreso (id_egreso, id_ingreso, id_medico, id_diagnostico_principal, fecha_hora_egreso, motivo_egreso,
                    codigo_egreso, sin_consentimiento, motivo_sin_consentimiento)
select i.id_ingreso, i.id_ingreso, i.id_medico_encargado, 1 + (n * 5) % 19,
       i.fecha_hora_ingreso + (4 + n % 40) * interval '1 hour',
       case when n % 23 = 0 then 'Fallecimiento.' when n % 9 = 0 then 'Traslado a hospitalización.'
            else 'Mejoría clínica.' end,
       case when n % 23 = 0 then 'muerto' else 'vivo' end,
       n % 13 = 0 and n % 23 <> 0,
       case when n % 13 = 0 and n % 23 <> 0 then 'El paciente decidió retirarse antes del alta.' end
  from ingreso i
 cross join lateral (select i.id_ingreso - 1000 as n) x
 where i.id_ingreso between 1001 and 1150 and n % 7 <> 0;

-- los egresos por traslado a hospitalizacion tienen su ficha de traslado
insert into traslado (id_traslado, fecha_hora, id_medico_indica, id_ingreso, id_unidad_origen, tipo_traslado, id_unidad_destino)
select e.id_egreso, e.fecha_hora_egreso - interval '15 minutes', e.id_medico, e.id_ingreso,
       i.id_unidad, 'interno', i.id_unidad + 2
  from egreso e join ingreso i on i.id_ingreso = e.id_ingreso
 where e.id_egreso between 1001 and 1150 and e.motivo_egreso = 'Traslado a hospitalización.';

-- ============================================================
-- casos escritos a mano: accidentes con varias victimas el mismo dia
-- ============================================================

-- hospital 1, turno nocturno del 13 de junio: accidente de bus, 2 fallecen el 14
insert into ingreso (id_ingreso, id_paciente, id_unidad, id_servicio, id_medico_encargado, id_camilla, id_turno,
                     fecha_hora_ingreso, motivo_ingreso, diagnostico_presuntivo, prioridad_triage, turnos_espera) values
(1151, 20, 12, 3, 1001, 25, 1531, '2026-06-13 21:30-06', 'Politraumatismo por accidente de bus.', 'Trauma craneoencefálico severo.
', 1, 0),
(1152, 33, 12, 3, 1005, 26, 1531, '2026-06-13 21:40-06', 'Politraumatismo por accidente de bus.', 'Trauma torácico severo.', 1,
0),
(1153, 46, 12, 3, 1001, 27, 1531, '2026-06-13 21:50-06', 'Politraumatismo por accidente de bus.', 'Fractura de fémur.', 2, 0);
-- hospital 2, turno diurno del 21 de marzo: intoxicacion, 2 fallecen ese dia
insert into ingreso (id_ingreso, id_paciente, id_unidad, id_servicio, id_medico_encargado, id_camilla, id_turno,
                     fecha_hora_ingreso, motivo_ingreso, diagnostico_presuntivo, prioridad_triage, turnos_espera) values
(1154, 41, 22, 104, (select min(id_medico) from turno_medico where id_turno = 1252), 113, 1252, '2026-03-21 09:15-06',
'Intoxicación por plaguicida.', 'Intoxicación por organofosforados.', 1, 0),
(1155, 54, 22, 104, (select min(id_medico) from turno_medico where id_turno = 1252), 114, 1252, '2026-03-21 09:20-06',
'Intoxicación por plaguicida.', 'Intoxicación por organofosforados.', 1, 1);

insert into egreso (id_egreso, id_ingreso, id_medico, id_diagnostico_principal, fecha_hora_egreso, motivo_egreso, codigo_egreso)
select id_ingreso, id_ingreso, id_medico_encargado, 3,
       case id_ingreso when 1151 then timestamptz '2026-06-14 02:10-06' when 1152 then timestamptz '2026-06-14 05:45-06'
                       when 1153 then timestamptz '2026-06-15 10:00-06' when 1154 then timestamptz '2026-03-21 14:30-06'
                       else timestamptz '2026-03-21 18:05-06' end,
       case when id_ingreso = 1153 then 'Mejoría clínica.' else 'Fallecimiento.' end,
       case when id_ingreso = 1153 then 'vivo' else 'muerto' end
  from ingreso where id_ingreso between 1151 and 1155;

-- ============================================================
-- hospitalizacion (60 internamientos)
-- ============================================================

insert into ingreso (id_ingreso, id_paciente, id_unidad, id_servicio, id_medico_encargado, id_camilla,
                     fecha_hora_ingreso, motivo_ingreso, diagnostico_presuntivo)
select 2000 + g.n, 1 + (g.n * 13 + 5) % 67, 10 * g.h + 4, (g.h - 1) * 100 + 8 + g.n % 9,
       (select id_medico from medico where id_hospital = g.h and tipo_medico <> 'externo' order by id_medico
         offset g.n % (select count(*) from medico where id_hospital = g.h and tipo_medico <> 'externo') limit 1),
       (g.h - 1) * 100 + 1 + (g.n / 5) % (array[24, 12, 10, 14, 10])[g.h],
       ((date '2026-01-08' + (g.n * 5) % 280) + time '10:00' + (g.n % 8) * interval '1 hour') at time zone 'America/Guatemala',
       (array['Neumonía que requiere oxígeno.', 'Control de diabetes descompensada.', 'Recuperación postquirúrgica.',
              'Observación por trauma craneal.', 'Tratamiento de infección urinaria complicada.'])[1 + g.n % 5],
       (array['Neumonía.', 'Diabetes descompensada.', 'Postoperatorio.', 'Trauma craneal leve.', 'Pielonefritis.'])[1 + g.n % 5]
  from (select n, 1 + (n * 3) % 5 as h from generate_series(1, 60) n) g;

insert into egreso (id_egreso, id_ingreso, id_medico, id_diagnostico_principal, fecha_hora_egreso, motivo_egreso, codigo_egreso)
select i.id_ingreso, i.id_ingreso, i.id_medico_encargado,
       (array[10, 11, 2, 5, 19])[1 + (i.id_ingreso - 2000) % 5],
       i.fecha_hora_ingreso + (1 + (i.id_ingreso - 2000) % 9) * interval '1 day' + interval '2 hours',
       case when (i.id_ingreso - 2000) % 19 = 0 then 'Fallecimiento.' else 'Alta médica.' end,
       case when (i.id_ingreso - 2000) % 19 = 0 then 'muerto' else 'vivo' end
  from ingreso i
 where i.id_ingreso between 2001 and 2060 and (i.id_ingreso - 2000) % 8 <> 0;

-- insumos usados en cada internamiento, al costo vigente
insert into hospitalizacion_insumo (id_ingreso, id_insumo, cantidad, costo_unitario)
select i.id_ingreso, s.id_insumo, s.cantidad, ins.costo_unitario
  from ingreso i
 cross join lateral (values (1 + i.id_ingreso % 10, 2 + i.id_ingreso % 8),
                            (1 + (i.id_ingreso + 4) % 10, 1 + i.id_ingreso % 5)) as s(id_insumo, cantidad)
  join insumo ins on ins.id_insumo = s.id_insumo
 where i.id_ingreso between 2001 and 2060;

-- diagnosticos secundarios en 1 de cada 5 egresos de emergencias y hospitalizacion
```

```
insert into egreso_diagnostico (id_egreso, id_diagnostico)
select e.id_egreso, 1 + (e.id_egreso * 3) % 19
  from egreso e
 where e.id_egreso between 1001 and 2060 and e.id_egreso % 5 = 0
   and 1 + (e.id_egreso * 3) % 19 <> e.id_diagnostico_principal;

reset role;
```

sql/dml/05_cirugia.sql

```
-- Proyecto 2 - BD Hospitales de Occidente
-- 05_cirugia.sql : agendamientos, preoperatorio, cirugias y su registro de enero a octubre de 2026 (datos FICTICIOS)
--
-- Rangos de id: agendamientos 101-145; la cirugia usa el mismo id que su
-- agendamiento; su ingreso y traslado, 3000 + n.

set role admin;

-- procedimientos de las especialidades quirurgicas que no tenian ninguno
insert into tipo_cirugia (id_tipo_cirugia, nombre, id_especialidad) values
(11, 'Reparación de tendones flexores de la mano', 319),
(12, 'Craneotomía para drenaje de hematoma', 322),
(13, 'Amigdalectomía', 325),
(14, 'Herniorrafia umbilical pediátrica', 326),
(15, 'Rinoplastia reconstructiva', 327),
(16, 'Lobectomía pulmonar', 328),
(17, 'Prostatectomía', 329);

-- ============================================================
-- agendamientos: 45 solicitudes; 1 de cada 6 se rechaza y algunas quedan pendientes
-- ============================================================
-- algunos cirujanos aparecen varias veces en la lista para que tengan mas solicitudes

insert into agendamiento_quirurgico (id_agendamiento, id_paciente, id_medico_cirujano, id_tipo_cirugia, id_tipo_anestesia,
                                     caracter, resumen_clinico, tiempo_estimado_min, fecha_hora_solicitud,
                                     estado_solicitud, motivo_rechazo, id_quirofano_solicitado, fecha_hora_agendada)
select 100 + g.n, 1 + (g.n * 17) % 67, g.cirujano,
       (select id_tipo_cirugia from tipo_cirugia t where t.id_especialidad = m.id_especialidad order by id_tipo_cirugia
         offset g.n % (select count(*) from tipo_cirugia x where x.id_especialidad = m.id_especialidad) limit 1),
       (array[1, 1, 1, 2, 3, 4, 5])[1 + g.n % 7],
       case when g.n % 4 = 0 then 'urgente' else 'programado' end,
       'Paciente valorado en consulta; se indica tratamiento quirúrgico.',
       60 + (g.n * 13) % 150,
       g.solicitud,
       g.estado,
       case when g.estado = 'rechazada' then 'El comité solicita estudios complementarios antes de programar la intervención.'
end,
       4 * (m.id_hospital - 1) + 1 + g.n % 4,
       case when g.estado = 'aprobada' then g.solicitud + (1 + g.n % 4) * interval '1 day' + interval '7 hours' end
  from (select n,
               (array[1006, 1006, 1045, 1009, 1050, 1035, 1045, 1020, 1007, 1009, 1023, 1050, 1006, 1026, 1021,
                      1029, 1022, 1040, 1024, 1025, 1027, 1028, 1030, 1045, 1006])[1 + (n * 7) % 25] as cirujano,
               ((date '2026-01-12' + (n * 6) % 270) + time '08:00') at time zone 'America/Guatemala' as solicitud,
               case when n % 6 = 0 then 'rechazada' when n % 9 = 0 then 'pendiente' else 'aprobada' end as estado
          from generate_series(1, 45) n) g
  join medico m on m.id_medico = g.cirujano;

-- recursos solicitados: cada cirujano tiene su propia costumbre (factor 1 a 4)
insert into requerimiento_insumo (id_agendamiento, id_insumo, cantidad)
select a.id_agendamiento, r.id_insumo, r.cantidad
  from agendamiento_quirurgico a
 cross join lateral (values (1 + a.id_agendamiento % 10, (1 + a.id_medico_cirujano % 4) * 2 + a.id_agendamiento % 3),
                            (1 + (a.id_agendamiento + 5) % 10, 1 + a.id_medico_cirujano % 4)) r(id_insumo, cantidad)
 where a.id_agendamiento > 100;

insert into requerimiento_instrumento (id_agendamiento, id_instrumento, cantidad)
select a.id_agendamiento, r.id_instrumento, r.cantidad
  from agendamiento_quirurgico a
 cross join lateral (values (1 + a.id_agendamiento % 9, 1 + a.id_medico_cirujano % 4),
                            (1 + (a.id_agendamiento + 3) % 9, 1 + a.id_agendamiento % 2)) r(id_instrumento, cantidad)
 where a.id_agendamiento > 100;

insert into requerimiento_equipo (id_agendamiento, id_equipo, cantidad)
select a.id_agendamiento, 1 + a.id_agendamiento % 8, case when a.id_medico_cirujano % 4 > 1 then 2 else 1 end
  from agendamiento_quirurgico a
 where a.id_agendamiento > 100;

-- ============================================================
-- preoperatorio
-- ============================================================

insert into historia_clinica (id_historia_clinica, id_agendamiento, id_medico, fecha, fuente_interrogatorio,
                              antecedentes_heredofamiliares, antecedentes_personales_no_patologicos,
                              antecedentes_patologicos, padecimiento_actual, signos_vitales, exploracion_general)
```

```
select a.id_agendamiento, a.id_agendamiento, a.id_medico_cirujano, a.fecha_hora_solicitud::date,
       case when a.id_agendamiento % 5 = 0 then 'indirecto' else 'directo' end,
       'Sin antecedentes heredofamiliares de importancia.', 'No fuma; alimentación balanceada.',
       case when a.id_agendamiento % 3 = 0 then 'Hipertensión arterial controlada.' else 'Ninguno conocido.' end,
       'Padecimiento que motiva la cirugía, según valoración del cirujano.',
       'TA 120/80, FC 78, T° 36.8.', 'Paciente consciente y orientado.'
  from agendamiento_quirurgico a
 where a.id_agendamiento > 100;

insert into consentimiento_informado (id_consentimiento, id_agendamiento, id_medico, procedimiento_nombre, procedimiento_objetivo,
                                      procedimiento_caracteristicas, procedimiento_riesgos, medico_firmo,
                                      firmante_nombre, firmante_relacion, firmante_firmo, fecha_obtencion)
select a.id_agendamiento, a.id_agendamiento, a.id_medico_cirujano, t.nombre,
       'Tratar quirúrgicamente el padecimiento del paciente.', 'Procedimiento bajo la anestesia indicada en el agendamiento.',
       'Sangrado, infección y complicaciones anestésicas.', true,
       p.nombres || ' ' || p.apellidos,
       case when extract(year from age(p.fecha_nacimiento)) < 18 then 'tutor' else 'paciente' end,
       true, a.fecha_hora_solicitud::date
  from agendamiento_quirurgico a
  join tipo_cirugia t on t.id_tipo_cirugia = a.id_tipo_cirugia
  join paciente p on p.id_paciente = a.id_paciente
 where a.id_agendamiento > 100 and a.estado_solicitud = 'aprobada';

-- el anestesista es el anestesiologo del hospital cuando lo hay; si no, el mismo cirujano
insert into chequeo_preanestesico (id_chequeo, id_agendamiento, clasificacion_asa, id_medico_clasifica, plan_anestesia,
id_medico_anestesista)
select a.id_agendamiento, a.id_agendamiento, (array['I','II','II','III'])[1 + a.id_agendamiento % 4],
       a.id_medico_cirujano, 'Plan anestésico acorde al tipo de anestesia solicitado.',
       coalesce((select an.id_medico from medico an where an.id_especialidad = 318 and an.id_hospital = m.id_hospital limit 1),
                a.id_medico_cirujano)
  from agendamiento_quirurgico a
  join medico m on m.id_medico = a.id_medico_cirujano
 where a.id_agendamiento > 100 and a.estado_solicitud = 'aprobada';

-- ============================================================
-- cirugias realizadas: las aprobadas con fecha antes del 18 de octubre
-- ============================================================

-- ingreso del paciente al quirofano (ficha de ingreso de la unidad de cirugia)
insert into ingreso (id_ingreso, id_paciente, id_unidad, id_servicio, id_medico_encargado, id_quirofano,
                     fecha_hora_ingreso, motivo_ingreso, diagnostico_presuntivo)
select 3000 + (a.id_agendamiento - 100), a.id_paciente, 10 * m.id_hospital + 3,
       (m.id_hospital - 1) * 100 + case e.id_especialidad when 316 then 18 when 317 then 19 else 17 end,
       a.id_medico_cirujano, a.id_quirofano_solicitado,
       a.fecha_hora_agendada - interval '20 minutes',
       'Ingreso a quirófano: ' || t.nombre || '.', 'Indicación quirúrgica.'
  from agendamiento_quirurgico a
  join medico m on m.id_medico = a.id_medico_cirujano
  join tipo_cirugia t on t.id_tipo_cirugia = a.id_tipo_cirugia
  join especialidad e on e.id_especialidad = t.id_especialidad
 where a.id_agendamiento > 100 and a.estado_solicitud = 'aprobada'
   and a.fecha_hora_agendada < timestamptz '2026-10-18 00:00-06';

insert into cirugia (id_cirugia, id_agendamiento, id_quirofano, id_ingreso, fecha_hora_inicio, fecha_hora_fin, estado,
condicion_evolucion)
select a.id_agendamiento, a.id_agendamiento, a.id_quirofano_solicitado, i.id_ingreso,
       a.fecha_hora_agendada,
       a.fecha_hora_agendada + (a.tiempo_estimado_min + (a.id_agendamiento % 5) * 10 - 20) * interval '1 minute',
       'finalizada',
       case when a.id_agendamiento % 11 = 0 then 'complicaciones_graves'
            when a.id_agendamiento % 7 = 0 then 'complicaciones'
            else 'sin_complicaciones' end
  from agendamiento_quirurgico a
  join ingreso i on i.id_ingreso = 3000 + (a.id_agendamiento - 100);

-- equipo quirurgico: el cirujano y, si el hospital tiene, el anestesiologo
insert into cirugia_personal (id_cirugia, id_medico, id_enfermero, id_rol)
select c.id_cirugia, a.id_medico_cirujano, null::integer, 1
  from cirugia c join agendamiento_quirurgico a using (id_agendamiento) where c.id_cirugia > 100
union all
select c.id_cirugia, ch.id_medico_anestesista, null, 3
  from cirugia c join chequeo_preanestesico ch using (id_agendamiento)
 where c.id_cirugia > 100 and ch.id_medico_anestesista <> ch.id_medico_clasifica;

-- y dos enfermeros (instrumentista y circulante), rotando entre los de su hospital
insert into cirugia_personal (id_cirugia, id_medico, id_enfermero, id_rol)
select c.id_cirugia, null, x.id_enfermero, x.rol
  from cirugia c
  join quirofano q on q.id_quirofano = c.id_quirofano
  join unidad_medica u on u.id_unidad = q.id_unidad
 cross join lateral (
        select id_enfermero, 4 + row_number() over (order by id_enfermero) - 1 as rol
          from (select id_enfermero from enfermero where id_hospital = u.id_hospital order by id_enfermero
                offset c.id_cirugia % (select count(*) - 1 from enfermero where id_hospital = u.id_hospital) limit 2) e2) x
```

```
 where c.id_cirugia > 100;

-- resultado de los 38 aspectos de cada cirugia; algunas fallas y resultados no aceptables
insert into registro_procedimiento (id_cirugia, id_aspecto, resultado, descripcion, fecha_hora_registro, id_enfermero)
select c.id_cirugia, ap.id_aspecto,
       case when ap.tipo_evaluacion = 'exito_falla'
            then case when (c.id_cirugia * ap.id_aspecto) % 29 = 0 then 'falla' else 'exito' end
            else case (c.id_cirugia + ap.id_aspecto) % 10 when 0 then 'no_aceptable'
                                                          when 1 then 'medianamente_aceptable'
                                                          when 2 then 'medianamente_aceptable'
                                                          else 'aceptable' end end,
       null,
       case ap.etapa when 'preoperatorio' then c.fecha_hora_inicio - interval '40 minutes'
                     when 'intraoperatorio' then c.fecha_hora_inicio
                     else c.fecha_hora_fin end + ap.id_aspecto * interval '1 minute',
       (select min(id_enfermero) from cirugia_personal cp where cp.id_cirugia = c.id_cirugia)
  from cirugia c
 cross join aspecto_quirurgico ap
 where c.id_cirugia > 100;

insert into documentacion_etapa (id_cirugia, etapa, id_enfermero, fecha_envio)
select c.id_cirugia, et.etapa, (select min(id_enfermero) from cirugia_personal cp where cp.id_cirugia = c.id_cirugia),
       c.fecha_hora_fin::date + 1
  from cirugia c
 cross join (values ('preoperatorio'), ('intraoperatorio'), ('postoperatorio')) et(etapa)
 where c.id_cirugia > 100;

-- insumos usados realmente en la cirugia, al costo vigente
insert into cirugia_insumo (id_cirugia, id_insumo, cantidad, costo_unitario)
select c.id_cirugia, s.id_insumo, s.cantidad, ins.costo_unitario
  from cirugia c
 cross join lateral (values (1 + c.id_cirugia % 10, 2 + c.id_cirugia % 6),
                            (1 + (c.id_cirugia + 2) % 10, 1 + c.id_cirugia % 3),
                            (1 + (c.id_cirugia + 6) % 10, 3)) s(id_insumo, cantidad)
  join insumo ins on ins.id_insumo = s.id_insumo
 where c.id_cirugia > 100;

-- egreso de la unidad de cirugia y traslado a hospitalizacion para la recuperacion
insert into egreso (id_egreso, id_ingreso, id_medico, id_diagnostico_principal, fecha_hora_egreso, motivo_egreso, codigo_egreso)
select c.id_ingreso, c.id_ingreso, a.id_medico_cirujano, 2,
       c.fecha_hora_fin + interval '30 minutes',
       case when c.condicion_evolucion = 'complicaciones_graves' and c.id_cirugia % 2 = 0 then 'Fallecimiento en sala de
operaciones.'
            else 'Traslado a sala de recuperación.' end,
       case when c.condicion_evolucion = 'complicaciones_graves' and c.id_cirugia % 2 = 0 then 'muerto' else 'vivo' end
  from cirugia c join agendamiento_quirurgico a using (id_agendamiento)
 where c.id_cirugia > 100;

insert into traslado (id_traslado, fecha_hora, id_medico_indica, id_ingreso, id_unidad_origen, tipo_traslado, id_unidad_destino)
select e.id_egreso, e.fecha_hora_egreso - interval '10 minutes', e.id_medico, e.id_ingreso, i.id_unidad, 'interno', i.id_unidad +
1
  from egreso e join ingreso i using (id_ingreso)
 where e.id_egreso > 3000 and e.codigo_egreso = 'vivo';

reset role;
```

sql/dml/06_facturacion.sql

```
-- Proyecto 2 - BD Hospitales de Occidente
-- 06_facturacion.sql : facturas y pagos en cuotas de la actividad de enero a octubre de 2026 (datos FICTICIOS)
--
-- Se emite una factura por cada proceso terminado, con el costo que fija el
-- enunciado para cada unidad, y se pagan las cuotas vencidas al 20 de octubre.

set role admin;

-- consulta externa: el costo de la cita, en un solo pago en la recepcion del medico
insert into factura (id_cita, fecha_emision, descripcion, total, numero_cuotas)
select ci.id_cita, ci.fecha_hora + interval '1 hour', 'Consulta externa de ' || lower(es.nombre) || '.', ci.costo_final, 1
  from consulta co
  join cita ci on ci.id_cita = co.id_cita
  join medico me on me.id_medico = ci.id_medico
  join especialidad es on es.id_especialidad = me.id_especialidad
 where ci.id_cita > 100;

-- emergencias: el costo del servicio brindado, de 1 a 4 cuotas
insert into factura (id_ingreso, fecha_emision, descripcion, total, numero_cuotas)
select i.id_ingreso, e.fecha_hora_egreso + interval '30 minutes', 'Atención de emergencia: ' || lower(s.nombre) || '.',
       s.costo, 1 + i.id_ingreso % 4
  from ingreso i
  join egreso e on e.id_ingreso = i.id_ingreso
  join servicio_atencion s on s.id_servicio = i.id_servicio
 where i.id_ingreso between 1001 and 1155;
-- hospitalizacion: dias internado x costo por dia + servicio + insumos usados, de 1 a 12 cuotas
```

```
insert into factura (id_ingreso, fecha_emision, descripcion, total, numero_cuotas)
select i.id_ingreso, e.fecha_hora_egreso + interval '1 hour',
       format('Hospitalización %s días: %s + insumos.', greatest(1, e.fecha_hora_egreso::date - i.fecha_hora_ingreso::date),
lower(s.nombre)),
       greatest(1, e.fecha_hora_egreso::date - i.fecha_hora_ingreso::date) * u.costo_dia_hospitalizacion + s.costo
         + coalesce((select sum(hi.cantidad * hi.costo_unitario) from hospitalizacion_insumo hi where hi.id_ingreso = i.
id_ingreso), 0),
       1 + i.id_ingreso % 12
  from ingreso i
  join egreso e on e.id_ingreso = i.id_ingreso
  join unidad_medica u on u.id_unidad = i.id_unidad
  join servicio_atencion s on s.id_servicio = i.id_servicio
 where i.id_ingreso between 2001 and 2060;

-- cirugia: costo segun el tipo de cirugia + insumos usados, de 1 a 12 cuotas
insert into factura (id_cirugia, fecha_emision, descripcion, total, numero_cuotas)
select c.id_cirugia, c.fecha_hora_fin + interval '2 hours', 'Cirugía: ' || lower(t.nombre) || ' + insumos utilizados.',
       s.costo + coalesce((select sum(ci.cantidad * ci.costo_unitario) from cirugia_insumo ci where ci.id_cirugia = c.id_cirugia),
 0),
       1 + c.id_cirugia % 12
  from cirugia c
  join agendamiento_quirurgico a on a.id_agendamiento = c.id_agendamiento
  join tipo_cirugia t on t.id_tipo_cirugia = a.id_tipo_cirugia
  join ingreso i on i.id_ingreso = c.id_ingreso
  join servicio_atencion s on s.id_servicio = i.id_servicio
 where c.id_cirugia > 100;

-- ============================================================
-- pagos: la primera cuota al emitir la factura y una por cada mes transcurrido
-- ============================================================
-- la ultima cuota ajusta los centavos para que la suma sea exactamente el total

insert into pago (id_factura, numero_cuota, monto, fecha_pago, id_metodo_pago)
select f.id_factura, k.cuota,
       case when k.cuota = f.numero_cuotas
            then f.total - round(f.total / f.numero_cuotas, 2) * (f.numero_cuotas - 1)
            else round(f.total / f.numero_cuotas, 2) end,
       (f.fecha_emision::date + (k.cuota - 1) * interval '1 month')::date,
       case when (f.id_factura * k.cuota) % 11 = 0 then 4 else 1 + (f.id_factura + k.cuota) % 3 end
  from factura f
 cross join lateral generate_series(1, least(f.numero_cuotas,
        1 + (extract(year from age(date '2026-10-20', f.fecha_emision::date)) * 12
             + extract(month from age(date '2026-10-20', f.fecha_emision::date)))::int)) as k(cuota)
 where f.id_factura > 5;

reset role;
```

sql/dml/07_laboratorio_y_clinicas.sql

```
-- Proyecto 2 - BD Hospitales de Occidente
-- 07_laboratorio_y_clinicas.sql : examenes de laboratorio y ordenes de las consultas (datos FICTICIOS)
-- (las clinicas se crean en ddl/04 a partir de las citas)

set role admin;

insert into examen_laboratorio (id_examen, nombre, descripcion, costo) values
(1, 'Hematología completa', 'Conteo de glóbulos rojos, blancos y plaquetas', 75.00),
(2, 'Glucosa en ayunas', 'Nivel de azúcar en sangre', 35.00),
(3, 'Hemoglobina glicosilada', 'Control de diabetes de los últimos 3 meses', 120.00),
(4, 'Perfil de lípidos', 'Colesterol total, HDL, LDL y triglicéridos', 110.00),
(5, 'Creatinina', 'Función de los riñones', 40.00),
(6, 'Ácido úrico', 'Nivel de ácido úrico en sangre', 40.00),
(7, 'Examen general de orina', 'Análisis físico, químico y microscópico de la orina', 45.00),
(8, 'Examen de heces', 'Búsqueda de parásitos y sangre oculta', 45.00),
(9, 'TSH', 'Función de la tiroides', 130.00),
(10, 'Pruebas hepáticas', 'Función del hígado', 150.00),
(11, 'Electrolitos', 'Sodio, potasio y cloro', 95.00),
(12, 'Proteína C reactiva', 'Indicador de inflamación', 80.00),
(13, 'Prueba de embarazo', 'Hormona hCG en sangre', 60.00),
(14, 'Rayos X de tórax', 'Radiografía de pulmones y corazón', 175.00),
(15, 'Ultrasonido abdominal', 'Imagen de los órganos del abdomen', 300.00),
(16, 'Electrocardiograma', 'Actividad eléctrica del corazón', 150.00);

-- una de cada tres consultas termina con orden de laboratorio
insert into orden_laboratorio (id_orden, id_consulta, fecha, indicaciones)
select co.id_consulta, co.id_consulta, c.fecha_hora::date,
       case when co.id_consulta % 2 = 0 then 'Presentarse en ayunas de 8 horas.' else 'Traer resultados a la próxima cita.' end
from consulta co
inner join cita c on c.id_cita = co.id_cita
where co.id_consulta % 3 = 0;

-- cada orden pide uno o dos examenes
insert into orden_examen (id_orden, id_examen)
select id_orden, 1 + id_orden % 16 from orden_laboratorio
```

```
union
select id_orden, 1 + (id_orden + 5) % 16 from orden_laboratorio where id_orden % 2 = 0;

select setval(pg_get_serial_sequence('examen_laboratorio', 'id_examen'), (select max(id_examen) from examen_laboratorio));
select setval(pg_get_serial_sequence('orden_laboratorio', 'id_orden'), (select max(id_orden) from orden_laboratorio));

reset role;
```

sql/auditoria/01_auditoria.sql

```
-- Proyecto 2 - BD Hospitales de Occidente
-- 01_auditoria.sql : bitacora automatica de cambios sensibles del personal (Bloque F)
--
-- Cada vez que cambia el salario, el nivel de acceso o la especialidad de alguien,
-- un disparador lo anota solo en la bitacora: quien lo cambio, que campo, el valor
-- de antes, el valor nuevo y la hora. Asi no depende de que alguien se acuerde.
--
-- Lo crea el rol administrativo, como pide el enunciado.

set role admin;

-- la bitacora: una fila por cada campo que cambia
create table auditoria_personal (
    id_auditoria integer generated always as identity primary key,
    fecha_hora timestamptz not null default now(),   -- momento exacto del cambio
    usuario varchar(63) not null,                    -- quien hizo el cambio
    tabla varchar(30) not null,                      -- personal o medico
    id_registro integer not null,                    -- a quien se le cambio el dato
    campo varchar(30) not null,                      -- que campo cambio
    valor_anterior text,                             -- como estaba antes
    valor_nuevo text                                 -- como quedo
);

-- cuando cambia el salario o el nivel de acceso de alguien del personal
-- old = la fila antes del update, new = la fila despues
create function fn_auditar_personal() returns trigger language plpgsql as $$
begin
    -- is distinct from es como <> pero tambien sirve cuando el valor es null
    if new.salario is distinct from old.salario then
        insert into auditoria_personal (usuario, tabla, id_registro, campo, valor_anterior, valor_nuevo)
        values (current_user, 'personal', new.id_personal, 'salario', old.salario, new.salario);
    end if;
    if new.nivel_acceso is distinct from old.nivel_acceso then
        insert into auditoria_personal (usuario, tabla, id_registro, campo, valor_anterior, valor_nuevo)
        values (current_user, 'personal', new.id_personal, 'nivel_acceso', old.nivel_acceso, new.nivel_acceso);
    end if;
    return new;
end $$;

-- despues de cada update en personal se ejecuta la funcion de arriba, fila por fila
create trigger trg_auditar_personal after update on personal
    for each row execute function fn_auditar_personal();

-- cuando cambia la especialidad de un medico
create function fn_auditar_medico() returns trigger language plpgsql as $$
begin
    if new.id_especialidad is distinct from old.id_especialidad then
        insert into auditoria_personal (usuario, tabla, id_registro, campo, valor_anterior, valor_nuevo)
        values (current_user, 'medico', new.id_medico, 'id_especialidad', old.id_especialidad, new.id_especialidad);
    end if;
    return new;
end $$;

-- despues de cada update en medico se ejecuta la funcion de arriba, fila por fila
create trigger trg_auditar_medico after update on medico
    for each row execute function fn_auditar_medico();

reset role;
```

sql/auditoria/02_prueba.sql

```
-- Proyecto 2 - BD Hospitales de Occidente
-- 02_prueba.sql : prueba de la bitacora de cambios sensibles (objetivo 20)
--
-- Se hacen 3 cambios con el rol administrativo y al final se ve lo que quedo anotado.

set role admin;

-- cambio 1: aumento de salario a la doctora Carmen Ruiz (1001)
update personal set salario = salario + 1500.00 where id_personal = 1001;

-- cambio 2: el doctor Andrés Chinchilla (1015) pasa de Odontología General a Medicina General
update medico set id_especialidad = 307 where id_medico = 1015;

-- cambio 3: en un mismo update cambian salario y nivel de acceso, se anotan las dos cosas
```

```
update personal set salario = 22000.00, nivel_acceso = 'supervisor' where id_personal = 1004;

-- este update deja el mismo valor, asi que no se anota nada
update personal set salario = salario where id_personal = 1002;

-- lo que quedo en la bitacora
select id_auditoria, fecha_hora, usuario, tabla, id_registro, campo, valor_anterior, valor_nuevo
from auditoria_personal
order by id_auditoria;

reset role;
```

sql/consultas/objetivo_01.sql

```
-- Objetivo 1: los 3 hospitales con mas demanda en consulta externa
-- demanda = citas no canceladas (reservaciones) + consultas realizadas + proximas citas de las recetas
-- lapso: enero a octubre de 2026



select h.nombre as hospital,
       count(case when c.estado <> 'cancelada' then 1 end) as reservaciones,
       count(co.id_consulta) as consultas,
       count(r.fecha_proxima_cita) as proximas_consultas,
       count(case when c.estado <> 'cancelada' then 1 end) + count(co.id_consulta) + count(r.fecha_proxima_cita) as demanda_total
from cita c
inner join medico m on m.id_medico = c.id_medico   --a cada cita se busca su medico
inner join hospital h on h.id_hospital = m.id_hospital --a cada medico se busca su hospital
left join consulta co on co.id_cita = c.id_cita  --aqui  se busca a la cita su consulta, solo si la tuvo
left join receta r on r.id_consulta = co.id_consulta --a cada consulta se busca su receta, solo si la tuvo
where c.fecha_hora between '2026-01-01' and '2026-10-31 23:59:59' --aqui se filtra por el lapso de tiempo
group by h.nombre --aqui agurpo por nombre de hospital
order by demanda_total desc -- ordeno de manera descendente por demanda total
limit 3; --y solo muestro los 3 primeros hospitales con mas demanda
```

sql/consultas/objetivo_02.sql

```
-- Objetivo 2: los 5 medicos de consulta externa con menor calificacion
-- se usa la calificacion (1 a 5) que el paciente le dio al medico en cada consulta
-- lapso: enero a octubre de 2026

select m.nombres, m.apellidos,
       e.nombre as especialidad,
       h.nombre as hospital,
       round(avg(ca.puntuacion), 2) as calificacion_promedio,
       count(co.id_consulta) as consultas
from consulta co
inner join cita c on c.id_cita = co.id_cita
inner join medico m on m.id_medico = c.id_medico
inner join especialidad e on e.id_especialidad = m.id_especialidad
inner join hospital h on h.id_hospital = m.id_hospital
left join calificacion ca on ca.id_consulta = co.id_consulta
where c.fecha_hora between '2026-01-01' and '2026-10-31 23:59:59'
group by m.id_medico, m.nombres, m.apellidos, e.nombre, h.nombre
having count(ca.id_calificacion) > 0   --aqui solo se consideran los medicos que tengan al menos una calificacion
order by calificacion_promedio, consultas desc, m.apellidos
limit 5;
```

sql/consultas/objetivo_03.sql

```
-- Objetivo 3: los 5 mejores medicos de emergencias (turnos de 12 horas)
-- lapso: enero a octubre de 2026

select m.nombres, m.apellidos,
       h.nombre as hospital,
       count(*) as turnos,
       count(*) * 12 as horas_laboradas, -- 12 horas por turno
       (select count(*)
        from egreso e
        inner join ingreso i on i.id_ingreso = e.id_ingreso
        where e.id_medico = m.id_medico
          and i.id_turno is not null      -- solo los ingresos de emergencias tienen turno
          and e.codigo_egreso = 'vivo'
          and e.sin_consentimiento = false
          and e.fecha_hora_egreso between '2026-01-01' and '2026-10-31 23:59:59') as egresos_vivos
from turno_medico tm
inner join turno_emergencia t on t.id_turno = tm.id_turno
inner join medico m on m.id_medico = tm.id_medico
inner join hospital h on h.id_hospital = m.id_hospital
where t.fecha between '2026-01-01' and '2026-10-31'
group by m.id_medico, m.nombres, m.apellidos, h.nombre
order by egresos_vivos desc, turnos desc
  limit 5;
```

sql/consultas/objetivo_04.sql

```
-- Objetivo 4: los 3 hospitales mas eficientes en emergencias
-- eficiente = tiene camilla libre cuando llega el paciente: mayor porcentaje de
-- ingresos atendidos sin esperar turno y menor promedio de turnos de espera
-- lapso: enero a octubre de 2026

select h.nombre as hospital,
       count(*) as ingresos,
       count(e.id_egreso) as egresos,
       count(case when i.turnos_espera = 0 then 1 end) as atendidos_sin_espera,
       round(100.0 * count(case when i.turnos_espera = 0 then 1 end) / count(*), 1) as porcentaje_sin_espera,
       round(avg(i.turnos_espera), 2) as promedio_turnos_espera
from ingreso i
inner join unidad_medica u on u.id_unidad = i.id_unidad
inner join hospital h on h.id_hospital = u.id_hospital
left join egreso e on e.id_ingreso = i.id_ingreso
where u.tipo_unidad = 'Emergencias'
  and i.fecha_hora_ingreso between '2026-01-01' and '2026-10-31 23:59:59'
group by h.nombre
order by porcentaje_sin_espera desc, promedio_turnos_espera, ingresos desc
limit 3;
```

sql/consultas/objetivo_05.sql

```
-- Objetivo 5: los 3 cirujanos mas eficientes
-- primero los que tienen mas agendamientos aprobados y, entre ellos, los que pidieron
-- menos insumos, instrumentos y equipos (cada uno por separado)
-- lapso: enero a octubre de 2026

select m.nombres, m.apellidos,
       count(*) as agendamientos_aprobados,
       sum(coalesce(ri.cantidad, 0)) as insumos,
       sum(coalesce(rn.cantidad, 0)) as instrumentos,
       sum(coalesce(re.cantidad, 0)) as equipos
from agendamiento_quirurgico a
inner join medico m on m.id_medico = a.id_medico_cirujano
left join (select id_agendamiento, sum(cantidad) as cantidad
           from requerimiento_insumo group by id_agendamiento) ri on ri.id_agendamiento = a.id_agendamiento
left join (select id_agendamiento, sum(cantidad) as cantidad
           from requerimiento_instrumento group by id_agendamiento) rn on rn.id_agendamiento = a.id_agendamiento
left join (select id_agendamiento, sum(cantidad) as cantidad
           from requerimiento_equipo group by id_agendamiento) re on re.id_agendamiento = a.id_agendamiento
where a.estado_solicitud = 'aprobada'
  and a.fecha_hora_solicitud between '2026-01-01' and '2026-10-31 23:59:59'
group by m.id_medico, m.nombres, m.apellidos
order by agendamientos_aprobados desc, insumos, instrumentos, equipos
limit 3;
```

sql/consultas/objetivo_06.sql

```
-- Objetivo 6: los 3 tipos de cirugia con mas ingresos
-- solo cirugias "sin complicaciones" en las que todos los aspectos de la fase
-- intraoperatoria salieron en exito o aceptables (ninguna falla ni no aceptable)
-- lapso: enero a octubre de 2026

select tc.nombre as tipo_cirugia,
       e.nombre as especialidad,
       count(*) as cirugias,
       sum(f.total) as ingresos
from cirugia ci
inner join agendamiento_quirurgico a on a.id_agendamiento = ci.id_agendamiento
inner join tipo_cirugia tc on tc.id_tipo_cirugia = a.id_tipo_cirugia
inner join especialidad e on e.id_especialidad = tc.id_especialidad
inner join factura f on f.id_cirugia = ci.id_cirugia
where ci.condicion_evolucion = 'sin_complicaciones'
  and ci.fecha_hora_inicio between '2026-01-01' and '2026-10-31 23:59:59'
  and not exists (select 1
                  from registro_procedimiento rp
                  inner join aspecto_quirurgico aq on aq.id_aspecto = rp.id_aspecto
                  where rp.id_cirugia = ci.id_cirugia
                    and aq.etapa = 'intraoperatorio'
                    and rp.resultado not in ('exito', 'aceptable'))
group by tc.nombre, e.nombre
order by ingresos desc
limit 3;
```

sql/consultas/objetivo_07.sql

```
-- Objetivo 7: los 3 hospitales con mas ingresos en hospitalizacion
-- con menos camillas asignadas y menor costo de servicio. Se juntan los tres
-- criterios en el ingreso por camilla (lo facturado / camillas usadas)
-- lapso: enero a octubre de 2026

select h.nombre as hospital,
```

```
       sum(f.total) as ingresos,
       count(distinct i.id_camilla) as camillas_asignadas,
       round(avg(s.costo), 2) as costo_promedio_servicio,
       round(sum(f.total) / count(distinct i.id_camilla), 2) as ingreso_por_camilla
from factura f
inner join ingreso i on i.id_ingreso = f.id_ingreso
inner join unidad_medica u on u.id_unidad = i.id_unidad
inner join hospital h on h.id_hospital = u.id_hospital
inner join servicio_atencion s on s.id_servicio = i.id_servicio
where u.tipo_unidad = 'Hospitalización'
  and f.fecha_emision between '2026-01-01' and '2026-10-31 23:59:59'
group by h.nombre
order by ingreso_por_camilla desc, costo_promedio_servicio
limit 3;
```

sql/consultas/objetivo_08.sql

```
-- Objetivo 8: total pagado en efectivo, tarjeta de credito y tarjeta de debito
-- en cada hospital, sumando los pagos de cada una de sus unidades.
-- La unidad de cada factura sale de su origen: la clinica de la cita, la unidad
-- del ingreso o el quirofano de la cirugia (cada factura tiene solo uno).
-- lapso: enero a octubre de 2026

select h.nombre as hospital,
       u.tipo_unidad as unidad,
       sum(case when mp.nombre = 'Efectivo' then p.monto else 0 end) as efectivo,
       sum(case when mp.nombre = 'Tarjeta de crédito' then p.monto else 0 end) as tarjeta_credito,
       sum(case when mp.nombre = 'Tarjeta de débito' then p.monto else 0 end) as tarjeta_debito
from pago p
inner join metodo_pago mp on mp.id_metodo_pago = p.id_metodo_pago
inner join factura f on f.id_factura = p.id_factura
left join cita c on c.id_cita = f.id_cita
left join clinica cl on cl.id_clinica = c.id_clinica
left join ingreso i on i.id_ingreso = f.id_ingreso
left join cirugia ci on ci.id_cirugia = f.id_cirugia
left join quirofano q on q.id_quirofano = ci.id_quirofano
inner join unidad_medica u on u.id_unidad in (cl.id_unidad, i.id_unidad, q.id_unidad)
inner join hospital h on h.id_hospital = u.id_hospital
where p.fecha_pago between '2026-01-01' and '2026-10-31'
group by h.id_hospital, h.nombre, u.tipo_unidad
order by h.id_hospital, u.tipo_unidad;
```

sql/consultas/objetivo_09.sql

```
-- Objetivo 9: los 3 hospitales con mas ingreso por visita
-- visita = ficha de ingreso (emergencias, cirugia, hospitalizacion) o cita no cancelada.
-- Las consultas se muestran pero no se suman porque ya estan contadas en su cita.
-- El hospital de cada factura sale de su origen: la clinica de la cita, la unidad
-- del ingreso o el quirofano de la cirugia (cada factura tiene solo uno).
-- lapso: enero a octubre de 2026

select hospital, ingresos_fichas, citas, consultas,
       ingresos_fichas + citas as visitas,
       monto_facturado,
       round(monto_facturado / (ingresos_fichas + citas), 2) as ingreso_por_visita
from (select h.nombre as hospital,
             (select count(*)
              from ingreso i
              inner join unidad_medica u on u.id_unidad = i.id_unidad
              where u.id_hospital = h.id_hospital
                and i.fecha_hora_ingreso between '2026-01-01' and '2026-10-31 23:59:59') as ingresos_fichas,
             (select count(*)
              from cita c
              inner join medico m on m.id_medico = c.id_medico
              where m.id_hospital = h.id_hospital
                and c.estado <> 'cancelada'
                and c.fecha_hora between '2026-01-01' and '2026-10-31 23:59:59') as citas,
             (select count(*)
              from consulta co
              inner join cita c on c.id_cita = co.id_cita
              inner join medico m on m.id_medico = c.id_medico
              where m.id_hospital = h.id_hospital
                and c.fecha_hora between '2026-01-01' and '2026-10-31 23:59:59') as consultas,
             (select coalesce(sum(f.total), 0)
              from factura f
              left join cita c on c.id_cita = f.id_cita
              left join clinica cl on cl.id_clinica = c.id_clinica
              left join ingreso i on i.id_ingreso = f.id_ingreso
              left join cirugia ci on ci.id_cirugia = f.id_cirugia
              left join quirofano q on q.id_quirofano = ci.id_quirofano
              inner join unidad_medica u on u.id_unidad in (cl.id_unidad, i.id_unidad, q.id_unidad)
              where u.id_hospital = h.id_hospital
                and f.fecha_emision between '2026-01-01' and '2026-10-31 23:59:59') as monto_facturado
      from hospital h) datos
order by ingreso_por_visita desc
```

```
limit 3;
```

sql/consultas/objetivo_10.sql

```
-- Objetivo 10: por hospital, la fecha con mas decesos (egresos con codigo "muerto")
-- y la unidad medica donde se han registrado mas.
-- Se resuelve con dos consultas. En cada una se cuentan los decesos por grupo y el
-- HAVING se queda con el grupo que tiene el maximo de su hospital (si hay empate,
-- salen todos los empatados). No lleva lapso: el objetivo pide la fecha historica.

-- fecha con mas decesos de cada hospital
select h.nombre as hospital,
       e.fecha_hora_egreso::date as fecha,
       count(*) as decesos
from egreso e
inner join ingreso i on i.id_ingreso = e.id_ingreso
inner join unidad_medica u on u.id_unidad = i.id_unidad
inner join hospital h on h.id_hospital = u.id_hospital
where e.codigo_egreso = 'muerto'
group by h.id_hospital, h.nombre, e.fecha_hora_egreso::date
having count(*) = (select max(total)
                   from (select count(*) as total
                         from egreso e2
                         inner join ingreso i2 on i2.id_ingreso = e2.id_ingreso
                         inner join unidad_medica u2 on u2.id_unidad = i2.id_unidad
                         where e2.codigo_egreso = 'muerto'
                           and u2.id_hospital = h.id_hospital
                         group by e2.fecha_hora_egreso::date) por_fecha)
order by decesos desc, hospital, fecha;

-- unidad medica con mas decesos de cada hospital
select h.nombre as hospital,
       u.tipo_unidad as unidad,
       count(*) as decesos
from egreso e
inner join ingreso i on i.id_ingreso = e.id_ingreso
inner join unidad_medica u on u.id_unidad = i.id_unidad
inner join hospital h on h.id_hospital = u.id_hospital
where e.codigo_egreso = 'muerto'
group by h.id_hospital, h.nombre, u.tipo_unidad
having count(*) = (select max(total)
                   from (select count(*) as total
                         from egreso e2
                         inner join ingreso i2 on i2.id_ingreso = e2.id_ingreso
                         inner join unidad_medica u2 on u2.id_unidad = i2.id_unidad
                         where e2.codigo_egreso = 'muerto'
                           and u2.id_hospital = h.id_hospital
                         group by u2.tipo_unidad) por_unidad)
order by decesos desc, hospital, unidad;
```

sql/consultas/objetivo_11.sql

```
-- Objetivo 11: ingresos monetarios totales de cada hospital
-- lo facturado (servicios prestados), lo cobrado (cuotas pagadas) y lo pendiente.
-- El hospital de cada factura sale de su origen: la clinica de la cita, la unidad
-- del ingreso o el quirofano de la cirugia (cada factura tiene solo uno).
-- lapso: enero a octubre de 2026

select hospital, facturas, total_facturado, total_cobrado,
       total_facturado - total_cobrado as pendiente_de_cobro
from (select h.nombre as hospital,
             (select count(*)
              from factura f
              left join cita c on c.id_cita = f.id_cita
              left join clinica cl on cl.id_clinica = c.id_clinica
              left join ingreso i on i.id_ingreso = f.id_ingreso
              left join cirugia ci on ci.id_cirugia = f.id_cirugia
              left join quirofano q on q.id_quirofano = ci.id_quirofano
              inner join unidad_medica u on u.id_unidad in (cl.id_unidad, i.id_unidad, q.id_unidad)
              where u.id_hospital = h.id_hospital
                and f.fecha_emision between '2026-01-01' and '2026-10-31 23:59:59') as facturas,
             (select coalesce(sum(f.total), 0)
              from factura f
              left join cita c on c.id_cita = f.id_cita
              left join clinica cl on cl.id_clinica = c.id_clinica
              left join ingreso i on i.id_ingreso = f.id_ingreso
              left join cirugia ci on ci.id_cirugia = f.id_cirugia
              left join quirofano q on q.id_quirofano = ci.id_quirofano
              inner join unidad_medica u on u.id_unidad in (cl.id_unidad, i.id_unidad, q.id_unidad)
              where u.id_hospital = h.id_hospital
                and f.fecha_emision between '2026-01-01' and '2026-10-31 23:59:59') as total_facturado,
             (select coalesce(sum(p.monto), 0)
              from pago p
              inner join factura f on f.id_factura = p.id_factura
              left join cita c on c.id_cita = f.id_cita
```

```
              left join clinica cl on cl.id_clinica = c.id_clinica
              left join ingreso i on i.id_ingreso = f.id_ingreso
              left join cirugia ci on ci.id_cirugia = f.id_cirugia
              left join quirofano q on q.id_quirofano = ci.id_quirofano
              inner join unidad_medica u on u.id_unidad in (cl.id_unidad, i.id_unidad, q.id_unidad)
              where u.id_hospital = h.id_hospital
                and p.fecha_pago between '2026-01-01' and '2026-10-31') as total_cobrado
      from hospital h) datos
order by total_facturado desc;
```

sql/consultas/objetivo_12.sql

```
-- Objetivo 12: edad promedio y cantidad por sexo de los ingresos a emergencias
-- de cada hospital, agrupados por turno (jornada diurna o nocturna).
-- La edad es la que tenia el paciente el dia que ingreso
-- lapso: enero a octubre de 2026

select h.nombre as hospital,
       t.jornada,
       count(*) as ingresos,
       round(avg(extract(year from age(i.fecha_hora_ingreso::date, p.fecha_nacimiento))), 1) as edad_promedio,
       count(case when p.sexo = 'Masculino' then 1 end) as masculino,
       count(case when p.sexo = 'Femenino' then 1 end) as femenino
from ingreso i
inner join turno_emergencia t on t.id_turno = i.id_turno
inner join unidad_medica u on u.id_unidad = i.id_unidad
inner join hospital h on h.id_hospital = u.id_hospital
inner join paciente p on p.id_paciente = i.id_paciente
where i.fecha_hora_ingreso between '2026-01-01' and '2026-10-31 23:59:59'
group by h.id_hospital, h.nombre, t.jornada
order by h.id_hospital, t.jornada;
```

sql/consultas/objetivo_13.sql

```
-- Objetivo 13: reporte de citas de consulta externa de cada hospital,
-- ordenado por fecha y agrupado por sexo del paciente y especialidad del medico.
-- lapso: enero a octubre de 2026

select h.nombre as hospital,
       c.fecha_hora::date as fecha,
       e.nombre as especialidad,
       p.sexo,
       count(*) as citas
from cita c
inner join paciente p on p.id_paciente = c.id_paciente
inner join medico m on m.id_medico = c.id_medico
inner join especialidad e on e.id_especialidad = m.id_especialidad
inner join hospital h on h.id_hospital = m.id_hospital
where c.fecha_hora between '2026-01-01' and '2026-10-31 23:59:59'
group by h.id_hospital, h.nombre, c.fecha_hora::date, e.nombre, p.sexo
order by h.id_hospital, fecha, especialidad, p.sexo;
```

sql/consultas/objetivo_14.sql

```
-- Objetivo 14: promedio del numero de cuotas de pago por unidad medica en cada hospital
-- (segun el plan de cuotas de cada factura emitida)
-- La unidad de cada factura sale de su origen: la clinica de la cita, la unidad
-- del ingreso o el quirofano de la cirugia (cada factura tiene solo uno).
-- lapso: enero a octubre de 2026

select h.nombre as hospital,
       u.tipo_unidad as unidad,
       count(*) as facturas,
       round(avg(f.numero_cuotas), 2) as promedio_cuotas,
       min(f.numero_cuotas) as minimo_cuotas,
       max(f.numero_cuotas) as maximo_cuotas
from factura f
left join cita c on c.id_cita = f.id_cita
left join clinica cl on cl.id_clinica = c.id_clinica
left join ingreso i on i.id_ingreso = f.id_ingreso
left join cirugia ci on ci.id_cirugia = f.id_cirugia
left join quirofano q on q.id_quirofano = ci.id_quirofano
inner join unidad_medica u on u.id_unidad in (cl.id_unidad, i.id_unidad, q.id_unidad)
inner join hospital h on h.id_hospital = u.id_hospital
where f.fecha_emision between '2026-01-01' and '2026-10-31 23:59:59'
group by h.id_hospital, h.nombre, u.tipo_unidad
order by h.id_hospital, u.tipo_unidad;
```

sql/consultas/objetivo_16.sql

```
-- Objetivo 16: cadena de supervision de un medico hasta el director medico de su hospital,
-- sin saber de antemano cuantos niveles hay entre ellos.
-- Se usa una CTE recursiva: el caso base es el medico indicado (nivel 0) y la parte
-- recursiva sube a su superior directo hasta llegar a quien ya no tiene superior.
-- Para consultar otro medico solo se cambia el id del caso base.
```

```
-- ============================================================
-- ejemplo 1: medico 1018 (cadena de 4 niveles en el hospital 1)
-- ============================================================

with recursive cadena as (                          -- tabla temporal "cadena"; recursive = se usa a si misma para crecer

    -- CASO BASE: el medico que se quiere consultar
    select m.id_medico,                             -- el id del mesico
           m.id_medico_superior,                    -- el id de su jefe directo
           0 as nivel                               -- nivel 0 = el medico consultado
    from medico m                                   -- se buca en la tabla medico
    where m.id_medico = 1018                        -- el medico del que se parte (cambiar este id para otro medico)

    union all                                       -- junta el caso base con las filas que agrega la parte recursiva

    -- PARTE RECURSIVA: se repite sola, cada vuelta sube un nivel
    select m.id_medico,                             -- el id del jefe encontrado
           m.id_medico_superior,                    -- el jefe de ese jefe (para la siguiente vuelta)
           c.nivel + 1                              -- un nivel mas ariba que la fila anterior
    from medico m                                   -- se busca otra vez en medico
    inner join cadena c                             -- se une con lo que ya hay en "cadena"
            on m.id_medico = c.id_medico_superior   -- el medico buscado es el jefe de la ultima fila;
                                                    -- cuando llegs al director (superior null) ya no encuentra a nadie y termina
)
-- con la cadena armada se muestran los datos de cada medico
select c.nivel,                                     -- 0 = el consultado ... el mas alto = el director
       m.id_medico,                                 -- id del medico
       m.nombres,                                   -- nombres
       m.apellidos,                                 -- apellidos
       m.cargo,                                     -- medico, jefe_servicio o director_medico
       e.nombre as especialidad,                    -- nombre de su especialidad
       h.nombre as hospital                         -- nombre de su hospital
from cadena c                                       -- la tabla temporal que armo la parte de arriba
inner join medico m on m.id_medico = c.id_medico    -- se trae el nombre y cargo de cada medico de la cadena
inner join especialidad e on e.id_especialidad = m.id_especialidad   -- su especialidad
inner join hospital h on h.id_hospital = m.id_hospital               -- su hospital
order by c.nivel;                                   -- del medico consultado hasta el director

-- ============================================================
-- ejemplo 2: medico 1046 (hospital 4)
-- es la misma consulta: solo cambia el id del caso base
-- ============================================================

with recursive cadena as (                          -- taba temporl "cadena" que se usa a si misma
    -- caso base: el medico 1046 en el nivel 0
    select m.id_medico, m.id_medico_superior, 0 as nivel
    from medico m
    where m.id_medico = 1046                        -- unico cambio respecto al ejemplo 1
    union all                                       -- junta el caso base con lo que agrega la parte recursiva
    -- parte recursiva: trae al jefe de la ultima fila, un nivel mas arriba
    select m.id_medico, m.id_medico_superior, c.nivel + 1
    from medico m
    inner join cadena c on m.id_medico = c.id_medico_superior
)
select c.nivel, m.id_medico, m.nombres, m.apellidos, m.cargo,
       e.nombre as especialidad, h.nombre as hospital
from cadena c                                       -- la cadena ya armada
inner join medico m on m.id_medico = c.id_medico    -- nombre y cargo
inner join especialidad e on e.id_especialidad = m.id_especialidad   -- especialidad
inner join hospital h on h.id_hospital = m.id_hospital               -- hospital
order by c.nivel;                                   -- del medico hasta el director
```

sql/consultas/objetivo_17.sql

```
-- Objetivo 17: ranking de medicos por calificacion como el objetuvo 2
-- con su posicion en su hospital y en toda la cadena.
-- La posicion 1 es la nota mas baja.
-- Hay dos formas que dan lo mismo, para ver cual es mas rapida (ver rendimiento/).
-- lapso: enero a octubre de 2026

-- Forma A: funciones de ventana
with promedios as (                                     -- tabla temporal con el promedio de cada medico
    select m.id_medico,                                 -- id del medico
           m.nombres || ' ' || m.apellidos as medico,   -- nombre completo (|| junta los textos)
           h.nombre as hospital,                        -- su hospital
           avg(ca.puntuacion) as promedio               -- su nota promedio (de 1 a 5)
    from calificacion ca                                -- las notas que dieron los pacientes
    inner join consulta co on co.id_consulta = ca.id_consulta   -- la consulta que se califico
    inner join cita c on c.id_cita = co.id_cita                 -- su cita, para la fecha y el medico
    inner join medico m on m.id_medico = c.id_medico            -- el medico que atendio
    inner join hospital h on h.id_hospital = m.id_hospital      -- el hospital del medico
    where c.fecha_hora between '2026-01-01' and '2026-10-31 23:59:59'   -- solo el lapso
    group by m.id_medico, m.nombres, m.apellidos, h.nombre      -- una fila por medico
)
```

```
select medico,                                          -- nombre
       hospital,                                        -- hospital
       round(promedio, 2) as calificacion_promedio,     -- promedio con 2 decimales
       rank() over (partition by hospital order by promedio) as posicion_en_hospital,   -- puesto contando solo su hospital
       rank() over (order by promedio) as posicion_en_cadena                            -- puesto contando a todos los medicos
from promedios                                          -- sale de la tabla temporal
order by posicion_en_cadena, hospital, medico;          -- del peor al mejor

-- Forma B: subconsultas correlacionadas (la posicion es 1 + cuantos medicos tienen menor promedio)
with promedios as (                                     -- la misma tabla temporal de la forma A
    select m.id_medico, m.nombres || ' ' || m.apellidos as medico, h.nombre as hospital,   -- id, nombre y hospital
           avg(ca.puntuacion) as promedio               -- nota promedio
    from calificacion ca                                -- las notas
    inner join consulta co on co.id_consulta = ca.id_consulta   -- su consulta
    inner join cita c on c.id_cita = co.id_cita                 -- su cita
    inner join medico m on m.id_medico = c.id_medico            -- el medico
    inner join hospital h on h.id_hospital = m.id_hospital      -- su hospital
    where c.fecha_hora between '2026-01-01' and '2026-10-31 23:59:59'   -- solo el lapso
    group by m.id_medico, m.nombres, m.apellidos, h.nombre      -- una fila por medico
)
-- p1 = el medico al que se le busca el puesto, p2 = los demas medicos con los que se compara.
-- puesto = 1 + cuantos van delante (los que tienen menos nota), como el lugar en una fila:
-- si nadie va delante eres el 1, si van 2 delante eres el 3
select p1.medico,                                       -- el medico al que se le busca el puesto
       p1.hospital,                                     -- su hospital
       round(p1.promedio, 2) as calificacion_promedio,  -- su promedio con 2 decimales
       1 + (select count(*) from promedios p2           -- 1 + los que van delante = su puesto en el hospital
            where p2.hospital = p1.hospital             -- el where no ordena, elige a quien contar: solo los de su hospital
              and p2.promedio < p1.promedio) as posicion_en_hospital,   -- y que tengan menos nota que el
       1 + (select count(*) from promedios p2           -- 1 + los que van delante = su puesto en la cadena
            where p2.promedio < p1.promedio) as posicion_en_cadena      -- cuenta a todos los que tienen menos nota (de cualquier
hospital)
from promedios p1                                       -- se hace para cada medico de la lista
order by posicion_en_cadena, hospital, medico;          -- este si ordena: del puesto 1 (peor nota) al ultimo
```

sql/consultas/objetivo_18.sql

```
-- Objetivo 18: pacientes atendidos en TODAS las unidades medicas (Consulta Externa,
-- Emergencias, Cirugia y Hospitalizacion) para invitarlos al programa de fidelidad.
-- lapso: enero a octubre de 2026

select p.id_paciente,
       p.numero_expediente,
       p.nombres,
       p.apellidos,
       p.sexo
from paciente p
inner join ingreso i on i.id_paciente = p.id_paciente           -- sus fichas de ingreso (emergencias, cirugia, hospitalizacion)
inner join unidad_medica u on u.id_unidad = i.id_unidad         -- para saber en que unidad fue cada ingreso
where i.fecha_hora_ingreso between '2026-01-01' and '2026-10-31 23:59:59'
  and p.id_paciente in (select c.id_paciente                    -- y que tambien haya tenido una consulta (consulta externa)
                        from consulta co
                        inner join cita c on c.id_cita = co.id_cita
                        where c.fecha_hora between '2026-01-01' and '2026-10-31 23:59:59')
group by p.id_paciente, p.numero_expediente, p.nombres, p.apellidos, p.sexo   -- una fila por paciente
having count(distinct u.tipo_unidad) = 3                        -- que sus ingresos sean de las 3 unidades distintas
order by p.id_paciente;
```

sql/consultas/objetivo_19.sql

```
-- Proyecto 2 - Objetivo 19 (Bloque E)
-- Aqui reviso el reporte de cirugias que nos dieron y propongo como separarlo en tablas.
--
-- El reporte viene todo en una sola tabla:
--   Reporte_Cirugia (Id_cirugia, Nombre_cirujano, Especialidad_cirujano, Nombre_paciente,
--                    Codigo_diagnostico, Descripcion_diagnostico, Nombre_insumo_1,
--                    Cantidad_insumo_1, Nombre_insumo_2, Cantidad_insumo_2, Fecha)
--
-- Que problemas tiene (una frase cada uno):
--   * 1FN: insumo_1 e insumo_2 son un grupo repetitivo, asi que no caben mas de 2 insumos y si usa uno queda vacio el otro.
--   * 2FN: al poner un insumo por fila la llave pasa a ser (cirugia, insumo), pero el cirujano, paciente, diagnostico y fecha
solo dependen de la cirugia (media llave).
--   * 3FN: la especialidad depende del cirujano y la descripcion depende del codigo del diagnostico, no de la cirugia.
--   * Datos repetidos: la especialidad y la descripcion se vuelven a escribir en cada cirugia.
--   * Al actualizar: si un cirujano cambia de especialidad hay que cambiar todas sus filas, y si falta una queda contradictorio.
--   * Al insertar: no puedo guardar un cirujano, diagnostico o insumo nuevo hasta que salga en una cirugia.
--   * Al borrar: si borro la unica cirugia de un diagnostico tambien pierdo su descripcion.
--   * Identificador: usa nombres para identificar a las personas, y dos se pueden llamar igual.
--
-- Mi propueta en 3FN son las 7 tablas de abajo. Es solo la propuesta: no se corre
-- sobre la base del hospital porque ahi ya existen tablas con estos nombre.

-- las especialidades, cada una una sola vez
create table especialidad (
    id_especialidad integer generated always as identity primary key,
```

```
    nombre varchar(80) not null,
    constraint uq_especialidad_nombre unique (nombre)
);

-- cirujanos: la especialidad va aqui porque depende del cirujano, no de la cirugia
create table cirujano (
    id_cirujano integer generated always as identity primary key,
    nombres varchar(100) not null,
    apellidos varchar(100) not null,
    id_especialidad integer not null references especialidad(id_especialidad)
        on delete restrict on update cascade
);

-- los pacientes
create table paciente (
    id_paciente integer generated always as identity primary key,
    nombres varchar(100) not null,
    apellidos varchar(100) not null
);

-- diagnosticos: la descripcion va aqui porque depende del codigo, no de la cirugia
create table diagnostico (
    codigo_cie10 varchar(7) primary key,
    descripcion varchar(200) not null,
    -- el codigo CIE-10 tiene que medir de 3 a 7 (de K35 a K35.81)
    constraint ck_diagnostico_codigo check (length(codigo_cie10) between 3 and 7)
);

-- los insumos, cada uno una sola vez
create table insumo (
    id_insumo integer generated always as identity primary key,
    nombre varchar(120) not null,
    constraint uq_insumo_nombre unique (nombre)
);

-- cada cirugia: quien opera, a quien, por que diagnostico y cuando
create table cirugia (
    id_cirugia integer primary key,
    id_cirujano integer not null references cirujano(id_cirujano)
        on delete restrict on update cascade,
    id_paciente integer not null references paciente(id_paciente)
        on delete restrict on update cascade,
    codigo_cie10 varchar(7) not null references diagnostico(codigo_cie10)
        on delete restrict on update cascade,
    fecha date not null
);

-- los insumos de cada cirugia, ya sin limite de 2 (tabla intermedia N:M)
create table cirugia_insumo (
    id_cirugia integer not null references cirugia(id_cirugia)
        on delete cascade on update cascade,
    id_insumo integer not null references insumo(id_insumo)
        on delete restrict on update cascade,
    cantidad smallint not null,
    primary key (id_cirugia, id_insumo),
    -- por lo menos tiene que usar 1
    constraint ck_cirugia_insumo_cantidad check (cantidad > 0)
);
```

sql/consultas/prueba.sql

```
--consulta para obtener
--los 5 mejores medicos, de emergencias, basados en el numero de horas,

select m.nombres
, m.apellidos
, count(co.id_consulta) as horas
from cita c
inner join medico m on m.id_medico = c.id_medico
inner join consulta co on co.id_cita = c.id_cita
inner join
group by m.id_medico, m.nombres, m.apellidos
order by horas desc
limit 5;
```

sql/consultas/rendimiento/inflar.sql

```
-- Proyecto 2 - Objetivo 17, medicion de rendimiento
-- inflar.sql : agrega medicos, citas, consultas y calificaciones de prueba a
-- una COPIA de la base (nunca a la base entregada).
--
-- Variables de psql: :medicos y :calificaciones (una por consulta).
-- Los datos pasan por las mismas restricciones que los reales.

insert into personal (id_personal, tipo_personal, nombres, apellidos, dpi, direccion, id_municipio, area,
```

```
                      id_hospital, fecha_contratacion, salario, nivel_acceso)
select 100000 + k, 'medico', 'Médico', 'Prueba ' || k, '9' || lpad(k::text, 12, '0'), 'Dirección de prueba', 1, 'Urbana',
       1 + k % 5, date '2020-01-01', 10000.00, 'operativo'
  from generate_series(1, :medicos) k;

insert into medico (id_medico, nombres, apellidos, dpi, telefono, id_hospital, id_municipio, area,
                    id_especialidad, tipo_medico, costo_primera_consulta, horario_inicio, horario_fin)
select id_personal, nombres, apellidos, dpi, telefono, id_hospital, id_municipio, area,
       307, 'general', 150.00, '08:00', '12:00'
  from personal where id_personal > 100000;

insert into cita (id_cita, id_paciente, id_medico, fecha_hora, numero_clinica, medio_asignacion, estado, costo_base, costo_final)
select 1000000 + k, 1 + k % 67, 100001 + k % :medicos,
       timestamptz '2026-02-01 09:00-06' + (k % 240) * interval '1 day',
       'P-1', 'telefonico', 'realizada', 150.00, 150.00
  from generate_series(1, :calificaciones) k;

insert into consulta (id_consulta, id_cita, id_diagnostico)
select id_cita, id_cita, 18 from cita where id_cita > 1000000;

insert into calificacion (id_paciente, fecha, puntuacion, id_medico, id_consulta)
select ci.id_paciente, ci.fecha_hora::date, 1 + (ci.id_cita * 7 + ci.id_medico) % 5, ci.id_medico, ci.id_cita
  from cita ci where ci.id_cita > 1000000;

analyze;
```